EF CORE PLUGIN-ENTWICKLUNG · REFERENZ UND PRAXIS

Das EF Core Plugin-Handbuch

Von der Options-Extension bis zum NuGet-Paket: alles, was man für professionelle und auch sehr komplexe Erweiterungen von Entity Framework Core braucht. Durchgängig erklärt an einem vollständigen Plugin für Mandantenfähigkeit mit PostgreSQL und SQL Server.

Ausgabe	1.0
Versionen	EF Core 10, Npgsql.EntityFrameworkCore.PostgreSQL 10, .NET 10
Beispiel	Vollständige Solution unter **github.com/guenni81/EF-Core-Plugin-Handbook**: Kernpaket, zwei Provider-Pakete, zwei Beispiel-Apps, Unit- und Integrationstests
Geprüft	Der Beispielcode wird gegen EF Core 10 und Npgsql 10 gebaut; Unit- und Integrationstests laufen gegen echtes PostgreSQL und SQL Server.
Aufbau	9 Teile, 42 Kapitel, 4 Anhänge
Lizenz	MIT-Lizenz, © 2026 guenni81. Frei nutzbar, kopierbar, veränderbar und weitergebbar, auch kommerziell; siehe letzte Seite.

Über dieses Buch

Viele Erweiterungen für EF Core beginnen als ein paar Extension-Methods und wachsen dann unkontrolliert. Wer verstanden hat, wie EF Core intern aufgebaut ist, kann dagegen Erweiterungen schreiben, die sich wie ein Teil von EF Core anfühlen: Sie werden über die Options konfiguriert, laufen im internen Service Provider, übersetzen eigene Funktionen nach SQL, erzeugen Migrationen, schreiben lesbare Snapshots und lassen sich mit anderen Plugins kombinieren.

Dieses Buch erklärt jeden dieser Bausteine in zwei Schichten. Zuerst kommt das **Prinzip**: Wie funktioniert der Mechanismus in EF Core, welche Varianten gibt es, worauf muss man achten? Danach folgt die **Umsetzung** im Beispiel-Plugin, mit dem Code, der auch im Repository steht. Die Kapitel bauen aufeinander auf, lassen sich aber auch einzeln als Nachschlagewerk lesen. Die Codeauszüge dieser deutschen Fassung haben deutsche Kommentare; im Repository sind Kommentare und Meldungen englisch, der Code ist sonst identisch. Das Label jedes Auszugs nennt die Datei.

Kasten	Bedeutung
Wichtig	Fehlerquellen, die zu falschem Verhalten oder Sicherheitslücken führen.
Tipp	Bewährte Praxis, die Arbeit spart oder die Qualität erhöht.
Hinweis	Hintergrund, Einordnung und Verweise auf andere Kapitel.

> **Öffentliche und interne APIs**
>
> Einige APIs, die Plugins brauchen, liegen in Namespaces mit `.Internal`. EF Core nennt sie „pubternal“: öffentlich sichtbar, aber ohne Kompatibilitätsgarantie. Das Buch kennzeichnet diese Stellen und erklärt in Kapitel 24, wie man damit umgeht.

Inhaltsverzeichnis

I Grundlagen 5

1 Was ist ein EF-Core-Plugin? 6

2 Die Architektur von EF Core aus Plugin-Sicht 7

3 Das Beispielszenario 11

II Infrastruktur 13

4 Die Options-Extension 14

5 Dienste registrieren 17

6 Öffentliche API und Integration in die Anwendung 21

III Das Modell 24

7 Annotations und Fluent API 25

8 Conventions 28

9 Modellvalidierung 33

10 Type Mappings und Value Converter 34

IV Abfragen 37

11 Die Query-Pipeline 38

12 Methoden und Member übersetzen 39

13 Clientseitige Auswertung verhindern 43

14 Weitere Erweiterungspunkte der Abfrage 44

V Laufzeit 45

15 Interceptors 46

16 Diagnose und Logging 51

VI Migrations und Design-Time 53

17 Wie Migrations funktionieren 54

18 Deklaratives SQL aus Annotations 55

19 Eigene Migrations-Operationen 58

20 Design-Time-Services 60

21 Compiled Models, Trimming und NativeAOT 62

22 Scaffolding (Reverse Engineering) 63

VII Qualität und Auslieferung 64

23 Teststrategie 65

24 Versionierung und Kompatibilität 69

25 Packaging und Dokumentation 71

26 Die Checkliste von A bis Z 72

VIII Fortgeschrittene Themen 75

27 Massenoperationen und die Grenzen von Interceptors 76

28 Value Generation 77

29 Die Update-Pipeline 79

30 Complex Types und JSON 80

31 Modelle pro Laufzeitkontext 81

32 Migrations für Fortgeschrittene 82

33 Row Level Security in Produktion 84

34 Performance von Plugins 86

35 Einsatz in ASP.NET Core 88

36 CI und API-Stabilität 91

IX Tiefe Erweiterungen 93

37 Eigene SQL-Ausdrucksknoten 94

38 Aggregat-Translators 99

39 Fluent API auf Property-Ebene und Laufzeit-Annotations 101

40 Scaffolding in der Praxis 104

41 Was in den Kern gehört 106

42 Ein zweiter Provider: SQL Server 107

X Anhänge 110

A Erweiterungspunkte und Lebensdauern 111

B Dateien des Beispiel-Plugins 112

C Glossar 114

D Debugging-Werkzeugkasten 115

Lizenz 117

Teil I

Grundlagen

Was ein Plugin ist, wie EF Core intern aufgebaut ist und welches Szenario uns durch das Buch begleitet.

- Kapitel 1: Was ist ein EF-Core-Plugin?
- Kapitel 2: Die Architektur von EF Core aus Plugin-Sicht
- Kapitel 3: Das Beispielszenario

1 Was ist ein EF-Core-Plugin?

Der Begriff „Plugin“ wird für sehr unterschiedliche Dinge verwendet. Für dieses Buch hilft eine Einteilung in vier Stufen, die sich darin unterscheiden, wie tief sie in EF Core eingreifen:

Stufe	Beschreibung	Beispiele
1 · Hilfsmethoden	Extension-Methods auf `DbContext` oder `IQueryable`, die nur öffentliche API aufrufen.	Paging-Helfer, `WhereIf`, Seed-Methoden
2 · Modell-Helfer	Fluent-API-Erweiterungen und Annotations, ohne eigene Dienste in EF Core.	Eigene `HasXyz()`-Methoden, ModelBuilder-Helfer
3 · Echte Plugins	Eigene Dienste im internen Service Provider: Conventions, Query-Übersetzung, Type Mappings, Migrations, Design-Time.	NetTopologySuite-Plugin, NodaTime-Plugin, EFCore.NamingConventions
4 · Provider	Komplette Datenbankanbindung inklusive Verbindung, Update-Pipeline und SQL-Generierung.	Npgsql, Pomelo (MySQL), SQLite, Cosmos

Dieses Buch behandelt **Stufe 3** vollständig. Stufen 1 und 2 brauchen kein eigenes Buch: Sie nutzen nur die öffentliche API von EF, und Kapitel 7 behandelt, was an Fluent-API-Erweiterungen speziell ist. Die Werkzeuge von Stufe 4 (etwa der Austausch des SQL-Generators) werden dort erklärt, wo Plugins sie brauchen.

Für wen dieses Buch ist

Für Entwickler, die EF Core bereits sicher einsetzen: `DbContext`, Migrationen, LINQ-Abfragen und Dependency Injection werden vorausgesetzt. Kenntnisse der Interna von EF sind nicht nötig; das Buch führt sie dort ein, wo sie gebraucht werden. Erfahrung mit PostgreSQL hilft beim Beispiel, aber jedes SQL-Konstrukt wird beim ersten Auftreten erklärt. Kapitel 2 endet mit einem vollständigen Mini-Plugin, das alle Bausteine auf einer Seite zeigt, bevor das große Beispiel beginnt.

Was ein Plugin der Stufe 3 leisten kann

Ein vollständiges Plugin kann EF Core an praktisch jeder Stelle erweitern: Es kann neue CLR-Typen als Spaltentypen verfügbar machen, eigene Funktionen in LINQ nach SQL übersetzen, das Modell automatisch per Convention anpassen, Metadaten über die Fluent API und Attribute annehmen, das Modell validieren, beim Speichern und beim Öffnen von Verbindungen eingreifen, eigene Ereignisse protokollieren, Migrationen um eigenes SQL ergänzen und die Werkzeuge von `dotnet ef` erweitern. Das Beispiel in diesem Buch nutzt jede dieser Möglichkeiten.

> **Vorbilder im Ökosystem**
>
> Bevor man selbst schreibt, lohnt der Blick in bestehende Plugins. **EFCore.NamingConventions** ist ein kleines, sauberes Convention-Plugin. Das **NetTopologySuite**-Plugin im EF-Core-Repository zeigt Type Mappings und Übersetzungen. Das **NodaTime**-Plugin von Npgsql zeigt ein provider-spezifisches Plugin, das sich über den Options-Builder des Providers aktivieren lässt.

2 Die Architektur von EF Core aus Plugin-Sicht

Zwei Container: der interne Service Provider

Jeder `DbContext` löst seine Infrastruktur aus einem **internen** `IServiceProvider` auf. Er ist vom DI-Container der Anwendung getrennt und enthält mehrere hundert Dienste: Type Mapping, Model Building, Query-Pipeline, Change Tracking, Update-Pipeline, Migrations. Ein Plugin klinkt sich ein, indem es in diesen Container zusätzliche Dienste registriert oder vorhandene ersetzt.



Zwei Caches sind dabei entscheidend. Der **Service Provider** wird anhand der Options gecacht: Contexts mit gleicher Konfiguration teilen sich einen Provider. Das **Modell** wird pro Provider (und Context-Typ) gecacht. Beides macht EF Core schnell, hat aber direkte Folgen für Plugins, die Kapitel 4 ausführlich behandelt.

Lebensdauern

Jeder EF-Dienst hat eine feste Lebensdauer. **Singleton** heißt hier: einmal pro internem Service Provider, nicht pro Anwendung. **Scoped** heißt: einmal pro Context-Instanz. Ein Singleton darf keinen Scoped-Dienst halten, und ein Dienst, der pro Context unterschiedliche Daten braucht, darf kein Singleton sein. Anhang A listet die Lebensdauern aller Erweiterungspunkte.

Das Dependencies-Muster

EF-Dienste bekommen ihre Abhängigkeiten nicht einzeln, sondern als ein Parameterobjekt, etwa `MigrationsSqlGeneratorDependencies` oder `ProviderConventionSetBuilderDependencies`. Wer von einem EF-Dienst erbt, übernimmt dieses Objekt im Konstruktor und reicht es weiter. Neue Abhängigkeiten fügt EF Core dem Objekt hinzu, ohne Konstruktoren zu brechen. Für Plugins ist das ein Segen: Abgeleitete Dienste bleiben über Minor-Versionen kompatibel.

`Migrations/Internal/TenancyMigrationsSqlGenerator.cs (Kopf)`

```
public class TenancyMigrationsSqlGenerator : NpgsqlMigrationsSqlGenerator
{
    public TenancyMigrationsSqlGenerator(
        MigrationsSqlGeneratorDependencies dependencies,     // Parameterobjekt von EF
        INpgsqlSingletonOptions npgsqlSingletonOptions)      // zusätzlich vom Provider
        : base(dependencies, npgsqlSingletonOptions)
    {
    }
```

Code-Stil im Beispiel

Alle Klassen im Beispiel verwenden klassische Konstruktoren mit `private readonly`-Feldern und `ArgumentNullException.ThrowIfNull`, keine Primary Constructors. Deren Parameter sind im ganzen Klassenkörper veränderbar, bieten keinen natürlichen Ort für Validierung und verbergen, welcher Zustand gespeichert wird. Klassische Konstruktoren entsprechen außerdem dem Stil von EF Core und Npgsql, von deren Klassen ein Plugin erbt. Einzige Ausnahme sind positionelle Records wie `RlsPolicy`: Dort erzeugt die Syntax echte `init`-Properties.

Erweitern oder ersetzen?

Für Erweiterungen gibt es zwei grundverschiedene Wege. **Plugin-Interfaces** wie `IMethodCallTranslatorPlugin` oder `IConventionSetPlugin` sind ausdrücklich dafür gedacht. Sie sind mehrfach registrierbar, EF fragt alle Registrierungen ab, und mehrere Plugins vertragen sich. **Ersetzen** bedeutet, einen Einzeldienst wie `IMigrationsSqlGenerator` durch eine abgeleitete Klasse auszutauschen. Das ist manchmal unvermeidlich, aber es kann pro Context immer nur einer gewinnen.

Plugin-Interfaces vor Ersetzen

Immer zuerst prüfen, ob es ein Plugin-Interface gibt. Wer einen Einzeldienst ersetzt, kollidiert mit jedem anderen Plugin, das denselben Dienst ersetzt, und muss von der Provider-Implementierung erben. Das bindet das Plugin an einen Provider und an dessen interne API. Kapitel 5 zeigt, wie man trotzdem kombinierbar bleibt.

Zum Aufwärmen: ein vollständiges Plugin in fünf Teilen

Vor dem großen Beispiel kommt hier das kleinste Plugin, das trotzdem vollständig ist. Es macht PostgreSQLs `reverse()` in LINQ als `EF.Functions.Reverse(...)` verfügbar. Jedes spätere Kapitel vertieft einen seiner fünf Teile:

`tests/Acme.EntityFrameworkCore.Tenancy.Tests/MinimalPluginTests.cs (1/2, gekürzt)`

```
// 1. Die Funktion, die Anwender in LINQ aufrufen; sie wird nie ausgeführt
public static class ReverseDbFunctionsExtensions
{
    public static string Reverse(this DbFunctions _, string value)
        => throw new InvalidOperationException("Reverse can only be used in LINQ queries.");
}

// 2. Der Translator macht aus dem Methodenaufruf eine SQL-Funktion
//    (Kapitel „Methoden und Member übersetzen“)
public class ReverseTranslator : IMethodCallTranslator
{
```

```
    // Konstruktor speichert die ISqlExpressionFactory in _sql

    public SqlExpression? Translate(
        SqlExpression? instance, MethodInfo method, IReadOnlyList<SqlExpression> arguments,
        IDiagnosticsLogger<DbLoggerCategory.Query> logger)
        => method == ReverseMethod
            ? _sql.Function("reverse", [arguments[1]], nullable: true,
                argumentsPropagateNullability: [true], typeof(string),
                arguments[1].TypeMapping)
            : null;
}

// 3. Das Plugin übergibt den Translator an EF (Kapitel „Methoden und Member übersetzen“)
public class ReverseTranslatorPlugin : IMethodCallTranslatorPlugin
{
    public ReverseTranslatorPlugin(ISqlExpressionFactory sql)
    {
        Translators = [new ReverseTranslator(sql)];
    }

    public IEnumerable<IMethodCallTranslator> Translators { get; }
}
```

tests/Acme.EntityFrameworkCore.Tenancy.Tests/MinimalPluginTests.cs (2/2, gekürzt)

```
// 4. Die Options-Extension registriert das Plugin im internen Provider
//    (Kapitel „Die Options-Extension“ und „Dienste registrieren“)
public class ReverseOptionsExtension : IDbContextOptionsExtension
{
    private DbContextOptionsExtensionInfo? _info;

    public DbContextOptionsExtensionInfo Info => _info ??= new ExtensionInfo(this);

    public void ApplyServices(IServiceCollection services)
        => new EntityFrameworkRelationalServicesBuilder(services)
            .TryAdd<IMethodCallTranslatorPlugin, ReverseTranslatorPlugin>();

    public void Validate(IDbContextOptions options)
    {
    }

    // ExtensionInfo: kein Provider, keine Settings -> Hash 0, immer derselbe Provider
}

// 5. Die Use-Methode aktiviert das Plugin
//    (Kapitel „Öffentliche API und Integration in die Anwendung“)
public static DbContextOptionsBuilder UseReverse(this DbContextOptionsBuilder builder)
{
    ((IDbContextOptionsBuilderInfrastructure)builder).AddOrUpdateExtension(
        builder.Options.FindExtension<ReverseOptionsExtension>()
        ?? new ReverseOptionsExtension());
    return builder;
}
```

Anwendung

```
options.UseNpgsql(connectionString).UseReverse();

db.Customers.Where(c => EF.Functions.Reverse(c.Name) == "abc");
// SQL: WHERE reverse(c."Name") = 'abc'
```

Mehr braucht ein funktionierendes Plugin nicht. EF findet den Translator, weil die Extension ihn im internen Service Provider registriert, und alle Contexts, die mit `UseReverse()` konfiguriert sind, teilen sich diesen Provider. Weil das Argument auf eine Spalte verweist, braucht die Funktion keinen Evaluatable-Expression-Filter (Kapitel 13 erklärt, wann sie einen braucht). Der Test `MinimalPluginTests` prüft die Übersetzung und dass dieselbe Abfrage ohne das Plugin fehlschlägt. Das Mandanten-Plugin besteht aus genau diesen Teilen, nur aus sehr viel mehr davon.

3 Das Beispielszenario

Das Beispiel-Plugin **Acme.EntityFrameworkCore.Tenancy** macht EF Core mit PostgreSQL mandantenfähig. Die Trennung der Mandanten erzwingt die Datenbank selbst über **Row Level Security** (RLS): Jede Tabelle bekommt eine Policy, die nur Zeilen des Mandanten zeigt, dessen ID in der aktuellen Datenbank-Session gesetzt ist. Selbst eine fehlerhafte Abfrage oder rohes SQL kann so keine fremden Daten lesen.

Aus Sicht des Anwenders sieht das so aus:

Anwendersicht

```
public class Order : ITenantOwned               // Marker aus dem Kernpaket
{
    public int Id { get; set; }
    public TenantId TenantId { get; set; }      // stark typisiert, automatisch uuid
    public decimal Total { get; set; }
}

services.AddAcmeTenancy<HttpTenantProvider>();
services.AddDbContext<ShopContext>((sp, o) => o
    .UseNpgsql(connectionString)
    .UseAcmeTenancy(sp, tenancy => tenancy.ApplyQueryFilter()));

// Abfragen und Speichern ohne jede Mandantenlogik im Anwendungscode
db.Orders.Add(new Order { Total = 42 });         // TenantId setzt das Plugin
var mine = await db.Orders.ToListAsync();        // RLS filtert in der Datenbank
```

Damit das funktioniert, nutzt das Plugin jeden Erweiterungspunkt dieses Buchs:

Anforderung	Umsetzung	Kap.
Konfiguration über `UseAcmeTenancy()`	Options-Extension, Settings-Record, Options-Builder	4–6
`TenantId` als Spaltentyp ohne Konfiguration	Type-Mapping-Plugin, Value Converter, JSON-Reader/Writer	10
`EF.Functions.CurrentTenant()` in LINQ	Method-Translator, Evaluatable-Expression-Filter	12–13
`tenantId.IsEmpty` in LINQ	Member-Translator	12
RLS für alle `ITenantOwned`-Entitäten	Convention mit Index und optionalem Query-Filter	8
Fluent API, Attribut `[NoTenantIsolation]`	Annotations, Attribut-Convention, ConfigurationSource	7–8
Fehlkonfiguration früh erkennen	Validierungs-Convention, eigene Warnungen	9, 16
TenantId automatisch setzen	SaveChanges-Interceptor	15
Mandant in der DB-Session	Connection-Interceptor	15
`ENABLE ROW LEVEL SECURITY`, `CREATE POLICY`	Annotation-Provider, Migrations-SQL-Generator	17–18
Rollen und Rechte in Migrationen	Eigene Migrations-Operationen	19
Lesbare Snapshots	Annotation-Code-Generator, Design-Time-Services	20
Compiled Models und AOT	`Default`-Instanzen, Source-Generatoren	21
`EF.Functions.IsNotDistinctFrom`, `EF.Functions.Every`	Eigener SQL-Knoten, Aggregat-Translator	37–38

Anforderung	Umsetzung	Kap.
Mandanten-Property mit beliebigem Namen	`IsTenantProperty()`, Laufzeit-Annotation	39
RLS aus bestehender Datenbank scaffolden	Eigene Database-Model-Factory	40
SQL Server	Zweites Provider-Paket auf demselben Kern	41–42

Paketarchitektur

Das Plugin besteht aus drei NuGet-Paketen. Das **Kernpaket** enthält alles, was unabhängig von der Datenbank ist und was auch Assemblies mit Entitätsklassen referenzieren dürfen, ohne einen Provider mitzuziehen. Das **PostgreSQL-Paket** ist die vollständige Implementierung, an der dieses Buch alles erklärt. Das **SQL-Server-Paket** nutzt denselben Kern und zeigt in Kapitel 42, was ein zweiter Provider selbst umsetzen muss.

Struktur der drei Pakete

```
Acme.EntityFrameworkCore.Tenancy              Kern (EF Core relational, ohne Provider)
├─ TenantId, TenantIdConverter, TenantIdJsonReaderWriter, TenancyException
├─ ITenantOwned, ITenantProvider, NoTenantIsolationAttribute
├─ EF.Functions.CurrentTenant(), TenancyDbFunctionsAttribute
├─ Metadata/         Laufzeit-Annotation, IsTenantProperty(), Kandidatensuche
├─ Conventions/      Value Generation für TenantId-Schlüssel
├─ Query/            Evaluatable-Expression-Filter (für alle Pakete)
├─ Interceptors/     SaveChanges-Interceptor, abstrakter Session-Interceptor
├─ Diagnostics/      Event-IDs, Logging-Definitionen, Logger-Extensions
└─ DependencyInjection/  AddAcmeTenancy<T>()

Acme.EntityFrameworkCore.Tenancy.PostgreSQL    vollständige Implementierung
├─ Infrastructure/   Options-Extension, Builder, Singleton-Options
├─ Storage/          TenantIdTypeMapping, Type-Mapping-Plugin
├─ Query/            Translators, eigener SQL-Knoten, Aggregat, SQL-Generator
├─ Metadata/         RLS-Annotations, RlsPolicy, Annotation-Provider
├─ Builders/         Fluent API (HasRowLevelSecurity, HasPolicy, ...)
├─ Conventions/      Isolation, Attribut, Validierung, Convention-Set-Plugin
├─ Migrations/       SQL-Generator, eigene Operationen
├─ Design/           Code-Generatoren, Design-Time-Services
├─ Scaffolding/      RLS und Policies aus der Datenbank lesen
└─ Runtime/          TenantConnectionInterceptor

Acme.EntityFrameworkCore.Tenancy.SqlServer     zweiter Provider
                                               (Kapitel „Ein zweiter Provider: SQL Server“)
└─ Infrastructure/, Storage/, Query/, Metadata/, Conventions/, Migrations/, Runtime/
```

> **Tipp**
>
> Die Ordnerstruktur folgt der von EF Core selbst (Infrastructure, Storage, Query, Metadata, Migrations, Design). Wer EF Core kennt, findet sich sofort zurecht, und beim Nachschlagen im EF-Quellcode weiß man, wo das Gegenstück liegt.

Teil II

Infrastruktur

Wie ein Plugin konfiguriert wird, seine Dienste registriert und sich mit dem DI-Container der Anwendung verbindet.

- Kapitel 4: Die Options-Extension
- Kapitel 5: Dienste registrieren
- Kapitel 6: Öffentliche API und Integration in die Anwendung

4 Die Options-Extension

Der Einstiegspunkt jedes echten Plugins ist eine Klasse, die `IDbContextOptionsExtension` implementiert. Sie hat drei Aufgaben: Sie trägt die Konfiguration, sie registriert in `ApplyServices` die Dienste im internen Provider, und sie beschreibt sich über `Info` gegenüber dem Cache.

Konfiguration als Record

Alle modellrelevanten Einstellungen des Beispiels liegen in einem Record. Records liefern `Equals` und `GetHashCode` über alle Felder, was gleich für den Cache-Schlüssel gebraucht wird. Neue Einstellungen fügt man dem Record hinzu, ohne Hash und Vergleich anfassen zu müssen.

Infrastructure/TenancySettings.cs

```
public sealed record TenancySettings
{
    public static TenancySettings Default { get; } = new();

    public string SettingName { get; init; } = "app.tenant_id";
    public string PolicyName { get; init; } = "tenant_isolation";
    public bool ForceRowLevelSecurity { get; init; } = true;
    public bool ApplyQueryFilter { get; init; }
    public bool CreateTenantIndex { get; init; } = true;
}
```

Immutabilität

EF vergleicht und cached Options. Eine Extension darf sich nach dem Hinzufügen deshalb nie mehr ändern; jede Änderung erzeugt eine Kopie. Das Muster aus EF Core selbst: ein geschützter Kopierkonstruktor und eine virtuelle `Clone`-Methode, damit abgeleitete Extensions ihren eigenen Typ klonen.

Infrastructure/TenancyOptionsExtension.cs (Auszug)

```
public class TenancyOptionsExtension : IDbContextOptionsExtension
{
    private DbContextOptionsExtensionInfo? _info;

    public TenancyOptionsExtension()
    {
    }

    protected TenancyOptionsExtension(TenancyOptionsExtension copyFrom)
    {
        ArgumentNullException.ThrowIfNull(copyFrom);

        Settings = copyFrom.Settings;
    }

    public virtual TenancySettings Settings { get; private set; } = TenancySettings.Default;

    public virtual DbContextOptionsExtensionInfo Info => _info ??= new ExtensionInfo(this);
```

```
    protected virtual TenancyOptionsExtension Clone() => new(this);

    public virtual TenancyOptionsExtension WithSettings(TenancySettings settings)
    {
        var clone = Clone();
        clone.Settings = settings;
        return clone;
    }
```

Die Info-Klasse und der Service-Provider-Cache

Die Info-Klasse entscheidet, ob EF einen gecachten internen Service Provider wiederverwenden darf. Dafür gibt es zwei Methoden, die zusammenpassen müssen: `GetServiceProviderHashCode` als schneller Vorfilter und `ShouldUseSameServiceProvider` als exakter Vergleich.

Infrastructure/TenancyOptionsExtension.cs › ExtensionInfo

```
private sealed class ExtensionInfo : DbContextOptionsExtensionInfo
{
    public ExtensionInfo(TenancyOptionsExtension extension)
        : base(extension)
    {
    }

    private new TenancyOptionsExtension Extension => (TenancyOptionsExtension)base.Extension;

    public override bool IsDatabaseProvider => false;

    public override string LogFragment
        => $"AcmeTenancy(Setting={Extension.Settings.SettingName}) ";

    public override int GetServiceProviderHashCode() => Extension.Settings.GetHashCode();

    public override bool ShouldUseSameServiceProvider(DbContextOptionsExtensionInfo other)
        => other is ExtensionInfo o && o.Extension.Settings == Extension.Settings;

    public override void PopulateDebugInfo(IDictionary<string, string> debugInfo)
    {
        debugInfo["Acme:Tenancy:Force"] =
            Extension.Settings.ForceRowLevelSecurity.ToString();
        // ... weitere Einstellungen
    }
}
```

Bei diesen beiden Methoden gibt es zwei klassische Fehler mit gegensätzlichen Folgen:

Fehler	Folge
Modellrelevante Option fehlt im Hash	Zwei Contexts mit unterschiedlicher Konfiguration teilen sich Provider und Modell. Der zweite bekommt das gecachte, **falsche** Modell. Fällt im Test oft nicht auf, weil dort nur eine Konfiguration läuft.
Pro Context variierender Wert im Hash (Mandant, Connection-String, Instanz)	Für jeden Context entsteht ein neuer interner Provider samt Modell. Speicher und Startzeit explodieren; EF warnt nach 20 Providern mit `ManyServiceProvidersCreatedWarning`.

> **Die Faustregel für den Hash**
>
> In den Hash gehört genau das, was **beeinflusst, welche Dienste registriert werden oder wie das Modell aussieht**. Alles andere gehört nicht hinein. Der Mandant selbst ändert sich pro Request und darf deshalb nie in den Options landen; er kommt zur Laufzeit über einen Interceptor (Kapitel 15).

`LogFragment` erscheint in EFs Startmeldung („initialized using provider ... with options: ...“) und hilft bei der Fehlersuche. `PopulateDebugInfo` liefert Werte für EFs Diagnose, wenn zu viele Service Provider entstehen. `IsDatabaseProvider` ist nur für echte Provider `true`.

Validierung

`Validate` wird beim Erstellen des Service Providers aufgerufen. Hier prüft das Plugin Voraussetzungen, die sich aus der Kombination von Extensions ergeben, und meldet sie mit einer verständlichen Fehlermeldung statt eines späteren Folgefehlers:

Infrastructure/TenancyOptionsExtension.cs › Validate

```
public virtual void Validate(IDbContextOptions options)
{
    if (options.FindExtension<NpgsqlOptionsExtension>() is null)
        throw new InvalidOperationException(
            "UseAcmeTenancy() erfordert UseNpgsql(). Für andere Datenbanken wird ein "
            + "eigenes Provider-Paket benötigt.");
}
```

5 Dienste registrieren

`ApplyServices` füllt den internen Service Provider. EF stellt dafür einen Builder bereit, der die Lebensdauer und Kardinalität jedes EF-Dienstes kennt: `EntityFrameworkServicesBuilder` für Kerndienste und `EntityFrameworkRelationalServicesBuilder` zusätzlich für relationale Dienste. Er registriert jeden Dienst mit der richtigen Lebensdauer und nutzt für mehrfach registrierbare Dienste automatisch `TryAddEnumerable`.

Infrastructure/TenancyOptionsExtension.cs › ApplyServices

```
public virtual void ApplyServices(IServiceCollection services)
{
    // 1) Plugin-Interfaces: mehrfach registrierbar, kombinierbar mit anderen Plugins
    new EntityFrameworkRelationalServicesBuilder(services)
        .TryAdd<IRelationalTypeMappingSourcePlugin, TenancyTypeMappingSourcePlugin>()
        .TryAdd<IMethodCallTranslatorPlugin, TenancyMethodCallTranslatorPlugin>()
        .TryAdd<IMemberTranslatorPlugin, TenancyMemberTranslatorPlugin>()
        .TryAdd<IEvaluatableExpressionFilterPlugin,
                TenancyEvaluatableExpressionFilterPlugin>()
        .TryAdd<IConventionSetPlugin, TenancyConventionSetPlugin>()
        .TryAdd<ISingletonOptions, ITenancySingletonOptions>(
            p => p.GetRequiredService<ITenancySingletonOptions>())
        // 2) Eigene Dienste, die EF nicht kennt
        .TryAddProviderSpecificServices(m => m
            .TryAddSingleton<ITenancySingletonOptions, TenancySingletonOptions>()
            .TryAddSingleton<TenancyLoggingDefinitions, TenancyLoggingDefinitions>());

    // 3) Einzeldienste des Providers ersetzen
    services.Replace(ServiceDescriptor
        .Scoped<IMigrationsSqlGenerator, TenancyMigrationsSqlGenerator>());
    services.Replace(ServiceDescriptor
        .Singleton<IRelationalAnnotationProvider, TenancyAnnotationProvider>());
}
```

Methode	Wann	Warum
`TryAdd<TService, TImpl>()`	EF-Dienste und Plugin-Interfaces	Lebensdauer und Mehrfachregistrierung kommen aus EFs Tabelle. Falsche Lebensdauern führen sonst zu Exceptions oder subtil falschem Verhalten.
`TryAddProviderSpecificServices`	Eigene Dienste, die EF nicht kennt	Der Builder lehnt unbekannte Typen ab; hier legt man die Lebensdauer selbst fest.
`services.Replace(...)`	Einzeldienst des Providers ersetzen	Der Provider registriert per `TryAdd`. `Replace` gewinnt unabhängig davon, ob das Plugin vor oder nach dem Provider läuft.

> **Warum Replace in beiden Reihenfolgen funktioniert**
>
> EF ruft `ApplyServices` der Extensions in der Reihenfolge auf, in der sie hinzugefügt wurden. Läuft das Plugin zuerst, fügt `Replace` den Dienst hinzu und das spätere `TryAdd` des Providers greift nicht mehr. Läuft es danach, ersetzt `Replace` den Provider-Dienst. Beide Reihenfolgen führen zum selben Ergebnis.

Singleton-Options

Singleton-Dienste wie Type-Mapping-Plugins können `IDbContextOptions` nicht pro Context auflösen. Für sie gibt es `ISingletonOptions`: ein Singleton, das EF einmal beim Erstellen des Providers mit den Options initialisiert und bei jedem weiteren Context validiert.

Infrastructure/ITenancySingletonOptions.cs

```
public class TenancySingletonOptions : ITenancySingletonOptions
{
    public TenancySettings Settings { get; private set; } = TenancySettings.Default;

    // Einmal pro internem Service Provider
    public virtual void Initialize(IDbContextOptions options)
        => Settings = options.FindExtension<TenancyOptionsExtension>()?.Settings
                      ?? TenancySettings.Default;

    // Bei jedem Context: Optionen dürfen nicht vom Stand des Providers abweichen
    public virtual void Validate(IDbContextOptions options)
    {
        var settings = options.FindExtension<TenancyOptionsExtension>()?.Settings
                       ?? TenancySettings.Default;
        if (settings != Settings)
            throw new InvalidOperationException("Tenancy-Optionen nachträglich geändert ...");
    }
}
```

Die doppelte Registrierung in `ApplyServices` ist nötig: einmal unter dem eigenen Interface, damit Dienste es injizieren können, und einmal als `ISingletonOptions`, damit EF es findet und initialisiert. Die Factory-Variante sorgt dafür, dass beide Registrierungen *dieselbe* Instanz liefern.

Kombinierbarkeit trotz Ersetzen

Ersetzt ein Plugin einen Einzeldienst, kann ein zweites Plugin, das dasselbe tut, nicht gleichzeitig aktiv sein. Professionelle Plugins entschärfen das, indem sie die eigentliche Logik zusätzlich öffentlich und wiederverwendbar anbieten. Im Beispiel ist `CreatePolicySql` öffentlich und alle Generator-Methoden sind virtuell. Ein Nutzer, der zwei Plugins kombinieren muss, schreibt einen eigenen Generator, der von einem erbt und die Logik des anderen aufruft. Diese Möglichkeit gehört in die Dokumentation.

Noch besser vermeidet ein Plugin den Konflikt ganz, indem es den Dienst **dekoriert** statt ersetzt: Es nimmt die Implementierung, die zu diesem Zeitpunkt registriert ist (die des Providers oder eines anderen Plugins), umhüllt sie und delegiert an sie. Ein Beispiel ist ein (fiktives) Audit-Plugin, das an jede neue Tabelle einen Trigger hängt. In seinem `ApplyServices` sucht es die vorhandene Registrierung und ersetzt nur den Deskriptor, nicht die Logik:

tests/Acme.EntityFrameworkCore.Tenancy.Tests/CompositionTests.cs › AuditOptionsExtension

```
public void ApplyServices(IServiceCollection services)
{
    // Bisher registriert: der Generator des Providers oder eines anderen Plugins
    var inner = services.LastOrDefault(d => d.ServiceType == typeof(IMigrationsSqlGenerator))
        ?? throw new InvalidOperationException("Call UseContosoAudit() after UseNpgsql().");

    services.Replace(ServiceDescriptor.Describe(
```

```
        typeof(IMigrationsSqlGenerator),
        sp => new AuditMigrationsSqlGenerator(
            (IMigrationsSqlGenerator)CreateInstance(sp, inner),
            sp.GetRequiredService<MigrationsSqlGeneratorDependencies>()),
        inner.Lifetime));     // Lebensdauer des Originals
}

// Erzeugt die Implementierung so, wie es der ursprüngliche Deskriptor getan hätte
private static object CreateInstance(IServiceProvider sp, ServiceDescriptor descriptor)
    => descriptor.ImplementationInstance
       ?? descriptor.ImplementationFactory?.Invoke(sp)
       ?? ActivatorUtilities.CreateInstance(sp, descriptor.ImplementationType!);
```

Der Trick funktioniert, weil EF `ApplyServices` in der Reihenfolge aufruft, in der die Extensions hinzugefügt wurden; wenn das Audit-Plugin läuft, hat der Provider seinen Generator bereits registriert. `CreateInstance` bildet die ursprüngliche Registrierung exakt nach: eine Instanz, eine Factory oder einen Typ, dessen Abhängigkeiten (etwa Npgsqls Singleton-Options) `ActivatorUtilities` aus dem Container auflöst. Der Decorator selbst nutzt nur die öffentliche Schnittstelle:

tests/.../CompositionTests.cs › AuditMigrationsSqlGenerator (gekürzt)

```
public class AuditMigrationsSqlGenerator : IMigrationsSqlGenerator
{
    // Konstruktor: (IMigrationsSqlGenerator inner, MigrationsSqlGeneratorDependencies deps)

    public IReadOnlyList<MigrationCommand> Generate(
        IReadOnlyList<MigrationOperation> operations,
        IModel? model = null,
        MigrationsSqlGenerationOptions options = MigrationsSqlGenerationOptions.Default)
    {
        var commands = _inner.Generate(operations, model, options).ToList();   // unverändert
        var builder = new MigrationCommandListBuilder(_dependencies);
        var sql = _dependencies.SqlGenerationHelper;

        // Am Ende angehängt: dann existieren alle Tabellen der Migration
        foreach (var table in operations.OfType<CreateTableOperation>())
        {
            builder.Append("CREATE TRIGGER audit_changes ")
                .Append("AFTER INSERT OR UPDATE OR DELETE ON ")
                .Append(sql.DelimitIdentifier(table.Name, table.Schema))
                .Append(" FOR EACH ROW EXECUTE FUNCTION audit.log_change()")
                .AppendLine(sql.StatementTerminator)
                .EndCommand();
        }

        commands.AddRange(builder.GetCommandList());
        return commands;
    }
}
```

In der Anwendung lassen sich beide Plugins ohne eine Zeile Verbindungscode kombinieren:

Program.cs

```
services.AddDbContext<ShopContext>((sp, options) => options
    .UseNpgsql(connectionString)
    .UseAcmeTenancy(sp)       // ersetzt IMigrationsSqlGenerator
```

```
    .UseContosoAudit());      // umhüllt, was registriert ist: hier Acmes
```

Dekorieren, wo möglich; ersetzen, wo nötig

Decorators lassen sich in beliebiger Reihenfolge kombinieren, weil jeder umhüllt, was er vorfindet. Ein Plugin, das `Replace` verwendet, verwirft dagegen alles, was vor ihm registriert wurde, auch Decorators. Ein ersetzendes Plugin muss deshalb zuerst aktiviert werden, und ein Decorator muss prüfen, ob es etwas zu dekorieren gibt, und sonst mit einer klaren Meldung abbrechen (Provider fehlt oder wurde später hinzugefügt).

Dekorieren setzt voraus, dass die öffentliche Schnittstelle genügt. Das Tenancy-Plugin braucht geschützte Einstiegspunkte innerhalb von `CREATE TABLE` und erbt deshalb vom Npgsql-Generator. Für solche Fälle bleibt der oben beschriebene Ausweg: öffentliche Logik wie `CreatePolicySql`, virtuelle Methoden und `ReplaceService` in der Anwendung.

Tipp

Der Test `CompositionTests` im Beispiel enthält das vollständige Audit-Plugin. Er prüft, dass der Decorator Acmes Generator umhüllt, dass die Migration das SQL beider Plugins enthält und dass das Aktivieren vor dem Provider fehlschlägt.

6 Öffentliche API und Integration in die Anwendung

Die Use-Methode

Nutzer aktivieren das Plugin über eine Extension-Method auf `DbContextOptionsBuilder`. Sie holt die vorhandene Extension oder legt eine neue an und registriert sie über `IDbContextOptionsBuilderInfrastructure`:

Infrastructure/TenancyDbContextOptionsBuilderExtensions.cs (Auszug)

```
public static DbContextOptionsBuilder UseAcmeTenancy(
    this DbContextOptionsBuilder optionsBuilder,
    Action<TenancyOptionsBuilder>? configure = null)
{
    ArgumentNullException.ThrowIfNull(optionsBuilder);

    var extension = optionsBuilder.Options.FindExtension<TenancyOptionsExtension>()
                    ?? new TenancyOptionsExtension();

    ((IDbContextOptionsBuilderInfrastructure)optionsBuilder).AddOrUpdateExtension(extension);
    configure?.Invoke(new TenancyOptionsBuilder(optionsBuilder));
    return optionsBuilder;
}

// Generische Überladung: Method Chaining bleibt typisiert
public static DbContextOptionsBuilder<TContext> UseAcmeTenancy<TContext>(
    this DbContextOptionsBuilder<TContext> optionsBuilder,
    Action<TenancyOptionsBuilder>? configure = null) where TContext : DbContext
    => (DbContextOptionsBuilder<TContext>)UseAcmeTenancy(
        (DbContextOptionsBuilder)optionsBuilder, configure);
```

Die Datei liegt im Namespace `Microsoft.EntityFrameworkCore`. So ist die Methode ohne zusätzliches `using` auffindbar, genau wie `UseNpgsql`. Das ist die verbreitete Konvention im EF-Ökosystem.

Der Options-Builder

Einstellungen werden über einen eigenen Builder gesetzt. Er validiert Eingaben sofort und schreibt jede Änderung als neue Extension zurück:

Infrastructure/TenancyOptionsBuilder.cs (Auszug)

```
public class TenancyOptionsBuilder
{
    public TenancyOptionsBuilder(DbContextOptionsBuilder optionsBuilder)
    {
        ArgumentNullException.ThrowIfNull(optionsBuilder);

        OptionsBuilder = optionsBuilder;
    }

    protected virtual DbContextOptionsBuilder OptionsBuilder { get; }

    public virtual TenancyOptionsBuilder UseSettingName(string settingName)
```

```
        => With(s => s with
        {
            SettingName = TenancyIdentifiers.ValidateSettingName(settingName),
        });

    public virtual TenancyOptionsBuilder ApplyQueryFilter(bool apply = true)
        => With(s => s with { ApplyQueryFilter = apply });

    protected virtual TenancyOptionsBuilder With(
        Func<TenancySettings, TenancySettings> change)
    {
        var extension = OptionsBuilder.Options.FindExtension<TenancyOptionsExtension>()
                        ?? new TenancyOptionsExtension();

        ((IDbContextOptionsBuilderInfrastructure)OptionsBuilder)
            .AddOrUpdateExtension(extension.WithSettings(change(extension.Settings)));
        return this;
    }
}
```

Alternative: Aktivierung über den Provider-Builder

Provider-spezifische Plugins werden manchmal auch über den Options-Builder des Providers aktiviert, etwa `UseNpgsql(cs, o => o.UseNodaTime())`. Dafür schreibt man eine Extension-Method auf `NpgsqlDbContextOptionsBuilder` und kommt über `IRelationalDbContextOptionsBuilderInfrastructure.OptionsBuilder` an den eigentlichen Builder. Das Beispiel nutzt den einfacheren Weg über `DbContextOptionsBuilder`, weil das Plugin mehr ist als eine Provider-Option.

Zwei Container verbinden

Manche Teile eines Plugins brauchen Dienste der *Anwendung*: Der Connection-Interceptor braucht den aktuellen Mandanten, den nur die Anwendung kennt. Der interne EF-Provider kann App-Dienste aber nicht auflösen. Die saubere Lösung: Solche Komponenten leben im App-Container und werden per `AddInterceptors` an EF übergeben.

DependencyInjection/… (Kern), Infrastructure/… (PostgreSQL), Auszüge

```
// Kernpaket: nur den Mandanten-Provider im App-Container registrieren
public static IServiceCollection AddAcmeTenancy<TTenantProvider>(
    this IServiceCollection services,
    ServiceLifetime lifetime = ServiceLifetime.Scoped)
    where TTenantProvider : class, ITenantProvider
{
    services.TryAdd(ServiceDescriptor.Describe(
        typeof(ITenantProvider), typeof(TTenantProvider), lifetime));
    return services;
}

// Provider-Paket: Use-Überladung erzeugt die Interceptors aus diesem Provider
public static DbContextOptionsBuilder UseAcmeTenancy(
    this DbContextOptionsBuilder optionsBuilder,
    IServiceProvider applicationServices,
    Action<TenancyOptionsBuilder>? configure = null)
{
    var tenants = applicationServices.GetRequiredService<ITenantProvider>();
```

```
    return optionsBuilder
        .UseAcmeTenancy(configure)
        .AddInterceptors(
            new TenantConnectionInterceptor(tenants),
            new TenantSaveChangesInterceptor(tenants));
}
```

Bei `AddDbContext((sp, o) => ...)` ist `sp` der Scope der aktuellen Anfrage, weil EF die Options standardmäßig pro Scope baut. Der Mandanten-Provider und damit die Interceptors sehen den richtigen Mandanten. Die Registrierung liegt im Kernpaket, weil sie für jeden Provider gleich ist (Kapitel 41). Für die Brücke in die andere Richtung kann ein Interceptor über `eventData.Context.GetService<T>()` Dienste aus dem *internen* Provider lesen; der Connection-Interceptor holt sich so den Setting-Namen aus den Singleton-Options (Kapitel 15).

> **DbContext-Pooling**
>
> Bei `AddDbContextPool` werden Contexts wiederverwendet und bekommen ihre Options nur einmal. Scoped-Interceptors aus dem ersten Scope würden für alle späteren Anfragen weiterverwendet, mit dem Mandanten der ersten Anfrage. Für Pooling muss der Interceptor ein Singleton sein, der den Mandanten bei jedem Aufruf neu ermittelt, etwa über `IHttpContextAccessor`. Kapitel 35 zeigt diese Variante. Dasselbe gilt für `AddDbContextFactory`, dessen Factory standardmäßig ein Singleton ist.

Teil III

Das Modell

Metadaten, Fluent API, Conventions, Validierung und neue Spaltentypen.

› Kapitel 7: Annotations und Fluent API

› Kapitel 8: Conventions

› Kapitel 9: Modellvalidierung

› Kapitel 10: Type Mappings und Value Converter

7 Annotations und Fluent API

Annotations sind EFs Mechanismus für erweiterbare Metadaten: Schlüssel-Wert-Paare an jedem Modellelement (Modell, Entity-Typ, Property, Index, ...). Ein Plugin speichert seine Konfiguration als Annotations, und die gesamte restliche Infrastruktur (Conventions, Migrations, Snapshots, Compiled Models) kann damit arbeiten.

Regeln für Annotations

Regel	Begründung
Eigenes Präfix (`Acme:Rls:`)	Vermeidet Kollisionen mit EF, Providern und anderen Plugins.
Nur einfache Werte: string, bool, Zahlen, Enums	Snapshots und Migrationen schreiben Werte als C#-Literale. Eigene Klassen lassen sich nicht serialisieren.
Komplexe Werte als String (z. B. JSON)	Strings vergleicht der Differ zuverlässig per `Equals`. Ein Record mit `string[]` würde per Referenz verglichen und ständig „geändert“ wirken.
Default-Werte nicht speichern	„Annotation fehlt“ und „Annotation hat Default“ sind für den Differ verschieden und erzeugen leere Migrationen.
Je Teilkonfiguration eine Annotation	Jede Policy hat eine eigene Annotation. Dann zeigt der Diff genau, welche sich geändert hat.

Metadata/RlsAnnotationNames.cs, Metadata/RlsPolicy.cs (gekürzt)

```
public static class RlsAnnotationNames
{
    public const string Prefix = "Acme:Rls:";
    public const string Enabled = Prefix + "Enabled";
    public const string Forced = Prefix + "Forced";
    public const string PolicyPrefix = Prefix + "Policy:";     // + Policy-Name
    public const string QueryFilterKey = "Acme.Tenancy";
}

public sealed record RlsPolicy(string Name, PolicyCommand Command, bool Permissive,
    string[]? Roles, string? Using, string? WithCheck)
{
    public string Serialize()
        => JsonSerializer.Serialize(this, RlsJsonContext.Default.RlsPolicy);

    public static RlsPolicy Deserialize(string json) => /* ... */;
}
```

Die Fluent API

Fluent-API-Methoden sind Extension-Methods auf den Buildern von EF. Drei Gestaltungsregeln machen sie professionell: Die **nicht-generische** Variante ist die eigentliche Implementierung (der Model-Snapshot nutzt nicht-generische Builder, Kapitel 20). Die **generische** Variante delegiert nur und erhält den Typ für weiteres Chaining. Und die Klasse liegt im Namespace `Microsoft.EntityFrameworkCore`, damit sie überall ohne `using` verfügbar ist.

Builders/RlsEntityTypeBuilderExtensions.cs (gekürzt)

```
namespace Microsoft.EntityFrameworkCore;

public static class RlsEntityTypeBuilderExtensions
{
    public static EntityTypeBuilder HasRowLevelSecurity(
        this EntityTypeBuilder builder, bool force = false)
    {
        ArgumentNullException.ThrowIfNull(builder);
        builder.Metadata.SetAnnotation(RlsAnnotationNames.Enabled, true);

        if (force)
            builder.Metadata.SetAnnotation(RlsAnnotationNames.Forced, true);
        else
            builder.Metadata.RemoveAnnotation(RlsAnnotationNames.Forced);   // kein Default!

        return builder;
    }

    public static EntityTypeBuilder HasPolicy(
        this EntityTypeBuilder builder, string name, Action<PolicyBuilder> configure)
    {
        var policyBuilder = new PolicyBuilder(name);
        configure(policyBuilder);
        builder.Metadata.SetAnnotation(
            RlsAnnotationNames.PolicyPrefix + name, policyBuilder.Build().Serialize());
        return builder;
    }

    public static EntityTypeBuilder<TEntity> HasRowLevelSecurity<TEntity>(
        this EntityTypeBuilder<TEntity> builder, bool force = false) where TEntity : class
        => (EntityTypeBuilder<TEntity>)HasRowLevelSecurity(
            (EntityTypeBuilder)builder, force);

    // ... HasNoRowLevelSecurity, HasPolicy<TEntity> analog
}
```

Verschachtelte Konfiguration wie bei `HasPolicy` nimmt ein `Action<PolicyBuilder>` entgegen. Der Builder prüft die Regeln der Datenbank schon beim Modellbau. Ein Fehler wie „INSERT-Policy mit USING“ fällt so beim ersten Start der Anwendung auf und nicht erst beim Ausführen der Migration in Produktion:

Builders/PolicyBuilder.cs › Build

```
internal RlsPolicy Build()
{
    if (_using is null && _withCheck is null)
        throw new InvalidOperationException(
            $"Policy '{_name}' braucht USING oder WITH CHECK.");

    if (_command == PolicyCommand.Insert && _using is not null)
        throw new InvalidOperationException(
            $"INSERT-Policy '{_name}' erlaubt nur WITH CHECK.");

    if (_command is PolicyCommand.Select or PolicyCommand.Delete && _withCheck is not null)
        throw new InvalidOperationException(
```

```
            $"{_command}-Policy '{_name}' erlaubt nur USING.");

    return new RlsPolicy(_name, _command, _permissive,
        _roles.Count > 0 ? [.. _roles] : null, _using, _withCheck);
}
```

Konfigurationsquellen

EF merkt sich zu jedem Metadatum, *woher* es stammt. Eine Quelle kann eine gleich starke oder schwächere überschreiben, aber nie eine stärkere. Das ist der Mechanismus, mit dem Plugins sinnvolle Defaults setzen, die Nutzer jederzeit übersteuern können:

ConfigurationSource	Gesetzt durch	Stärke
`Explicit`	Fluent API im `OnModelCreating` (`Metadata.SetAnnotation`)	stärkste
`DataAnnotation`	Attribut-Conventions (`fromDataAnnotation: true`)	mittel
`Convention`	Conventions über `IConvention*Builder`	schwächste

Für eigene Conventions bietet das Plugin deshalb zusätzlich Überladungen auf dem Convention-Builder an. Sie geben `null` zurück, wenn eine stärkere Quelle anders entschieden hat:

Builders/RlsEntityTypeBuilderExtensions.cs › Convention-Variante

```
public static IConventionEntityTypeBuilder? HasRowLevelSecurity(
    this IConventionEntityTypeBuilder builder,
    bool enabled,
    bool fromDataAnnotation = false)
    => builder.HasAnnotation(RlsAnnotationNames.Enabled, enabled, fromDataAnnotation)
           is null
        ? null
        : builder;
```

8 Conventions

Conventions verändern das Modell automatisch. EF selbst besteht zum größten Teil aus Conventions: Schlüssel erkennen, Beziehungen finden, Tabellennamen ableiten. Ein Plugin fügt eigene hinzu, um Konfiguration überflüssig zu machen.

Convention-Typen und Zeitpunkte

Eine Convention implementiert ein oder mehrere Interfaces. Jedes steht für ein Ereignis während des Modellbaus. Die wichtigsten:

Interface	Ereignis	Typische Verwendung
`IEntityTypeAddedConvention`	Entity-Typ wurde hinzugefügt	Attribute auswerten, Grundkonfiguration
`IPropertyAddedConvention`	Property wurde hinzugefügt	Spaltentypen, Namenskonventionen
`IForeignKeyAddedConvention`	Beziehung entstanden	Indizes, Löschverhalten
`IEntityTypeBaseTypeChangedConvention`	Vererbung geändert	Konfiguration verschieben
`IModelFinalizingConvention`	Modell wird abgeschlossen (noch änderbar)	Alles, was vom fertigen Modell abhängt
`IModelFinalizedConvention`	Modell ist fertig (read-only)	Validierung

> **Der häufigste Convention-Fehler**
>
> Reaktive Conventions (`*Added`, `*Changed`) laufen, während der Nutzer noch konfiguriert. Ein Spaltenname kann sich danach noch ändern, ein Entity-Typ kann noch einen Basistyp bekommen. Wer davon abhängt, muss auf *alle* relevanten Ereignisse reagieren oder `IModelFinalizingConvention` verwenden. Letzteres ist einfacher und für die meisten Plugins richtig.

Conventions einhängen

Das Plugin-Interface `IConventionSetPlugin` bekommt das Convention-Set und fügt eigene Conventions hinzu. `ConventionSet.Add` trägt eine Convention in alle Listen ein, deren Interfaces sie implementiert, jeweils am Ende:

Conventions/TenancyConventionSetPlugin.cs

```
public class TenancyConventionSetPlugin : IConventionSetPlugin
{
    private readonly ProviderConventionSetBuilderDependencies _dependencies;
    private readonly IDbContextOptions _options;
    private readonly ISqlGenerationHelper _sqlGenerationHelper;
    private readonly TenancyLoggingDefinitions _loggingDefinitions;

    public TenancyConventionSetPlugin(
        ProviderConventionSetBuilderDependencies dependencies,
        IDbContextOptions options,
        ISqlGenerationHelper sqlGenerationHelper,
```

```
        TenancyLoggingDefinitions loggingDefinitions)
    {
        // ... ArgumentNullException.ThrowIfNull für alle Parameter ...
        _dependencies = dependencies;
        _options = options;
        _sqlGenerationHelper = sqlGenerationHelper;
        _loggingDefinitions = loggingDefinitions;
    }

    public virtual ConventionSet ModifyConventions(ConventionSet conventionSet)
    {
        var settings = _options.FindExtension<TenancyOptionsExtension>()?.Settings;
        if (settings is null)
            return conventionSet;

        // Kapitel „Value Generation“
        conventionSet.Add(new TenantIdKeyValueGenerationConvention());
        conventionSet.Add(new NoTenantIsolationAttributeConvention(_dependencies));
        conventionSet.Add(new TenantIsolationConvention(
            settings, _sqlGenerationHelper, _dependencies.Logger, _loggingDefinitions));
        conventionSet.Add(new TenancyModelValidationConvention(
            _dependencies.Logger, _loggingDefinitions));

        return conventionSet;
    }
}
```

`ProviderConventionSetBuilderDependencies` ist das Parameterobjekt, das auch EFs eigene Conventions bekommen. Es enthält unter anderem den Model-Logger, den Validation-Logger und die Type-Mapping-Source. Andere Dienste wie `ISqlGenerationHelper` lassen sich zusätzlich injizieren, weil das Plugin selbst im internen Provider aufgelöst wird.

Die Isolations-Convention

Die zentrale Convention des Beispiels läuft bei `ModelFinalizing`, weil Spaltennamen erst dann feststehen. Sie findet die Mandanten-Property (per `IsTenantProperty()` markiert oder `TenantId` bei `ITenantOwned`, Kapitel 39), aktiviert RLS mit Policy, legt einen Index an, setzt eine Laufzeit-Annotation für den SaveChanges-Interceptor und optional einen Query-Filter:

Conventions/TenantIsolationConvention.cs › ProcessModelFinalizing

```
public virtual void ProcessModelFinalizing(
    IConventionModelBuilder modelBuilder,
    IConventionContext<IConventionModelBuilder> context)
{
    foreach (var entityType in modelBuilder.Metadata.GetEntityTypes())
    {
        if (entityType.BaseType is not null       // nur Root-Typen einer Hierarchie
            || entityType.IsOwned())              // Owned Types teilen die Tabelle
            continue;

        // Markierte Property oder "TenantId" bei ITenantOwned (Kernpaket,
        // Kapitel „Fluent API auf Property-Ebene und Laufzeit-Annotations“)
        if (entityType.FindTenantPropertyCandidate() is not IConventionProperty property)
            continue;                             // (+ Typprüfung, gekürzt)
```

```
        if (entityType.IsRlsOptedOut())
        {
            _logger.TenantIsolationOptedOut(_loggingDefinitions, entityType);  // Warnung
            continue;
        }

        // Keine Tabelle (View, ToSqlQuery, Keyless) -> kein RLS
        if (StoreObjectIdentifier.Create(entityType, StoreObjectType.Table)
            is not { } table)
            continue;

        // Laufzeit-Annotation: der SaveChanges-Interceptor findet so die Property
        entityType.Builder.HasAnnotation(
            TenancyAnnotationNames.TenantProperty, property.Name);

        ApplyRowLevelSecurity(entityType, property, table);
        if (_settings.CreateTenantIndex) EnsureTenantIndex(entityType, property);
        if (_settings.ApplyQueryFilter) ApplyQueryFilter(entityType, property);

        _logger.TenantIsolationApplied(_loggingDefinitions, entityType);       // Debug
    }
}
```

Conventions/TenantIsolationConvention.cs › ApplyRowLevelSecurity

```
protected virtual void ApplyRowLevelSecurity(
    IConventionEntityType entityType,
    IConventionProperty property,
    StoreObjectIdentifier table)
{
    // Spaltenname für genau diese Tabelle (bei Table Splitting kann er abweichen)
    var column = _sqlGenerationHelper.DelimitIdentifier(property.GetColumnName(table)!);
    var condition =
        $"{column} = nullif(current_setting('{_settings.SettingName}', true), '')::uuid";

    var policy = new RlsPolicy(_settings.PolicyName, PolicyCommand.All, Permissive: true,
        Roles: null, Using: condition, WithCheck: condition);

    // Convention-Builder: überschreibt explizite Konfiguration NICHT
    var builder = entityType.Builder;
    builder.HasAnnotation(RlsAnnotationNames.Enabled, true);
    if (_settings.ForceRowLevelSecurity)
        builder.HasAnnotation(RlsAnnotationNames.Forced, true);
    builder.HasAnnotation(RlsAnnotationNames.PolicyPrefix + _settings.PolicyName,
        policy.Serialize());
}
```

Selbst erzeugtes SQL validieren

Der Setting-Name wird unmaskiert in den Policy-Ausdruck geschrieben. Anders als bei SQL, das der Entwickler selbst in `Using(...)` schreibt, setzt hier das *Plugin* SQL zusammen und ist dafür verantwortlich, dass nichts Ungültiges hineinkommt. Deshalb validiert der Options-Builder den Namen mit einem strengen regulären Ausdruck (`[a-z_][a-z0-9_]*\.[a-z_][a-z0-9_]*`).

Strukturelle Änderungen: Index und Query-Filter

Conventions dürfen nicht nur Annotations setzen, sondern das Modell strukturell verändern. Der Index wird nur angelegt, wenn nicht schon ein Index oder Schlüssel mit der Mandantenspalte beginnt. Der Query-Filter wird als Expression Tree gebaut und als **benannter Filter** registriert, eine Neuerung in EF Core 10. Benannte Filter koexistieren mit eigenen Filtern des Nutzers und lassen sich gezielt per `IgnoreQueryFilters(["Acme.Tenancy"])` abschalten.

Conventions/TenantIsolationConvention.cs (Auszug)

```
protected virtual void EnsureTenantIndex(
    IConventionEntityType entityType, IConventionProperty property)
{
    var covered = entityType.GetIndexes().Any(i => i.Properties[0] == property)
                  || entityType.GetKeys().Any(k => k.Properties[0] == property);
    if (!covered)
        entityType.Builder.HasIndex(new[] { property });
}

protected virtual void ApplyQueryFilter(
    IConventionEntityType entityType, IConventionProperty property)
{
    // e => EF.Property<TenantId>(e, "...") == EF.Functions.CurrentTenant()
    // EF.Property funktioniert für jeden Property-Namen und auch für Shadow-Properties
    var parameter = Expression.Parameter(entityType.ClrType, "e");
    var body = Expression.Equal(
        Expression.Call(typeof(EF), nameof(EF.Property), [typeof(TenantId)],
            parameter, Expression.Constant(property.Name)),
        Expression.Call(CurrentTenantMethod, Expression.Property(null, FunctionsProperty)));

    entityType.Builder.HasQueryFilter(
        RlsAnnotationNames.QueryFilterKey, Expression.Lambda(body, parameter));
}
```

Bemerkenswert ist, dass der Filter die eigene Funktion `EF.Functions.CurrentTenant()` verwendet. Er wird damit über den Translator aus Kapitel 12 nach SQL übersetzt. Die Bausteine des Plugins greifen ineinander.

Endgültige Namen, egal wer sie setzt

Die Isolations-Convention liest den Spaltennamen erst bei `ModelFinalizing` und nur über `GetColumnName(table)`. Namen, die andere Conventions oder Plugins ändern, stehen zu diesem Zeitpunkt fest. Das Beispiel prüft das gegen EFCore.NamingConventions, das Tabellen und Spalten in snake_case umbenennt, und zwar in beiden Aktivierungsreihenfolgen:

Anwendung

```
options.UseNpgsql(connectionString).UseAcmeTenancy().UseSnakeCaseNamingConvention();
```

Erzeugtes Migrations-SQL (Auszug, tests/.../NamingConventionTests.cs)

```
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON orders AS PERMISSIVE FOR ALL
    USING (tenant_id = nullif(current_setting('app.tenant_id', true), '')::uuid)
    WITH CHECK (tenant_id = nullif(current_setting('app.tenant_id', true), '')::uuid);
```

```
CREATE INDEX ix_orders_tenant_id ON orders (tenant_id);
```

Namen nie selbst zusammensetzen

Ein Plugin, das Namen selbst zusammensetzt (aus `Name`-Properties oder per String-Verkettung), bricht, sobald ein Naming-Plugin aktiv ist. Immer das Modell nach dem endgültigen Namen fragen, und zwar so spät wie möglich.

Attribut-Conventions

Data Annotations wertet man mit einer Attribut-Convention aus. EF bietet dafür die Basisklasse `TypeAttributeConventionBase<TAttribute>`, die das Attribut findet und nur noch die Reaktion verlangt. Der Wert wird mit `fromDataAnnotation: true` gesetzt, sodass die Fluent API ihn überstimmen kann:

Conventions/NoTenantIsolationAttributeConvention.cs

```
public class NoTenantIsolationAttributeConvention
    : TypeAttributeConventionBase<NoTenantIsolationAttribute>
{
    public NoTenantIsolationAttributeConvention(
        ProviderConventionSetBuilderDependencies dependencies)
        : base(dependencies)
    {
    }

    protected override void ProcessEntityTypeAdded(
        IConventionEntityTypeBuilder entityTypeBuilder,
        NoTenantIsolationAttribute attribute,
        IConventionContext<IConventionEntityTypeBuilder> context)
        => entityTypeBuilder.HasRowLevelSecurity(enabled: false, fromDataAnnotation: true);
}
```

Tipp

Das Attribut selbst liegt im *Kernpaket*, die Convention im Provider-Paket. Entitätsklassen können das Attribut verwenden, ohne von PostgreSQL abzuhängen. Das ist dieselbe Aufteilung wie bei EFs eigenen Attributen in `System.ComponentModel.DataAnnotations`.

9 Modellvalidierung

Ein Plugin sollte Fehlkonfiguration erkennen, bevor sie zu falschem SQL oder Datenlecks führt. EF validiert Modelle über `IModelValidator`, den es aber nur einmal gibt und der dem Provider gehört. Für Plugins ist eine `IModelFinalizedConvention` der bessere Ort: Sie läuft auf dem fertigen Modell und kollidiert mit niemandem.

Conventions/TenancyModelValidationConvention.cs

```
public virtual IModel ProcessModelFinalized(IModel model)
{
    foreach (var entityType in model.GetEntityTypes())
    {
        if (!entityType.HasRlsConfiguration())
            continue;

        if (entityType.BaseType is not null)
            throw new InvalidOperationException(
                $"RLS an '{entityType.DisplayName()}' ist ungültig: nur am Root-Typ "
                + $"('{entityType.GetRootType().DisplayName()}') erlaubt.");

        if (StoreObjectIdentifier.Create(entityType, StoreObjectType.Table) is null)
            throw new InvalidOperationException("... nicht auf eine Tabelle gemappt ...");

        if (!entityType.IsRlsEnabled() && entityType.GetRlsPolicies().Count > 0)
            _logger.PoliciesWithoutRowLevelSecurity(_loggingDefinitions, entityType);
    }
    return model;
}
```

Die Validierung unterscheidet zwei Arten von Problemen. **Strukturelle Fehler**, die garantiert zu falschem Verhalten führen, werfen eine Exception. **Fragwürdige Konfiguration**, die gewollt sein kann, erzeugt eine Warnung über das Logging-System. Nutzer können solche Warnungen gezielt in Fehler umwandeln (Kapitel 16).

10 Type Mappings und Value Converter

Ein Type Mapping verbindet einen CLR-Typ mit einem Datenbanktyp. Es bestimmt den Spaltentyp in Migrationen, wie Parameter gesetzt werden, wie Literale im SQL aussehen, wie Werte verglichen und kopiert werden und wie sie in JSON-Spalten gespeichert werden. Für eigene Typen hat ein Plugin zwei Möglichkeiten:

Weg	Vorteile	Nachteile
Value Converter (Nutzer konfiguriert `HasConversion`)	Kein Plugin nötig	Muss für jede Property oder per `ConfigureConventions` konfiguriert werden
`IRelationalTypeMapping SourcePlugin`	Typ funktioniert überall ohne Konfiguration, eigene Literale, volle Kontrolle	Provider-spezifisch (Store-Typ)

Der Typ und sein Konverter

Der stark typisierte `TenantId` und sein Konverter liegen im Kernpaket. Der Konverter hat eine gemeinsame Instanz; warum, zeigt der Abschnitt über Compiled Models.

TenantId.cs, TenantIdConverter.cs (gekürzt)

```
public readonly record struct TenantId(Guid Value)
{
    public static TenantId Empty => default;
    public bool IsEmpty => Value == Guid.Empty;              // wird nach SQL übersetzt
    public static TenantId New() => new(Guid.CreateVersion7());
}

public sealed class TenantIdConverter : ValueConverter<TenantId, Guid>
{
    public static TenantIdConverter Instance { get; } = new();

    public TenantIdConverter()
        : base(id => id.Value, value => new TenantId(value))
    {
    }
}
```

Das Type Mapping

```
public class TenantIdTypeMapping : RelationalTypeMapping
{
    // Pflicht für Compiled Models: öffentliche statische Default-Property
    public static TenantIdTypeMapping Default { get; } = new();

    public TenantIdTypeMapping()
        : base(new RelationalTypeMappingParameters(
            new CoreTypeMappingParameters(
                typeof(TenantId),
                converter: TenantIdConverter.Instance,
                jsonValueReaderWriter: TenantIdJsonReaderWriter.Instance),
```

```
            storeType: "uuid",
            dbType: System.Data.DbType.Guid))
    {
    }

    protected TenantIdTypeMapping(RelationalTypeMappingParameters parameters)
        : base(parameters)
    {
    }

    // EF klont Mappings, z. B. für Facetten oder geänderte Comparer
    protected override RelationalTypeMapping Clone(
        RelationalTypeMappingParameters parameters)
        => new TenantIdTypeMapping(parameters);

    protected override string GenerateNonNullSqlLiteral(object value)
    {
        var guid = value switch
        {
            TenantId id => id.Value,
            Guid g => g,
            _ => throw new ArgumentException($"Unerwarteter Typ {value.GetType()}.");
        };
        return $"'{guid}'::uuid";
    }
}
```

Jedes eigene Mapping braucht den geschützten Konstruktor mit `RelationalTypeMappingParameters` und eine `Clone`-Überschreibung, die den eigenen Typ erzeugt. Fehlt sie, verliert ein geklontes Mapping sein Verhalten, zum Beispiel die Literal-Erzeugung.

Das Type-Mapping-Plugin

`RelationalTypeMappingSource` befragt alle Plugins, *bevor* es auf die Mappings des Providers zurückgreift. Ein Plugin gibt `null` zurück, wenn es nicht zuständig ist:

Storage/TenancyTypeMappingSourcePlugin.cs

```
public class TenancyTypeMappingSourcePlugin : IRelationalTypeMappingSourcePlugin
{
    public virtual RelationalTypeMapping? FindMapping(
        in RelationalTypeMappingInfo mappingInfo)
    {
        if (mappingInfo.ClrType != typeof(TenantId))
            return null;

        // Explizit anderer Spaltentyp (HasColumnType("text")): nicht zuständig
        var storeTypeBase = mappingInfo.StoreTypeNameBase;
        if (storeTypeBase is not null
            && !string.Equals(storeTypeBase, "uuid", StringComparison.OrdinalIgnoreCase))
            return null;

        return TenantIdTypeMapping.Default;
    }
}
```

> **Hinweis**
>
> Ohne Mapping würde EF `TenantId` beim Modellbau nicht als Skalartyp erkennen und mit einer Fehlermeldung abbrechen. Mit dem Plugin funktioniert der Typ als Property, als Schlüssel, in Abfragen, als Parameter und in JSON-Spalten, ohne eine Zeile Konfiguration.

JSON-Unterstützung

Für JSON-Spalten braucht ein Mapping einen `JsonValueReaderWriter`. Er hat drei Pflichten: lesen, schreiben und beschreiben, wie er im generierten Code erzeugt wird. Das Muster folgt EFs eigenem `JsonGuidReaderWriter`:

TenantIdJsonReaderWriter.cs

```
public sealed class TenantIdJsonReaderWriter : JsonValueReaderWriter<TenantId>
{
    private static readonly PropertyInfo InstanceProperty =
        typeof(TenantIdJsonReaderWriter).GetProperty(nameof(Instance))!;

    public static TenantIdJsonReaderWriter Instance { get; } = new();

    private TenantIdJsonReaderWriter()
    {
    }

    public override TenantId FromJsonTyped(
        ref Utf8JsonReaderManager manager, object? existingObject = null)
        => new(manager.CurrentReader.GetGuid());

    public override void ToJsonTyped(Utf8JsonWriter writer, TenantId value)
        => writer.WriteStringValue(value.Value);

    // Für Compiled Models und vorkompilierte Abfragen
    public override Expression ConstructorExpression
        => Expression.Property(null, InstanceProperty);
}
```

> **Compiled Models brauchen Default und Instance**
>
> Beim Generieren für NativeAOT (`dotnet ef dbcontext optimize --nativeaot`) erzeugt der Generator für Compiled Models (Kapitel 21) für jedes Type Mapping Code der Form `TenantIdTypeMapping.Default.Clone(...)`. Er verlangt eine **öffentliche, statische Property** namens `Default` auf einem **öffentlichen** Typ und bricht sonst ab. Konverter und Reader/Writer übernimmt er nur dann ohne zusätzlichen Code, wenn sie *dieselbe Instanz* wie im Default-Mapping sind. Deshalb nutzt das Mapping die statischen Instanzen statt `new ...()`. Ohne NativeAOT löst EF 10 Type Mappings stattdessen beim Start auf; ein Plugin sollte die Anforderungen trotzdem erfüllen, denn über NativeAOT entscheiden die Anwender, nicht das Plugin.

Teil IV

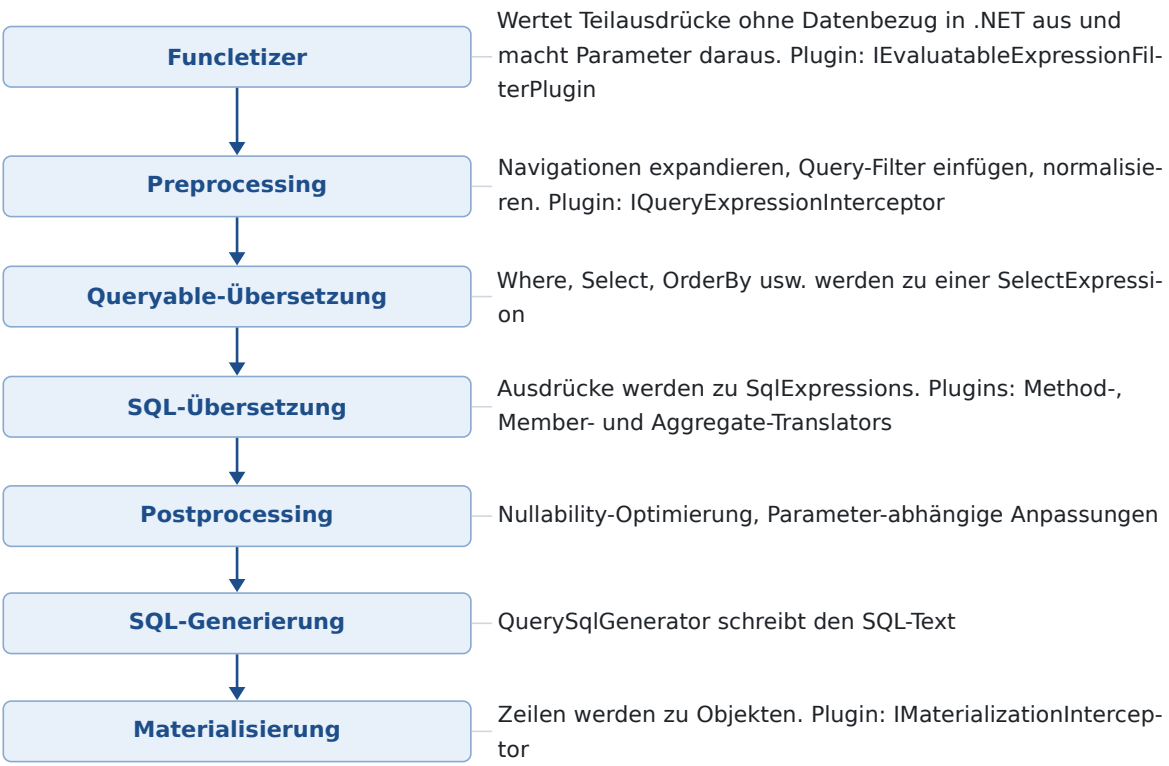
Abfragen

Wie LINQ zu SQL wird und wie ein Plugin eigene Funktionen und Member übersetzt.

- Kapitel 11: Die Query-Pipeline
- Kapitel 12: Methoden und Member übersetzen
- Kapitel 13: Clientseitige Auswertung verhindern
- Kapitel 14: Weitere Erweiterungspunkte der Abfrage

11 Die Query-Pipeline

Eine LINQ-Abfrage durchläuft in EF Core mehrere Stufen, bevor SQL entsteht. Wer weiß, wo welche Stufe sitzt, weiß auch, wo ein Plugin ansetzen muss:



Die kompilierte Abfrage wird gecacht. Translators laufen also nur einmal pro Abfrageform, nicht bei jeder Ausführung. Werte, die sich pro Ausführung ändern (Parameter, Session-Settings), müssen deshalb im SQL als Parameter oder als Datenbankfunktion stehen, nie als zur Übersetzungszeit eingesetzte Konstante. Genau deshalb übersetzt das Beispiel `CurrentTenant()` in einen Aufruf von `current_setting(...)` und setzt nicht den aktuellen Mandanten ein.

12 Methoden und Member übersetzen

Das EF.Functions-Muster

Datenbankfunktionen, die keine Entsprechung in .NET haben, werden als Extension-Methods auf `DbFunctions` deklariert. Sie werden nie ausgeführt; der Körper wirft nur eine Exception für den Fall, dass jemand sie außerhalb von LINQ aufruft. Die Deklaration liegt im Kernpaket, die Übersetzung im Provider-Paket:

TenancyDbFunctionsExtensions.cs (Kernpaket)

```
namespace Microsoft.EntityFrameworkCore;

public static class TenancyDbFunctionsExtensions
{
    public static TenantId CurrentTenant(this DbFunctions _)
        => throw new InvalidOperationException(
            $"{nameof(CurrentTenant)} ist nur in LINQ-to-Entities-Abfragen verwendbar.");
}
```

Ein Method-Call-Translator

Ein `IMethodCallTranslator` bekommt jeden Methodenaufruf, den EF nicht selbst kennt. Er prüft, ob er zuständig ist, und baut dann aus `ISqlExpressionFactory` einen SQL-Ausdrucksbaum:

Query/CurrentTenantTranslator.cs

```
public class CurrentTenantTranslator : IMethodCallTranslator
{
    private static readonly MethodInfo CurrentTenantMethod =
        typeof(TenancyDbFunctionsExtensions).GetRuntimeMethod(
            nameof(TenancyDbFunctionsExtensions.CurrentTenant), [typeof(DbFunctions)])!;

    private readonly ISqlExpressionFactory _sql;
    private readonly IRelationalTypeMappingSource _typeMappingSource;
    private readonly ITenancySingletonOptions _options;

    public CurrentTenantTranslator(
        ISqlExpressionFactory sql,
        IRelationalTypeMappingSource typeMappingSource,
        ITenancySingletonOptions options)
    {
        // ... ArgumentNullException.ThrowIfNull für alle Parameter ...
        _sql = sql;
        _typeMappingSource = typeMappingSource;
        _options = options;
    }

    public virtual SqlExpression? Translate(
        SqlExpression? instance, MethodInfo method,
        IReadOnlyList<SqlExpression> arguments,
        IDiagnosticsLogger<DbLoggerCategory.Query> logger)
    {
        if (method != CurrentTenantMethod)
```

```
            return null;                            // nicht zuständig -> nächster Translator

        var text = _typeMappingSource.FindMapping(typeof(string))!;
        var boolean = _typeMappingSource.FindMapping(typeof(bool))!;

        var currentSetting = _sql.Function(
            "current_setting",
            [
                _sql.Constant(_options.Settings.SettingName, text),
                _sql.Constant(true, boolean),
            ],
            nullable: true,
            argumentsPropagateNullability: [true, false],
            typeof(string), text);

        var nullIfEmpty = _sql.Function(
            "nullif",
            [currentSetting, _sql.Constant(string.Empty, text)],
            nullable: true,
            argumentsPropagateNullability: [true, false],
            typeof(string), text);

        return _sql.Convert(nullIfEmpty, typeof(TenantId), TenantIdTypeMapping.Default);
    }
}
```

Aus `o.TenantId == EF.Functions.CurrentTenant()` wird damit:

Erzeugtes SQL (Ausschnitt)

```
WHERE o."TenantId" = CAST(nullif(current_setting('app.tenant_id', TRUE), '') AS uuid)
```

Detail	Bedeutung
`return null`	Vertrag für „nicht zuständig“. Niemals eine Exception für fremde Methoden werfen; dann ist der nächste Translator an der Reihe.
`arguments`	Bei Extension-Methods ist `arguments[0]` der `this`-Parameter (hier `DbFunctions`).
`nullable`	Kann das Ergebnis NULL sein? EF optimiert damit `IS NULL`-Prüfungen. Falsche Angaben führen zu falschen Ergebnissen.
`argumentsPropagateNullability`	Wird das Ergebnis NULL, wenn dieses Argument NULL ist? Für `nullif(x, '')`: ja für x, nein für den Leerstring.
Type Mappings	Jeder SQL-Ausdruck braucht ein Mapping. Es bestimmt Literale, Parameter und Casts. `Convert` mit dem TenantId-Mapping erzeugt `CAST(... AS uuid)`.
`Constant(...)`	Konstanten werden als Literal über das Mapping erzeugt und korrekt maskiert, auch der Setting-Name.

Ein Member-Translator

Für Properties und Felder gibt es `IMemberTranslator`. Das Beispiel übersetzt `tenantId.IsEmpty`. Die Konstante bekommt das Type Mapping der Spalte, damit das Literal im richtigen Format entsteht:

Query/TenantIdMemberTranslator.cs

```
public class TenantIdMemberTranslator : IMemberTranslator
{
    private static readonly PropertyInfo IsEmptyProperty =
        typeof(TenantId).GetProperty(nameof(TenantId.IsEmpty))!;

    private readonly ISqlExpressionFactory _sql;

    public TenantIdMemberTranslator(ISqlExpressionFactory sql)
    {
        ArgumentNullException.ThrowIfNull(sql);

        _sql = sql;
    }

    public virtual SqlExpression? Translate(
        SqlExpression? instance, MemberInfo member, Type returnType,
        IDiagnosticsLogger<DbLoggerCategory.Query> logger)
    {
        if (instance is null || !Equals(member, IsEmptyProperty))
            return null;

        return _sql.Equal(instance, _sql.Constant(TenantId.Empty, instance.TypeMapping));
    }
}
```

Translator-Plugins

Translators werden nicht direkt registriert, sondern über Plugin-Klassen, die eine Liste liefern. Die Plugins sind **Scoped** (Anhang A) und können daher Scoped-Dienste wie `ISqlExpressionFactory` injizieren:

`Query/TenancyTranslatorPlugins.cs`

```
public class TenancyMethodCallTranslatorPlugin : IMethodCallTranslatorPlugin
{
    public TenancyMethodCallTranslatorPlugin(
        ISqlExpressionFactory sql,
        IRelationalTypeMappingSource typeMappingSource,
        ITenancySingletonOptions options)
    {
        Translators = [new CurrentTenantTranslator(sql, typeMappingSource, options)];
    }

    public virtual IEnumerable<IMethodCallTranslator> Translators { get; }
}

public class TenancyMemberTranslatorPlugin : IMemberTranslatorPlugin
{
    public TenancyMemberTranslatorPlugin(ISqlExpressionFactory sql)
    {
        Translators = [new TenantIdMemberTranslator(sql)];
    }

    public virtual IEnumerable<IMemberTranslator> Translators { get; }
}
```

> **Tipp**
>
> `query.ToQueryString()` erzeugt das SQL einer Abfrage, ohne sie auszuführen und ohne Datenbankverbindung. Es ist das wichtigste Werkzeug, um Translators zu entwickeln und zu testen (Kapitel 23).

13 Clientseitige Auswertung verhindern

Der erste Schritt der Pipeline, der Funcletizer, wertet jeden Teilausdruck vorab in .NET aus, der keinen Bezug zu Abfragedaten hat. Aus `o.Total > limit * 2` wird so ein Parameter. Für `EF.Functions.CurrentTenant()` wäre das fatal: Der Ausdruck hat keinen Datenbezug, also würde EF die Methode ausführen, und die wirft ihre Exception.

EF kennt nur wenige Ausnahmen. Im Quellcode von `EvaluatableExpressionFilter` (EF Core 10) sind das `DateTime.Now`, `DateTime.UtcNow`, `DateTime.Today`, die `DateTimeOffset`-Gegenstücke, `Guid.NewGuid()`, `Random.Next(...)` und alle Methoden, die in EFs eigener Klasse `DbFunctionsExtensions` deklariert sind. Eigene EF.Functions-Methoden liegen in einer anderen Klasse und fallen **nicht** darunter. Danach fragt EF alle registrierten Plugins:

Kernpaket: TenancyDbFunctionsAttribute.cs, Query/TenancyEvaluatable…Plugin.cs

```
[AttributeUsage(AttributeTargets.Class, Inherited = false)]
public sealed class TenancyDbFunctionsAttribute : Attribute;

public class TenancyEvaluatableExpressionFilterPlugin : IEvaluatableExpressionFilterPlugin
{
    public virtual bool IsEvaluatableExpression(Expression expression)
        => expression is not MethodCallExpression { Method.DeclaringType: { } type }
           || !type.IsDefined(typeof(TenancyDbFunctionsAttribute), inherit: false);
}

[TenancyDbFunctions]
public static class TenancyDbFunctionsExtensions { /* CurrentTenant() */ }
```

Das Plugin liegt im **Kernpaket** und erkennt Funktionsklassen an einem Attribut. So deckt ein einziges Plugin auch die PostgreSQL-spezifischen Funktionen aus Kapitel 37 und 38 und jedes künftige Provider-Paket ab, ohne dass der Kern diese Klassen kennen muss.

> **Wann das Plugin nötig ist**
>
> Ob eine eigene EF.Functions-Methode dieses Plugin braucht, hängt davon ab, ob sie Argumente mit Datenbezug hat. `EF.Functions.Soundex(c.Name)` ist unproblematisch, weil `c.Name` auf eine Spalte zeigt. Methoden ohne Argumente oder nur mit Konstanten (`CurrentTenant()`, `Random()`, `ServerTime()`) brauchen es immer. Fehlt es, kommt die Exception aus dem Methodenkörper, was schwer zu diagnostizieren ist.

14 Weitere Erweiterungspunkte der Abfrage

Das Beispiel braucht Method- und Member-Translators. Komplexe Plugins greifen auf weitere Punkte zurück; hier ein Überblick mit Einordnung:

Erweiterungspunkt	Zweck	Hinweise
`IAggregateMethodCallTranslatorPlugin`	Eigene Aggregatfunktionen in `GroupBy` (z. B. `bool_and`, Statistikfunktionen)	Scoped, mehrfach registrierbar; umgesetzt in Kapitel 38
`IQueryExpressionInterceptor`	LINQ-Ausdrucksbaum vor der Übersetzung umschreiben	Singleton-Interceptor; ideal für Query-Rewrites, Tags, Hints
`IMaterializationInterceptor`	Objekte bei der Materialisierung erzeugen, initialisieren oder prüfen	Singleton-Interceptor; muss zustandslos und thread-sicher sein
`IMethodCallTranslatorProvider` u. a.	Translator-Infrastruktur ersetzen	Einzeldienste des Providers; nur in Ausnahmefällen
`IQuerySqlGeneratorFactory`	SQL-Generierung für eigene SqlExpression-Typen	Singleton, Provider-Dienst; umgesetzt in Kapitel 37
`IRelationalParameterBasedSqlProcessorFactory`	Nullability-Verarbeitung eigener Ausdrucksknoten	Scoped, Provider-Dienst; umgesetzt in Kapitel 37
`IQueryTranslationPostprocessorFactory`	Ausdrucksbaum nach der Übersetzung anpassen	Provider-Dienst; tiefer Eingriff, eng an EF-Interna gebunden

> **Vorkompilierte Abfragen**
>
> Mit NativeAOT und vorkompilierten Abfragen (seit EF Core 9) übersetzt EF Abfragen bereits beim Build. Translators laufen dann im Build-Prozess, nicht zur Laufzeit. Sie dürfen deshalb nur von Options und Modell abhängen, nie von Laufzeitzustand. Das Beispiel erfüllt das: Der Setting-Name ist eine Option, der Mandant selbst wird erst in der Datenbank-Session ausgewertet.

Teil V

Laufzeit

Eingreifen beim Speichern, Verbinden und Ausführen, und Ereignisse sauber melden.

› Kapitel 15: Interceptors

› Kapitel 16: Diagnose und Logging

15 Interceptors

Interceptors sind Haken in EFs Laufzeit: vor und nach Befehlen, Verbindungen, Transaktionen, SaveChanges und der Materialisierung. Sie können beobachten, Werte ändern, Operationen unterdrücken oder Ergebnisse ersetzen.

Interface	Ereignisse	Lebensdauer
`ISaveChangesInterceptor`	SavingChanges, SavedChanges, SaveChangesFailed, ThrowingConcurrencyException	Scoped
`IDbConnectionInterceptor`	Öffnen, Schließen, Erzeugen der Verbindung	Scoped
`IDbCommandInterceptor`	Erzeugen und Ausführen von Befehlen, DataReader	Scoped
`IDbTransactionInterceptor`	Beginnen, Commit, Rollback, Savepoints	Scoped
`IIdentityResolutionInterceptor`	Konflikte beim Anhängen bereits getrackter Entitäten	Scoped
`IMaterializationInterceptor`	Erzeugen und Initialisieren von Entitäten	**Singleton**
`IQueryExpressionInterceptor`	LINQ-Ausdruck vor der Kompilierung	**Singleton**

Die beiden Singleton-Interceptors implementieren `ISingletonInterceptor`. Sie werden einmal pro internem Provider verwendet, nicht pro Context, und müssen deshalb zustandslos und thread-sicher sein.

Zwei Wege der Registrierung

Weg	Eigenschaften	Wann
`optionsBuilder.AddInterceptors(...)`	Instanzen aus der Anwendung; können App-Dienste nutzen; pro Context verschieden	Interceptors mit App-Abhängigkeiten (Mandant, Benutzer, Uhr)
`services.AddScoped<IInterceptor, X>()` in `ApplyServices`	Im internen Provider aufgelöst („injected interceptors“); haben Zugriff auf EF-Dienste, nicht auf App-Dienste	Rein technische Interceptors ohne App-Abhängigkeit

Beide Wege kombiniert EF zu einer Liste. Das Beispiel nutzt den ersten, weil beide Interceptors den Mandanten der Anwendung brauchen (Kapitel 6).

Der SaveChanges-Interceptor

Er setzt die `TenantId` neuer Entitäten und verhindert, dass Daten eines anderen Mandanten geschrieben oder umgehängt werden. Das ergänzt RLS (Defense in Depth) und liefert verständlichere Fehler als die Datenbank. Kapitel 27 zeigt, welche Wege an ihm vorbeiführen:

`Interceptors/TenantSaveChangesInterceptor.cs (1/2)`

```
public class TenantSaveChangesInterceptor : SaveChangesInterceptor
{
    private readonly ITenantProvider _tenants;

    public TenantSaveChangesInterceptor(ITenantProvider tenants)
```

```
    {
        ArgumentNullException.ThrowIfNull(tenants);

        _tenants = tenants;
    }

    public override InterceptionResult<int> SavingChanges(
        DbContextEventData eventData, InterceptionResult<int> result)
    {
        if (eventData.Context is { } context) ApplyTenant(context);
        return result;
    }

    public override ValueTask<InterceptionResult<int>> SavingChangesAsync(
        DbContextEventData eventData, InterceptionResult<int> result,
        CancellationToken cancellationToken = default)
    {
        if (eventData.Context is { } context) ApplyTenant(context);
        return ValueTask.FromResult(result);
    }
```

Interceptors/TenantSaveChangesInterceptor.cs (2/2, vereinfacht)

```
    internal void ApplyTenant(DbContext context)
    {
        // Welche Entitäten betroffen sind, steht in der Laufzeit-Annotation
        // (Kapitel „Fluent API auf Property-Ebene und Laufzeit-Annotations“)
        var changes = context.ChangeTracker.Entries()
            .Where(e => e.State is EntityState.Added or EntityState.Modified
                                or EntityState.Deleted)
            .Select(e => (Entry: e, Name: e.Metadata.GetTenantPropertyName()))
            .Where(x => x.Name is not null)
            .Select(x => (x.Entry, Property: x.Entry.Property(x.Name!)))
            .ToList();
        if (changes.Count == 0) return;

        var current = _tenants.CurrentTenantId
            ?? throw new TenancyException("Kein aktiver Mandant ...");

        foreach (var (entry, property) in changes)
        {
            var currentValue = (TenantId)property.CurrentValue!;
            var originalValue = (TenantId)property.OriginalValue!;
            switch (entry.State)
            {
                case EntityState.Added when currentValue.IsEmpty:
                    property.CurrentValue = current;
                    break;
                case EntityState.Added when currentValue != current:
                    throw Mismatch(entry, currentValue, current);
                case EntityState.Modified when property.IsModified:
                    throw new TenancyException("Mandanten-Property ist unveränderlich.");
                case EntityState.Modified or EntityState.Deleted
                    when originalValue != current:
                    throw Mismatch(entry, originalValue, current);
            }
        }
    }
```

```
}
```

> **Synchron und asynchron**
>
> Immer **beide** Varianten implementieren, synchron und asynchron. EF ruft je nach `SaveChanges` oder `SaveChangesAsync` nur eine davon auf. Ein Interceptor, der nur die asynchrone Methode überschreibt, ist bei synchronen Aufrufen wirkungslos, ohne dass es jemand bemerkt.

Die Logik liegt in einer `internal`-Methode `ApplyTenant`. Tests können sie über `InternalsVisibleTo` direkt aufrufen, ohne eine Datenbank zu brauchen (Kapitel 23).

Der Connection-Interceptor

Er setzt bei jedem Öffnen der Verbindung das Session-Setting, das die Policies und `CurrentTenant()` auswerten. Der Ablauf (Öffnen abfangen, sync und async, Mandant holen) ist für jede Datenbank gleich und liegt deshalb in einer abstrakten Basis im Kernpaket (Kapitel 41). Das PostgreSQL-Paket liefert nur den Befehl. Den Setting-Namen holt es über den Context aus den Event-Daten (`context.GetService<T>()`) aus dem *internen* Provider, den Mandanten aus dem *App*-Container:

`Interceptors/TenantSessionInterceptor.cs (Kern), Runtime/TenantConnectionInterceptor.cs`

```
// Kernpaket
public abstract class TenantSessionInterceptor : DbConnectionInterceptor
{
    protected TenantSessionInterceptor(ITenantProvider tenants)
    {
        ArgumentNullException.ThrowIfNull(tenants);

        Tenants = tenants;
    }

    protected ITenantProvider Tenants { get; }

    public override async Task ConnectionOpenedAsync(
        DbConnection connection, ConnectionEndEventData eventData,
        CancellationToken cancellationToken = default)
    {
        await using var command =
            CreateSessionCommand(connection, eventData.Context, Tenants.CurrentTenantId);
        await command.ExecuteNonQueryAsync(cancellationToken);
    }

    // ConnectionOpened (synchron) analog

    protected abstract DbCommand CreateSessionCommand(
        DbConnection connection, DbContext? context, TenantId? tenant);
}

// PostgreSQL-Paket
public class TenantConnectionInterceptor : TenantSessionInterceptor
{
    public TenantConnectionInterceptor(ITenantProvider tenants)
        : base(tenants)
    {
    }
```

```
    protected override DbCommand CreateSessionCommand(
        DbConnection connection, DbContext? context, TenantId? tenant)
    {
        var settingName = context?.GetService<ITenancySingletonOptions>()
                              .Settings.SettingName
                          ?? TenancySettings.Default.SettingName;

        var command = connection.CreateCommand();
        command.CommandText = "SELECT set_config(@name, @value, false)";
        AddParameter(command, "name", settingName);
        // Immer setzen, auch leer: kein alter Wert bleibt stehen
        AddParameter(command, "value", tenant?.ToString() ?? string.Empty);
        return command;
    }
}
```

Leerer String statt NULL

Npgsql setzt den Session-Zustand zurück, wenn eine Verbindung in den Pool zurückkehrt. Danach liefert `current_setting('app.tenant_id', true)` einen leeren String statt `NULL`. Ein `''::uuid` würde jede Abfrage abbrechen. Policy und Translator verwenden deshalb `nullif(..., '')`: Ohne Mandant ist der Wert `NULL`, der Vergleich nie wahr, und es sind keine Zeilen sichtbar. Das ist die sichere Richtung.

EF öffnet und schließt die Verbindung standardmäßig für jede Abfrage. Jedes Öffnen kostet hier einen zusätzlichen Roundtrip. Bei vielen kleinen Abfragen lohnt es sich, die Verbindung pro Anfrage einmal explizit zu öffnen (Kapitel 34).

Wo der Connection-Interceptor nicht läuft

Der Interceptor läuft, wenn EF eine Verbindung öffnet, und nur dann. Zwei Situationen umgehen ihn, und beide entstehen leicht aus Versehen:

Situation	Folge	Abhilfe
Eine bereits geöffnete Verbindung wird übergeben (`UseNpgsql(connection)`, `SetDbConnection`)	EF öffnet sie nie, also läuft der Interceptor nie. Die Session behält ihren bisherigen Wert: keinen oder den eines früheren Mandanten	Verbindungen geschlossen übergeben und EF öffnen lassen; sonst `Database.ApplyTenantSession()` aufrufen
Die Verbindung bleibt offen (`OpenConnection`, eine offene Transaktion), während der Mandant wechselt	Der neue Mandant wirkt erst, wenn die Verbindung erneut geöffnet wird	Ein Context, ein Mandant; ist das nicht möglich, nach dem Wechsel `Database.ApplyTenantSession()` aufrufen

tests/...IntegrationTests/ConnectionLifetimeTests.cs (gekürzt)

```
// Bereits offen, mit einem Session-Wert aus früherer Arbeit (Mandant B)
await using var connection = new NpgsqlConnection(appConnectionString);
await connection.OpenAsync();
// ... SELECT set_config('app.tenant_id', <tenant B>, false) ...

// Gedacht für Mandant A, aber EF öffnet die Verbindung nicht: kein Interceptor
await using var db = CreateAppContext(connection, tenantA);
var orders = await db.Orders.ToListAsync();       // die Zeilen von Mandant B!
```

Für diese Fälle bietet das Plugin das explizite Gegenstück zum Interceptor. `Database.ApplyTenantSession()` schreibt den aktuellen Mandanten in die Session, wenn die Verbindung offen ist, und tut nichts, wenn sie geschlossen ist, denn dann öffnet EF sie und der Interceptor übernimmt. Die Methode verwendet den Befehl des Interceptors selbst und funktioniert deshalb mit jedem Provider-Paket:

Verwendung

```
await using var db = CreateAppContext(connection, tenantA);   // Verbindung bereits offen
await db.Database.ApplyTenantSessionAsync();                   // Session jetzt: Mandant A
var orders = await db.Orders.ToListAsync();                    // die Zeilen von Mandant A
```

Interceptors/TenantSessionInterceptor.cs (Auszug)

```
public virtual async Task<bool> ApplyToOpenConnectionAsync(
    DbContext context, CancellationToken cancellationToken = default)
{
    var connection = context.Database.GetDbConnection();
    if (connection.State != ConnectionState.Open)
        return false;                    // EF öffnet sie später; der Interceptor übernimmt

    await using var command =
        CreateSessionCommand(connection, context, Tenants.CurrentTenantId);
    // In einer expliziten Transaktion muss der Befehl ihr zugeordnet sein
    command.Transaction = context.Database.CurrentTransaction?.GetDbTransaction();
    await command.ExecuteNonQueryAsync(cancellationToken);
    return true;
}
```

Die Erweiterungsmethode `ApplyTenantSession` im Kernpaket sucht den Interceptor unter den Interceptoren des Contexts und bricht mit einer klaren Meldung ab, wenn keiner registriert ist. Zwei Details sind wichtig: Die Methode braucht den Context, nicht nur die Verbindung, weil der Provider seine Einstellungen aus dem internen Service Provider liest; deshalb erhält `CreateSessionCommand` den `DbContext` statt der Event-Daten. Und der Befehl muss sich einer laufenden Transaktion anschließen: PostgreSQL ist hier nachsichtig, SQL Server lehnt einen Befehl ab, der ihr nicht zugeordnet ist. Die SQL-Server-Tests prüfen genau diesen Fall; ohne die Zeile schlagen sie fehl.

> **Ein Context, ein Mandant**
>
> Die Isolation folgt der Verbindung, nicht dem Context. Ein Context, der für mehrere Mandanten verwendet wird, oder eine Verbindung, die eine Anfrage überdauert, trägt den Mandanten des Moments, in dem sie geöffnet wurde. `ConnectionLifetimeTests` zeigt beide Fälle gegen eine echte Datenbank, zusammen mit den beiden Abhilfen: einer geschlossenen Verbindung, die EF selbst öffnet, und `ApplyTenantSession()`. Ein Plugin, dessen Hooks nicht jeden Weg abdecken, sollte eine solche explizite API anbieten, statt die Lücke nur zu dokumentieren.

16 Diagnose und Logging

Ein professionelles Plugin meldet Ereignisse über EFs Logging-System. Dann landen die Meldungen in denselben Kanälen, die Nutzer ohnehin konfigurieren (`ILogger`, `LogTo`, `DiagnosticSource`), und Warnungen lassen sich per `ConfigureWarnings` ignorieren, protokollieren oder zu Fehlern machen.

Event-IDs

Diagnostics/TenancyEventId.cs (gekürzt)

```
public static class TenancyEventId
{
    public const int TenancyBaseId = 1_150_000;     // eigener, dokumentierter Bereich

    private enum Id
    {
        TenantIsolationApplied = TenancyBaseId,
        TenantIsolationOptedOut,
        PoliciesWithoutRowLevelSecurity,
    }

    private static readonly string ModelPrefix = "Acme.EntityFrameworkCore.Tenancy.Model.";

    public static readonly EventId TenantIsolationApplied =
        Make(Id.TenantIsolationApplied);
    public static readonly EventId TenantIsolationOptedOut =
        Make(Id.TenantIsolationOptedOut);
    // ...

    private static EventId Make(Id id) => new((int)id, ModelPrefix + id);
}
```

EF und Provider belegen niedrige Bereiche. Ein Plugin wählt einen eigenen, hohen Bereich und dokumentiert ihn. Die Namen folgen EFs Schema „Assembly.Kategorie.Ereignis“, damit Filter nach Präfix funktionieren.

Event-Definitionen

Eine `EventDefinition<T>` bündelt Event-ID, Log-Level, Meldungsvorlage und das Warnverhalten aus den Options. EF cached seine Definitionen pro internem Provider; das Plugin macht es genauso mit einer eigenen Klasse, die als Singleton registriert ist (Kapitel 5):

Diagnostics/TenancyLoggingDefinitions.cs (Auszug)

```
public class TenancyLoggingDefinitions
{
    private EventDefinition<string>? _tenantIsolationOptedOut;

    public virtual EventDefinition<string> LogTenantIsolationOptedOut(
        IDiagnosticsLogger logger)
        => LazyInitializer.EnsureInitialized(ref _tenantIsolationOptedOut, () =>
            Create(logger, TenancyEventId.TenantIsolationOptedOut, LogLevel.Warning,
                nameof(TenancyEventId.TenantIsolationOptedOut),
```

```
                "Entität '{entityType}' implementiert ITenantOwned, ist aber von der "
                + "Mandantenisolation ausgenommen."));

    private static EventDefinition<string> Create(
        IDiagnosticsLogger logger, EventId eventId, LogLevel level,
        string name, string message)
        => new(
            logger.Options,                      // enthält das Warnverhalten
            eventId,
            level,
            "TenancyEventId." + name,            // steht in Warning-as-Error-Meldungen
            l => LoggerMessage.Define<string>(l, eventId, message));
}
```

Die Logger-Extension

Die eigentliche Log-Methode folgt exakt dem Muster aus EFs `CoreLoggerExtensions`: erst klassisches Logging, dann die Ereignisdaten für DiagnosticSource und `LogTo`:

Diagnostics/TenancyLoggerExtensions.cs (Auszug)

```
private static void LogEntityType(
    IDiagnosticsLogger<DbLoggerCategory.Model> diagnostics,
    EventDefinition<string> definition,
    IReadOnlyEntityType entityType)
{
    // 1. ILogger; wirft bei ConfigureWarnings(w => w.Throw(...))
    if (diagnostics.ShouldLog(definition))
        definition.Log(diagnostics, entityType.DisplayName());

    // 2. DiagnosticSource und LogTo (simple logging)
    if (diagnostics.NeedsEventData(definition, out var diagnosticSource, out var simpleLog))
    {
        var eventData = new EntityTypeEventData(definition, GenerateMessage, entityType);
        diagnostics.DispatchEventData(definition, eventData, diagnosticSource, simpleLog);
    }
}
```

Nutzer können die Warnung damit in einen Fehler verwandeln:

Anwendung

```
options.UseNpgsql(cs)
       .UseAcmeTenancy()
       .ConfigureWarnings(w => w.Throw(TenancyEventId.TenantIsolationOptedOut));

// -> InvalidOperationException beim Modellbau:
//    "An error was generated for warning '...TenantIsolationOptedOut': Entität
//     'AuditEntry' ... This exception can be suppressed or logged by passing event ID
//     'TenancyEventId.TenantIsolationOptedOut' to the 'ConfigureWarnings' method ..."
```

> **Tipp**
>
> Sicherheitsrelevante Abweichungen gehören als **Warnung** ins Log und nicht als Debug-Meldung. Teams mit strengen Anforderungen eskalieren sie per `ConfigureWarnings` zu Fehlern; andere sehen sie zumindest beim Start. Das Plugin muss diese Entscheidung nicht für alle treffen.

Teil VI

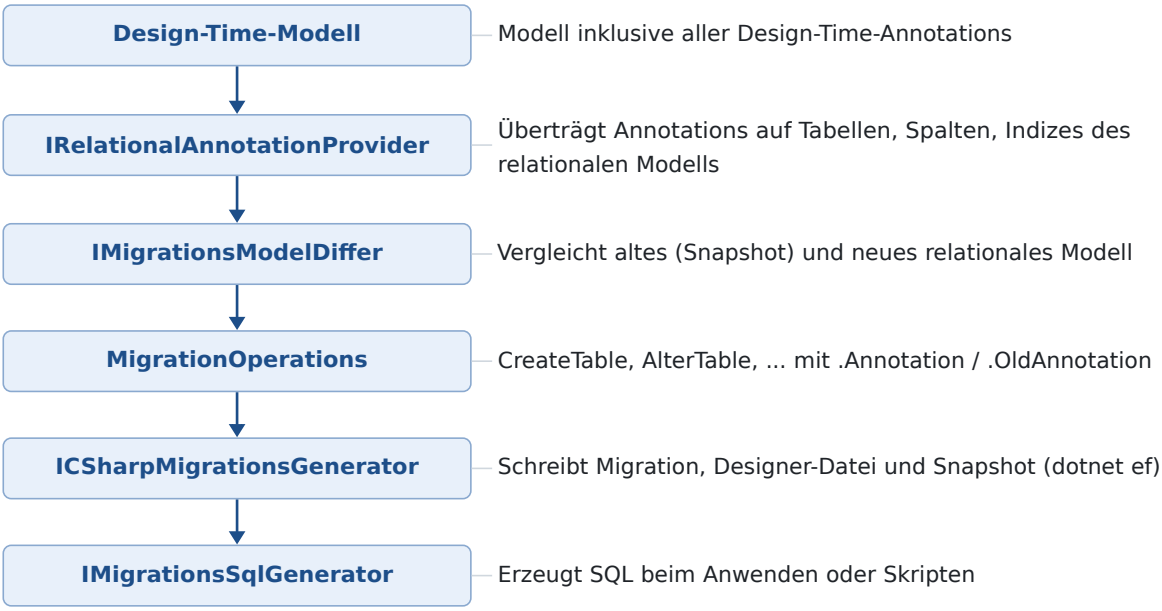
Migrations und Design-Time

Schema-Änderungen erzeugen, die Werkzeuge von dotnet ef erweitern und Compiled Models unterstützen.

- Kapitel 17: Wie Migrations funktionieren
- Kapitel 18: Deklaratives SQL aus Annotations
- Kapitel 19: Eigene Migrations-Operationen
- Kapitel 20: Design-Time-Services
- Kapitel 21: Compiled Models, Trimming und NativeAOT
- Kapitel 22: Scaffolding (Reverse Engineering)

17 Wie Migrations funktionieren

Migrations vergleichen zwei Modelle und erzeugen daraus Operationen und SQL. Ein Plugin kann sich an mehreren Stellen einklinken; um die richtige zu wählen, muss man den Ablauf kennen:



Für Plugins gibt es zwei grundsätzliche Strategien. **Deklarativ**: Das Plugin beschreibt den gewünschten Zustand als Annotations; EF erkennt Änderungen von selbst, und das Plugin muss nur SQL erzeugen. **Imperativ**: Das Plugin bietet eigene Operationen an, die Nutzer von Hand in Migrationen schreiben. Gute Plugins nutzen beide: deklarativ für alles, was zum Modell gehört, imperativ für alles andere.

> **Hinweis**
>
> `Database.EnsureCreated()` nutzt dieselbe Pipeline: Es berechnet den Diff vom leeren Zustand zum Design-Time-Modell und führt das SQL des Migrations-Generators aus. Eigene Migrations-SQL eines Plugins wirkt also auch dort, was Integrationstests erheblich vereinfacht (Kapitel 23).

18 Deklaratives SQL aus Annotations

Der Annotation-Provider

Der Differ vergleicht das *relationale* Modell (Tabellen, Spalten), nicht die Entity-Typen. Welche Annotations eine Tabelle trägt, entscheidet der `IRelationalAnnotationProvider`. Den gibt es pro Context einmal, und er gehört dem Provider; das Plugin erbt deshalb von der Npgsql-Implementierung:

Metadata/Internal/TenancyAnnotationProvider.cs

```
public class TenancyAnnotationProvider : NpgsqlAnnotationProvider
{
    public TenancyAnnotationProvider(RelationalAnnotationProviderDependencies dependencies)
        : base(dependencies)
    {
    }

    public override IEnumerable<IAnnotation> For(ITable table, bool designTime)
    {
        foreach (var annotation in base.For(table, designTime))
            yield return annotation;

        if (!designTime)          // nur für Migrations: Laufzeitmodell bleibt schlank
            yield break;

        var entityType = (IEntityType)table.EntityTypeMappings.First().TypeBase;
        foreach (var annotation in entityType.GetAnnotations())
        {
            if (!annotation.Name.StartsWith(RlsAnnotationNames.Prefix,
                                            StringComparison.Ordinal))
                continue;
            // Opt-out-Marker nicht weitergeben, sonst entstehen leere Migrationen
            if (annotation.Name == RlsAnnotationNames.Enabled && annotation.Value is false)
                continue;
            yield return annotation;
        }
    }
}
```

Ab hier übernimmt EF: Ändert sich eine Policy, erzeugt der Differ eine `AlterTableOperation` mit neuen und alten Annotations. In der generierten Migration sieht das so aus, ganz ohne eigenen Code-Generator:

Generierte Migration (Ausschnitt)

```
migrationBuilder.AlterTable(name: "Orders")
    .Annotation("Acme:Rls:Policy:tenant_isolation", "{...neu...}")
    .OldAnnotation("Acme:Rls:Policy:tenant_isolation", "{...alt...}");
```

Der SQL-Generator

Auch den `IMigrationsSqlGenerator` gibt es nur einmal. Das Plugin erbt vom Npgsql-Generator und ergänzt `CreateTable` und `AlterTable`:

Migrations/Internal/TenancyMigrationsSqlGenerator.cs (Auszug)

```
protected override void Generate(
    CreateTableOperation operation, IModel? model,
    MigrationCommandListBuilder builder, bool terminate = true)
{
    // Unterminiertes SQL kann keine weiteren Statements aufnehmen (wie bei Npgsql)
    if (!terminate && operation.HasRlsConfiguration())
        throw new ArgumentException(
            "RLS kann nicht in unterminiertem CREATE TABLE entstehen.");

    base.Generate(operation, model, builder, terminate);
    if (terminate)
        GenerateRowLevelSecurity(operation.Name, operation.Schema, previous: null,
            operation, builder);
}

protected override void Generate(
    AlterTableOperation operation, IModel? model, MigrationCommandListBuilder builder)
{
    base.Generate(operation, model, builder);
    GenerateRowLevelSecurity(operation.Name, operation.Schema, operation.OldTable,
        operation, builder);
}
```

Migrations/Internal/TenancyMigrationsSqlGenerator.cs (gekürzt)

```
protected virtual void GenerateRowLevelSecurity(string tableName, string? schema,
    IReadOnlyAnnotatable? previous, IReadOnlyAnnotatable current,
    MigrationCommandListBuilder builder)
{
    var table = Delimit(tableName, schema);
    var oldPolicies = previous?.GetRlsPolicies() ?? new Dictionary<string, string>();
    var newPolicies = current.GetRlsPolicies();

    // 1. Entfernte oder geänderte Policies löschen
    foreach (var (name, json) in oldPolicies)
        if (!newPolicies.TryGetValue(name, out var n) || n != json)
            Statement(builder, $"DROP POLICY {Delimit(name)} ON {table}");

    // 2. RLS-Status (ENABLE/DISABLE, FORCE/NO FORCE)
    if ((previous?.IsRlsEnabled() ?? false) != current.IsRlsEnabled())
        Statement(builder, $"ALTER TABLE {table} "
            + (current.IsRlsEnabled() ? "ENABLE" : "DISABLE") + " ROW LEVEL SECURITY");
    // ... FORCE analog

    // 3. Neue oder geänderte Policies anlegen
    foreach (var (name, json) in newPolicies)
        if (!oldPolicies.TryGetValue(name, out var o) || o != json)
            Statement(builder, CreatePolicySql(table, RlsPolicy.Deserialize(json)));
}

protected virtual void Statement(MigrationCommandListBuilder builder, string sql)
{
    builder.Append(sql).AppendLine(Dependencies.SqlGenerationHelper.StatementTerminator);
    EndStatement(builder);
}
```

Detail	Begründung
DROP + CREATE statt `ALTER POLICY`	`ALTER POLICY` kann weder `FOR` noch PERMISSIVE/RESTRICTIVE ändern. DDL ist in PostgreSQL transaktional; es entsteht keine sichtbare Lücke.
`EndStatement(builder)`	Beendet den Befehl wie die Basisklasse; berücksichtigt dabei Transaktions-unterdrückung.
Eigene `Delimit`-Helfer	Npgsqls `DelimitIdentifier`-Helfer sind privat. Bezeichner immer über `Dependencies.SqlGenerationHelper` maskieren, nie per String-Verkettung.
DropTable, RenameTable	Nichts zu tun: PostgreSQL entfernt bzw. verschiebt Policies mit der Tabelle.

19 Eigene Migrations-Operationen

Manches gehört nicht ins Modell, aber in Migrationen: Rollen, Rechte, Extensions, Daten-Backfills. Dafür bietet das Plugin eigene Operationen an. Sie bestehen aus drei Teilen: der Operationsklasse, einer Extension-Method auf `MigrationBuilder` und der SQL-Erzeugung im Generator.

Migrations/Operations/TenancyOperations.cs (Auszug)

```
public class CreateApplicationRoleOperation : MigrationOperation
{
    public required string Name { get; init; }
    public bool CanLogin { get; init; }       // Passwörter gehören NICHT in Migrationen
}
```

Migrations/Operations/TenancyMigrationBuilderExtensions.cs (Auszug)

```
namespace Microsoft.EntityFrameworkCore.Migrations;      // in Migrationen immer importiert

public static class TenancyMigrationBuilderExtensions
{
    public static OperationBuilder<CreateApplicationRoleOperation> CreateApplicationRole(
        this MigrationBuilder migrationBuilder, string name, bool canLogin = false)
    {
        var operation = new CreateApplicationRoleOperation
        {
            Name = TenancyIdentifiers.ValidateIdentifier(name, "Rollenname"),
            CanLogin = canLogin,
        };
        migrationBuilder.Operations.Add(operation);
        return new OperationBuilder<CreateApplicationRoleOperation>(operation);
    }
}
```

Der Generator verteilt Operationen über die zentrale `Generate(MigrationOperation, ...)`-Methode. Unbekannte Typen gehen an die Basisklasse:

Migrations/Internal/TenancyMigrationsSqlGenerator.cs (Auszug)

```
protected override void Generate(
    MigrationOperation operation, IModel? model, MigrationCommandListBuilder builder)
{
    switch (operation)
    {
        case CreateApplicationRoleOperation createRole: Generate(createRole, builder); break;
        case DropApplicationRoleOperation dropRole:     Generate(dropRole, builder); break;
        case GrantTablePrivilegesOperation grant:       Generate(grant, builder); break;
        default: base.Generate(operation, model, builder); break;
    }
}

protected virtual void Generate(CreateApplicationRoleOperation operation,
    MigrationCommandListBuilder builder)
{
    var literal = Literal(operation.Name);           // 'shop_app'  (für den Vergleich)
    var identifier = Delimit(operation.Name);    // shop_app (CREATE ROLE, ggf. in "...")
    var login = operation.CanLogin ? "LOGIN" : "NOLOGIN";
```

```
    builder
        .AppendLine("DO $$")
        .AppendLine("BEGIN")
        .AppendLine("    IF NOT EXISTS (SELECT FROM pg_catalog.pg_roles")
        .AppendLine($"                   WHERE rolname = {literal}) THEN")
        .AppendLine($"        CREATE ROLE {identifier} {login} NOSUPERUSER NOBYPASSRLS;")
        .AppendLine("    END IF;")
        .AppendLine("END")
        .Append("$$");
    builder.AppendLine(Dependencies.SqlGenerationHelper.StatementTerminator);
    EndStatement(builder);
}
```

Literal oder Bezeichner

Der Rollenname kommt zweimal vor, aber in zwei Formen: als **String-Literal** im Vergleich (`'shop_app'`, erzeugt über das String-Type-Mapping) und als **Bezeichner** in `CREATE ROLE` (`shop_app`, bei Bedarf in doppelten Anführungszeichen). Wer beides verwechselt oder per String-Verkettung baut, erzeugt ungültiges SQL oder eine Injection-Lücke. Zusätzlich validiert die Extension-Method den Namen.

Tipp

Operationen idempotent formulieren (`IF NOT EXISTS`, `DROP ... IF EXISTS`). Rollen existieren clusterweit und überleben Datenbanken; eine Migration, die beim zweiten Ausführen in einer neuen Datenbank scheitert, ist ein häufiger Ärger.

In einer Migration nutzt man die Operationen wie EFs eigene:

`Migrations/20261021_Initial.cs (handergänzt)`

```
protected override void Up(MigrationBuilder migrationBuilder)
{
    // VOR den Tabellen (Kapitel „Migrations für Fortgeschrittene“)
    migrationBuilder.CreateApplicationRole("shop_support");
    // ... von EF generierte Operationen (Tabellen, RLS über Annotations) ...
    migrationBuilder.CreateApplicationRole("shop_app", canLogin: true);
    migrationBuilder.GrantTablePrivileges("shop_app");
}

protected override void Down(MigrationBuilder migrationBuilder)
{
    migrationBuilder.DropApplicationRole("shop_app");
    // ...
}
```

Wenn der Differ eigene Operationen erzeugen soll

Sollen eigene Operationen vom *Differ* automatisch erzeugt werden, braucht es einen abgeleiteten `IMigrationsModelDiffer` und einen `ICSharpMigrationOperationGenerator`, der die Operation als C# schreibt. Beides sind Einzeldienste mit viel interner API. Fast immer ist der deklarative Weg über Annotations (Kapitel 18) die bessere Wahl.

20 Design-Time-Services

Die Werkzeuge von `dotnet ef` (Migrationen anlegen, Snapshots schreiben, Scaffolding, Compiled Models) verwenden einen eigenen Service Provider mit Design-Time-Diensten. Ein Plugin erweitert ihn über `IDesignTimeServices`.

Automatisch geladen

Damit Nutzer nichts konfigurieren müssen, verweist ein Assembly-Attribut auf die Klasse. Das Tooling findet es in jeder Assembly, die das Startprojekt referenziert. Das zweite Argument beschränkt das Laden auf Contexts mit dem passenden Provider:

Properties/AssemblyInfo.cs

```
[assembly: DesignTimeServicesReference(
    "Acme.EntityFrameworkCore.Tenancy.PostgreSQL.Design.TenancyDesignTimeServices, "
        + "Acme.EntityFrameworkCore.Tenancy.PostgreSQL",
    "Npgsql.EntityFrameworkCore.PostgreSQL")]
```

Design/TenancyDesignTimeServices.cs

```
public class TenancyDesignTimeServices : IDesignTimeServices
{
    public void ConfigureDesignTimeServices(IServiceCollection services)
    {
        // Alle Registrierungen entfernen (EF-Default, Npgsql), dann unsere setzen:
        // unabhängig von der Reihenfolge, in der das Tooling Services lädt
        services.RemoveAll<IAnnotationCodeGenerator>();
        services.AddSingleton<IAnnotationCodeGenerator, TenancyAnnotationCodeGenerator>();
    }
}
```

Lesbare Snapshots: der Annotation-Code-Generator

Ohne Zutun schreibt EF unbekannte Annotations als `HasAnnotation(...)` in den Snapshot, bei Policies mit JSON-Werten. Das funktioniert, ist aber kaum lesbar. Ein eigener `IAnnotationCodeGenerator` übersetzt sie in die Fluent API:

ShopContextModelSnapshot.cs

```
// Vorher
b.HasAnnotation("Acme:Rls:Enabled", true);
b.HasAnnotation("Acme:Rls:Policy:tenant_isolation", "{\"Name\":\"tenant_isolation\",...}");

// Nachher
RlsEntityTypeBuilderExtensions.HasRowLevelSecurity(b, true);
RlsEntityTypeBuilderExtensions.HasPolicy(b, "tenant_isolation", p => p
    .Using("\"TenantId\" = nullif(current_setting('app.tenant_id', true), '')::uuid")
    .WithCheck("\"TenantId\" = nullif(current_setting('app.tenant_id', true), '')::uuid"));
```

EF Core 10 schreibt Erweiterungsmethoden im Snapshot als statische Aufrufe (`Extensions.Methode(b, ...)`) statt als `b.Methode(...)`. Das vermeidet Mehrdeutigkeiten, wenn mehrere

Provider oder Plugins gleichnamige Methoden definieren. Tests, die den erzeugten Code prüfen, sollten diese Form erwarten.

Design/Internal/TenancyAnnotationCodeGenerator.cs (Auszug)

```
public override IReadOnlyList<MethodCallCodeFragment> GenerateFluentApiCalls(
    IEntityType entityType, IDictionary<string, IAnnotation> annotations)
{
    var calls = new List<MethodCallCodeFragment>();

    // VERTRAG: Übersetzte Annotations aus dem Dictionary ENTFERNEN
    if (annotations.TryGetValue(RlsAnnotationNames.Enabled, out var enabled)
        && enabled.Value is true)
    {
        annotations.Remove(RlsAnnotationNames.Enabled);
        var forced = annotations.Remove(RlsAnnotationNames.Forced, out var f)
                     && f.Value is true;
        calls.Add(new MethodCallCodeFragment(HasRowLevelSecurityMethod, forced));
    }

    foreach (var key in annotations.Keys
                 .Where(k => k.StartsWith(RlsAnnotationNames.PolicyPrefix,
                                          StringComparison.Ordinal))
                 .ToList())                          // Kopie: Dictionary ändert sich
    {
        var policy = RlsPolicy.Deserialize((string)annotations[key].Value!);
        annotations.Remove(key);
        calls.Add(new MethodCallCodeFragment(HasPolicyMethod, policy.Name,
            new NestedClosureCodeFragment("p", BuildPolicyChain(policy))));
    }

    calls.AddRange(base.GenerateFluentApiCalls(entityType, annotations));
    return calls;
}
```

Stolperstelle	Lösung
Übersetzte Annotation bleibt im Dictionary	Erscheint doppelt im Snapshot. Immer entfernen; was übrig bleibt, schreibt EF als `HasAnnotation`.
Generische Methode im `MethodInfo`	Der Snapshot nutzt den nicht-generischen `EntityTypeBuilder`. Nicht-generische Überladung per `GetRuntimeMethod` mit exakten Parametertypen wählen.
`params`-Argument als Array	`MethodCallCodeFragment` nimmt `params object?[]`. Ein `string[]` würde wegen Array-Kovarianz zur Argumentliste: in `new object?[] { roles }` verpacken.
Enums im Snapshot	Werden als `PolicyCommand.Select` gerendert. Das Enum liegt deshalb im immer importierten Namespace `Microsoft.EntityFrameworkCore`.
Lambda-Argumente	`NestedClosureCodeFragment("p", kette)` erzeugt `p => p.Using(...)`; Ketten entstehen mit `fragment.Chain(...)`.

Der Generator betrifft nur Snapshot und Designer-Dateien. Die eigentlichen Migrationen behalten `.Annotation(...)` an `AlterTable`; das ist der vorgesehene Mechanismus für Änderungen.

21 Compiled Models, Trimming und NativeAOT

`dotnet ef dbcontext optimize` erzeugt ein **Compiled Model**: C#-Code, der das Laufzeitmodell direkt aufbaut, statt es beim Start über Conventions zu berechnen. Das verkürzt die Startzeit großer Modelle deutlich und ist Voraussetzung für NativeAOT. Ein Plugin muss dafür an vier Stellen vorbereitet sein:

Baustein	Anforderung	Im Beispiel
Type Mappings	Öffentlicher Typ mit öffentlicher statischer Property `Default` (für NativeAOT genutzt); Konverter/Comparer als gemeinsame Instanzen	`TenantIdTypeMapping.Default`, `TenantIdConverter.Instance`
JSON-Reader/Writer	Statische `Instance` und `Constructor Expression`	`TenantIdJsonReaderWriter`
Value Generators	Konfiguriert mit `HasValueGenerator Factory(Type)`; Factory mit öffentlichem parameterlosem Konstruktor	`TenantIdValueGeneratorFactory` (Kapitel 28)
Annotations	Einfache Werte übernimmt EF automatisch; ein `ICSharpRuntimeAnnotationCode Generator` filtert Design-Time-Annotations und übersetzt komplexe Werte	Filtert RLS-Policies, behält die Laufzeit-Annotation (Kapitel 39)

Der dritte Punkt wird oft missverstanden. EF übernimmt Annotations mit einfachen Werten (Strings, Zahlen, bool, Enums) **automatisch** ins Compiled Model; das zeigt der Quellcode von `CSharpRuntimeAnnotationCodeGenerator`, der sie mit `GenerateSimpleAnnotations` schreibt. Ein eigener Generator ist deshalb nicht nötig, damit Annotations *erhalten* bleiben, sondern um reine Design-Time-Annotations *herauszufiltern*, die zur Laufzeit niemand liest, und um Annotations mit komplexen Werten in Code zu übersetzen. Kapitel 39 zeigt beides im Beispiel.

Trimming und AOT

Beide Pakete setzen `<IsAotCompatible>true</IsAotCompatible>`. Der Compiler meldet dann Code, der Reflection ohne Annotationen nutzt. Die JSON-Serialisierung der Policies verwendet einen Source-Generator statt Reflection:

Metadata/RlsPolicy.cs (Auszug)

```
[JsonSourceGenerationOptions(UseStringEnumConverter = true)]
[JsonSerializable(typeof(RlsPolicy))]
internal partial class RlsJsonContext : JsonSerializerContext;
```

> **Hinweis**
>
> Conventions, Snapshots und Migrations laufen nur zur Design-Zeit oder beim Modellbau; mit Compiled Model und vorkompilierten Abfragen fallen sie zur Laufzeit weg. Für AOT kritisch sind deshalb vor allem Type Mappings, Konverter, Translators (laufen beim Build) und Interceptors.

22 Scaffolding (Reverse Engineering)

`dotnet ef dbcontext scaffold` liest ein bestehendes Datenbankschema und erzeugt Entitäten und einen Context. Ein Plugin, das eigene Schema-Elemente verwaltet, kann sie dabei ebenfalls erkennen. Der Weg führt über zwei Design-Time-Dienste:

Dienst	Aufgabe
`IDatabaseModelFactory`	Liest das Schema in ein `DatabaseModel`. Ein abgeleiteter Npgsql-Factory könnte `pg_policies` und `pg_class.relrowsecurity` abfragen und als Annotations an `DatabaseTable` hängen.
`IAnnotationCodeGenerator`	Schreibt diese Annotations als Fluent API in den generierten `OnModelCreating`, mit derselben Logik wie beim Snapshot.

Das Beispiel implementiert beides: Eine eigene Database-Model-Factory liest RLS-Status und Policies aus der Datenbank, und der Annotation-Code-Generator aus Kapitel 20 schreibt sie in den gescaffoldeten Context. Kapitel 40 zeigt die Umsetzung im Detail.

Teil VII

Qualität und Auslieferung

Testen, versionieren, paketieren, und die Checkliste für den Weg von A bis Z.

- Kapitel 23: Teststrategie
- Kapitel 24: Versionierung und Kompatibilität
- Kapitel 25: Packaging und Dokumentation
- Kapitel 26: Die Checkliste von A bis Z

23 Teststrategie

Plugins lassen sich erstaunlich weit ohne Datenbank testen. Modellbau, Type Mappings, SQL-Übersetzung, Migrations-SQL und Snapshots brauchen nur einen Connection-String, keine Verbindung. Die Teststrategie des Beispiels hat vier Ebenen:

Ebene	Prüft	Datenbank
Modelltests	Annotations, Conventions, Validierung, Optionen, Caching	nein
SQL-Tests	Übersetzungen per `ToQueryString`, Migrations-SQL per Differ und Generator	nein
Snapshot-Tests	Code-Generator als String und als Roundtrip (Roslyn)	nein
Integrationstests	Echtes Verhalten: RLS, Rollen, Interceptors (Testcontainers)	ja (Docker)

Testhilfen

Tests bauen viele Contexts mit unterschiedlichem `OnModelCreating`. Weil EF das Modell pro internem Provider cached, muss jeder Test einen eigenen Provider bekommen; dafür gibt es `EnableServiceProviderCaching(false)`:

tests/.../TestModel.cs (gekürzt)

```
public static TestContext Create(
    Action<TenancyOptionsBuilder>? tenancy = null,
    Action<ModelBuilder>? model = null,
    bool cacheServiceProvider = false,
    Action<DbContextOptionsBuilder>? configure = null)
{
    var builder = new DbContextOptionsBuilder<TestContext>()
        .UseNpgsql("Host=localhost;Database=unused")       // keine Verbindung nötig
        .UseAcmeTenancy(tenancy)
        .EnableServiceProviderCaching(cacheServiceProvider);
    configure?.Invoke(builder);
    return new TestContext(builder.Options, model);
}

public static IReadOnlyList<string> MigrationSql(DbContext? source, DbContext target)
{
    var operations = target.GetService<IMigrationsModelDiffer>().GetDifferences(
        source?.GetService<IDesignTimeModel>().Model.GetRelationalModel(),
        target.GetService<IDesignTimeModel>().Model.GetRelationalModel());

    return target.GetService<IMigrationsSqlGenerator>()
        .Generate(operations, target.GetService<IDesignTimeModel>().Model)
        .Select(c => c.CommandText.Trim()).ToList();
}
```

Design-Time-Modell verwenden

Für Migrations-Tests immer das **Design-Time-Modell** verwenden (`IDesignTimeModel`). `context.Model` ist das schlanke Laufzeitmodell; Annotations, die der Annotation-Provider nur zur Design-Zeit weitergibt, fehlen darin.

> **Warum nicht SQLite oder InMemory?**
>
> Tests gegen SQLite oder den InMemory-Provider sind verlockend, weil sie kein Docker brauchen. Für ein Plugin beweisen sie wenig: InMemory ist nicht relational, Type Mappings, Translators und Migrationen laufen also nie; SQLite hat einen anderen SQL-Dialekt, andere Type Mappings und keine Row Level Security. Ein provider-spezifisches Plugin wird mit seinem eigenen Provider getestet: ohne Datenbank (nur Connection-String) für alles bis zum SQL-Text, gegen die echte Datenbank für das Verhalten.

Was getestet wird

Test	Sichert ab
Caching mit verschiedenen Optionen	Zwei Contexts mit verschiedenen Settings bekommen verschiedene Modelle (Hash-Fehler aus Kapitel 4).
Ungültige Setting-Namen	Validierung inklusive Injection-Versuch.
Type Mapping ohne Konfiguration	`TenantId` wird `uuid`; Literal-Format; anderer Spaltentyp wird nicht beansprucht.
`CurrentTenant()` per `ToQueryString`	Übersetzung und dass nichts clientseitig ausgewertet wird (Kapitel 13).
Explizit schlägt Convention	Eigene Policy gleichen Namens bleibt erhalten; Fluent API überstimmt das Attribut.
Warning-as-Error	Opt-out-Warnung wird mit `ConfigureWarnings` zum Fehler.
Validierung	RLS am abgeleiteten Typ und ungültige Policies werfen beim Modellbau.
Keine Phantom-Migration	Unverändertes Modell erzeugt keine einzige Operation.
Migrations-SQL	Initial, Änderung (DROP + CREATE), Opt-out, eigene Operationen.
SaveChanges-Logik	Setzen, fremder Mandant, Umhängen, kein Mandant.
Snapshot	Fluent API statt Annotations; Roundtrip ohne Unterschiede.
RLS in PostgreSQL	Isolation, keine Zeilen ohne Mandant, WITH CHECK greift.
Grenzfälle (Kapitel 27–32)	ExecuteUpdate und Raw SQL gegen RLS, Value Generation, JSON-Spalten, Compiled Model, idempotentes Skript.
Kombinierbarkeit	Decorator um den Migrations-Generator (Kapitel 5), EFCore.NamingConventions in beiden Reihenfolgen (Kapitel 8).
Annotation-Format	Serialisierte Policies bleiben identisch; tolerantes Lesen (Kapitel 24).
Context-Lebensdauern	DbContext-Factory, Hintergrundjobs, offene Verbindungen (Kapitel 15, 35).
Mandantenübergreifender Zugriff	Reporting-Rolle liest alle Mandanten, kann aber nicht schreiben; die App-Rolle kann RLS nicht abschalten (Kapitel 33).

Der Snapshot-Roundtrip

Der wertvollste Test für Design-Time-Code: Snapshot generieren, mit Roslyn kompilieren, Modell daraus bauen und gegen das Original diffen. Jede Abweichung zwischen Code-Generator und Fluent API würde sonst bei jedem Nutzer eine Phantom-Migration erzeugen:

`tests/.../SnapshotTests.cs (Auszug)`

```
[Fact]
public void Snapshot_roundtrip_produces_no_differences()
{
    using var ctx = CreateContext();
    var snapshot = Compile(GenerateSnapshot(ctx));

    // Wie im EF-Migrator: finalisieren und initialisieren
    var model = snapshot.Model;
    if (model is IMutableModel mutable) model = mutable.FinalizeModel();
    model = ctx.GetService<IModelRuntimeInitializer>()
        .Initialize(model, designTime: true, validationLogger: null);

    var differences = ctx.GetService<IMigrationsModelDiffer>().GetDifferences(
        model.GetRelationalModel(), ctx.DesignModel().GetRelationalModel());

    Assert.Empty(differences);
}
```

Integrationstests mit Testcontainers

Das echte Sicherheitsverhalten lässt sich nur gegen PostgreSQL prüfen. Die Fixture startet einen Container, legt das Schema mit `EnsureCreated` an (inklusive RLS, Kapitel 17), führt die eigenen Migrations-Operationen real aus und verbindet die Tests mit einer eingeschränkten Rolle:

tests/...IntegrationTests/PostgresFixture.cs (Auszug)

```
await owner.Database.EnsureCreatedAsync();          // RLS, Policies, Index

var commands = owner.GetService<IMigrationsSqlGenerator>().Generate(
[
    new CreateApplicationRoleOperation { Name = AppRole, CanLogin = true },
    new GrantTablePrivilegesOperation { Role = AppRole },
]);
await owner.GetService<IMigrationCommandExecutor>().ExecuteNonQueryAsync(
    commands, owner.GetService<IRelationalConnection>(),
    new MigrationExecutionState(), commitTransaction: true);
```

tests/...IntegrationTests/RowLevelSecurityTests.cs (Auszug)

```
[Fact]
public async Task Database_rejects_rows_of_other_tenant()
{
    // Interceptor umgehen: direktes SQL. WITH CHECK der Policy muss greifen.
    var tenants = new MutableTenantProvider { CurrentTenantId = TenantId.New() };
    await using var db = _fixture.CreateAppContext(tenants);

    var ex = await Assert.ThrowsAsync<PostgresException>(() => db.Database.ExecuteSqlAsync(
        $"INSERT INTO \"Orders\" (\"TenantId\", \"Total\") VALUES ({Guid.NewGuid()}, 1)"));

    Assert.Equal(PostgresErrorCodes.InsufficientPrivilege, ex.SqlState);
}
```

> **Tipp**
>
> Für Provider und sehr tiefe Plugins stellt EF Core die **Specification Tests** bereit (`Microsoft.EntityFrameworkCore.Relational.Specification.Tests`). Das ist dieselbe Testsuite, gegen die EF und die

> Provider selbst getestet werden. Das NetTopologySuite-Plugin im EF-Repository zeigt, wie ein Plugin sie nutzt.

24 Versionierung und Kompatibilität

Versionen an EF Core ausrichten

Die Major-Version des Plugins folgt der von EF Core: Version 10.x für EF Core 10. Nutzer erkennen sofort, welche Kombination passt, und das Plugin kann mit jeder EF-Major-Version Breaking Changes übernehmen. Die Abhängigkeiten werden auf diese Major-Version begrenzt:

Directory.Packages.props (Auszug)

```
<PackageVersion Include="Microsoft.EntityFrameworkCore" Version="[10.0.0,11.0.0)" />
<PackageVersion Include="Npgsql.EntityFrameworkCore.PostgreSQL" Version="[10.0.0,11.0.0)" />
```

Umgang mit interner API

Plugins kommen um interne APIs selten herum: Das Beispiel erbt von `NpgsqlMigrationsSqlGenerator`, `NpgsqlAnnotationProvider` und `NpgsqlAnnotationCodeGenerator` und nutzt `NpgsqlOptionsExtension`. Solche Typen können sich auch in Minor-Versionen ändern. EF meldet ihre Verwendung mit der Warnung **EF1001**.

Maßnahme	Wirkung
Öffentliche Alternative suchen	Jede vermiedene interne API ist ein Risiko weniger.
Interne Zugriffe bündeln	Wenige Klassen kapseln sie; ein Bruch betrifft eine Stelle.
`<NoWarn>EF1001</NoWarn>` bewusst setzen	Projektweit und dokumentiert, nicht pro Zeile verstreut.
Versionsbereich begrenzen	Verhindert, dass Nutzer ungetestete Kombinationen bekommen.
CI gegen Previews	Ein Build gegen die nächste EF-Preview zeigt Brüche Monate früher.
Roundtrip- und SQL-Tests	Bemerken Verhaltensänderungen, die der Compiler nicht sieht.

> **Multi-Targeting**
>
> Mehrere EF-Major-Versionen gleichzeitig zu unterstützen ist per Multi-Targeting möglich, verdoppelt aber Testaufwand und bedingte Kompilierung. Für die meisten Plugins ist ein Release-Zweig pro EF-Major-Version einfacher.

Annotation-Formate weiterentwickeln

Alles, was ein Plugin in Annotations schreibt, landet in den Model-Snapshots seiner Anwender, oft lange vor dem nächsten Upgrade. Serialisiert eine neue Plugin-Version dieselbe Konfiguration anders (eine zusätzliche JSON-Property, eine andere Reihenfolge, eine andere Enum-Schreibweise), vergleicht der Differ zwei verschiedene Strings. Jeder Anwender bekommt dann eine Migration, die alle Policies ohne echte Änderung löscht und neu anlegt. Das serialisierte Format gehört deshalb genauso zur öffentlichen API wie die Methoden.

Regel	Wie
Format per Test festschreiben	Ein Golden-Value-Test vergleicht die serialisierte Form Zeichen für Zeichen.
Tolerant lesen	Fehlende Properties werden zu Defaults, unbekannte werden ignoriert (das Standardverhalten von System.Text.Json).
Nur ergänzen, Defaults nicht schreiben	Neue optionale Properties mit `[JsonIgnore(Condition = WhenWritingDefault)]` lassen bestehende Annotations unverändert.
Echte Formatänderungen nur mit Major-Version	Dokumentieren, am besten zusammen mit dem EF-Major-Upgrade, wenn Anwender ohnehin mit Migrationen rechnen.

tests/.../AnnotationFormatTests.cs (Auszug)

```
[Fact]
public void Serialized_policy_format_is_stable()
{
    var policy = new RlsPolicy("support_read", PolicyCommand.Select, Permissive: false,
        Roles: ["shop_support"], Using: "true", WithCheck: null);

    // Golden Value: eine Änderung bricht bestehende Snapshots
    Assert.Equal(
        """{"Name":"support_read","Command":"Select","Permissive":false"""
        + ""","Roles":["shop_support"],"Using":"true","WithCheck":null}""",
        policy.Serialize());
}
```

Zwei weitere Tests prüfen das tolerante Lesen in beide Richtungen. Der Snapshot-Roundtrip aus Kapitel 23 ergänzt sie: Der Roundtrip zeigt, dass der aktuelle Code-Generator und die Fluent API übereinstimmen, der Golden Value zeigt, dass die nächste Version noch zu den Snapshots passt, die Anwender bereits haben.

25 Packaging und Dokumentation

Gemeinsame Einstellungen liegen in `Directory.Build.props`, Paketversionen zentral in `Directory.Packages.props` (Central Package Management):

Directory.Build.props (gekürzt)

```
<PropertyGroup>
  <TargetFramework>net10.0</TargetFramework>
  <Nullable>enable</Nullable>
  <ManagePackageVersionsCentrally>true</ManagePackageVersionsCentrally>
  <NoWarn>$(NoWarn);EF1001</NoWarn>
  <Deterministic>true</Deterministic>
</PropertyGroup>
<PropertyGroup>
  <PackageLicenseExpression>MIT</PackageLicenseExpression>
  <PackageReadmeFile>README.md</PackageReadmeFile>
  <VersionPrefix>10.0.0</VersionPrefix>
  <IncludeSymbols>true</IncludeSymbols>
  <SymbolPackageFormat>snupkg</SymbolPackageFormat>
  <PublishRepositoryUrl>true</PublishRepositoryUrl>      <!-- SourceLink -->
  <EmbedUntrackedSources>true</EmbedUntrackedSources>
</PropertyGroup>
```

Element	Warum
`GenerateDocumentationFile`	XML-Doku im Paket: Nutzer sehen Erklärungen in der IDE.
`IsAotCompatible`	Analyzer melden AOT- und Trimming-Probleme beim Build.
SourceLink, `.snupkg`	Nutzer können in den Plugin-Code debuggen. Seit dem .NET-8-SDK eingebaut.
README im Paket	Erscheint auf nuget.org; Minimalbeispiel und Kompatibilität.
CHANGELOG	Breaking Changes klar markieren, besonders bei EF-Major-Wechseln.
`InternalsVisibleTo`	Tests erreichen interne Logik ohne öffentliche API.
Paketnamen	Kern ohne Provider, Provider-Paket mit Suffix (`.PostgreSQL`, `.SqlServer`).

Zur Dokumentation gehören neben der API-Doku vor allem die Dinge, die Nutzer nicht im Code sehen: welche Einzeldienste ersetzt werden (Kombinierbarkeit), welche Event-IDs es gibt, welche Einschränkungen bestehen (Pooling) und welche Sicherheitsannahmen gelten (Datenbankrollen).

26 Die Checkliste von A bis Z

Für jedes neue Plugin, in der Reihenfolge der Entwicklung:

Architektur

☐ Stufe festlegen: Hilfsmethoden, Modell-Helfer, echtes Plugin oder Provider?

☐ Kern- und Provider-Paket trennen; Typen für Entitäts-Assemblies in den Kern.

☐ Für jeden Erweiterungspunkt zuerst ein Plugin-Interface suchen, erst dann ersetzen.

Options

☐ Extension immutable mit Kopierkonstruktor und virtuellem Clone.

☐ Alle modellrelevanten Optionen im Hash, nichts, was pro Context variiert.

☐ Validate für Voraussetzungen (Provider, Kombinationen).

☐ LogFragment und PopulateDebugInfo pflegen.

☐ Use-Methode im Namespace Microsoft.EntityFrameworkCore, generisch und nicht-generisch.

Dienste

☐ Registrierung über den ServicesBuilder (Lebensdauern aus EFs Tabelle).

☐ Einzeldienste dekorieren, wo das öffentliche Interface genügt (Kapitel 5).

☐ Sonst Replace, und die Logik öffentlich und virtuell anbieten.

☐ ISingletonOptions für Singleton-Dienste, doppelt registriert.

☐ App-Abhängigkeiten über AddInterceptors, Pooling-Verhalten dokumentieren.

Modell

☐ Annotations mit Präfix, nur einfache Werte, keine Defaults speichern.

☐ Fluent API nicht-generisch implementieren, generisch delegieren.

☐ Convention-Builder-Überladungen für ConfigurationSource.

☐ Conventions bei ModelFinalizing, wenn sie vom fertigen Modell abhängen.

☐ Attribute im Kernpaket, Attribut-Conventions im Provider-Paket.

☐ Validierung als IModelFinalizedConvention: Fehler werfen, Fragwürdiges warnen.

Type Mappings

☐ Geschützter Parameter-Konstruktor und Clone-Überschreibung.

☐ Literal-Erzeugung für Modell- und Provider-Typ.

☐ Öffentliche statische Default-Property, gemeinsame Konverter-Instanz.

☐ JsonValueReaderWriter mit Instance und ConstructorExpression.

Abfragen

☐ EF.Functions-Methoden im Kern, Übersetzung im Provider-Paket.

☐ Translators: null für fremde Methoden, Nullability korrekt, Type Mappings setzen.

☐ IEvaluatableExpressionFilterPlugin für Methoden ohne Datenbezug.

☐ Keine Laufzeitwerte zur Übersetzungszeit einsetzen (Query-Cache, AOT).

Laufzeit

☐ Interceptors synchron und asynchron implementieren.

☐ Singleton-Interceptors zustandslos und thread-sicher.

☐ Eigene Event-IDs in eigenem Bereich, EventDefinitions gecacht.

☐ Sicherheitsrelevantes als Warnung, eskalierbar per ConfigureWarnings.

☐ Wo Hooks nicht laufen, eine explizite API anbieten (z. B. ApplyTenantSession).

Migrations & Design-Time

☐ Deklarativ über Annotation-Provider (nur Design-Time) und SQL-Generator.

☐ Bezeichner maskieren, Literale über Type Mappings, selbst erzeugtes SQL validieren.

☐ Eigene Operationen idempotent, Extensions im Migrations-Namespace.

☐ DesignTimeServicesReference mit forProvider; Code-Generator entfernt Annotations.

☐ Compiled Models prüfen: Default, Instance, Laufzeit-Annotations.

Qualität

☐ Modell-, SQL-, Snapshot-Roundtrip- und Integrationstests.

☐ Caching-Test mit verschiedenen Optionen; Test auf keine Phantom-Migration.

☐ Versionsbereich je EF-Major, CI gegen Previews, EF1001 bewusst.

☐ XML-Doku, SourceLink, Symbole, README, CHANGELOG, Einschränkungen dokumentiert.

☐ Serialisierte Annotation-Formate mit Golden-Value-Tests festschreiben.

☐ Zusammen mit verbreiteten Plugins testen (Naming Conventions, Decorators).

Fortgeschritten (Teil VIII)

☐ Wege dokumentieren, die Interceptors umgehen (ExecuteUpdate, rohes SQL).

☐ Value Generators mit HasValueGeneratorFactory(Type) konfigurieren (Compiled Models).

☐ Conventions für Complex Types und JSON-Spalten prüfen.

☐ Reihenfolge eigener Migrations-Operationen dokumentieren und testen.

☐ Idempotente Skripte und suppressTransaction berücksichtigen.

☐ Betriebsannahmen dokumentieren (Rollen, Pooling, Transaction-Pooling).

☐ Context-Lebensdauern dokumentieren: Factories, Hintergrundjobs, offene Verbindungen.

☐ Mandantenübergreifender Zugriff nur über eigene Rollen, nie über App-Schalter.

☐ Heiße Pfade (Interceptors) messen, nicht vermuten.

☐ CI mit Preview-Build und Public-API-Analyzer.

Tiefe Erweiterungen (Teil IX)

☐ Eigene SQL-Knoten: VisitChildren, Update, Quote, Print, Equals/GetHashCode.

☐ Eigene Knoten im SQL-Generator UND im Nullability-Prozessor behandeln.

☐ Aggregate: Prädikat, DISTINCT und leere Mengen bewusst behandeln.

☐ Laufzeit-Annotations bewusst wählen, Design-Time-Annotations filtern.

☐ Scaffolding: Schema-Objekte des Plugins aus der Datenbank lesen.

☐ Provider-neutrale Bausteine in den Kern, Unterschiede ins Provider-Paket.

Teil VIII

Fortgeschrittene Themen

Grenzen der Laufzeit-Haken, weitere Erweiterungspunkte, Betrieb in Produktion und der Weg vom Plugin in die Anwendung.

- Kapitel 27: Massenoperationen und die Grenzen von Interceptors
- Kapitel 28: Value Generation
- Kapitel 29: Die Update-Pipeline
- Kapitel 30: Complex Types und JSON
- Kapitel 31: Modelle pro Laufzeitkontext
- Kapitel 32: Migrations für Fortgeschrittene
- Kapitel 33: Row Level Security in Produktion
- Kapitel 34: Performance von Plugins
- Kapitel 35: Einsatz in ASP.NET Core
- Kapitel 36: CI und API-Stabilität

27 Massenoperationen und die Grenzen von Interceptors

Ein SaveChanges-Interceptor sieht nur, was über `SaveChanges` läuft. `ExecuteUpdate` und `ExecuteDelete` schreiben direkt in die Datenbank, ohne Change Tracker und ohne SaveChanges. Dasselbe gilt für rohes SQL. Ein Plugin, das seine Garantien nur über SaveChanges durchsetzt, hat hier eine Lücke:

Weg	SaveChanges-Interceptor	Connection-Interceptor	RLS
`SaveChanges`	ja	ja	ja
`ExecuteUpdate`, `ExecuteDelete`	**nein**	ja	ja
`FromSql`, `SqlQuery`, `ExecuteSql`	**nein**	ja	ja
ADO.NET oder psql als App-Rolle	**nein**	**nein**	ja (ohne Mandant: keine Zeilen)

Im Beispiel fängt RLS jeden dieser Wege ab, weil die Datenbank die Regeln selbst durchsetzt. Ein Integrationstest beweist das, indem er den Interceptor gezielt umgeht:

`tests/...IntegrationTests/AdvancedTests.cs (Auszug)`

```
[Fact]
public async Task ExecuteUpdate_bypasses_interceptor_but_not_rls()
{
    // ... Bestellung für "tenant" anlegen ...
    await using var attacker = ContextFor(tenant);

    // Kein SaveChanges -> der SaveChanges-Interceptor läuft NICHT.
    // WITH CHECK der Policy verhindert das Umhängen trotzdem.
    var ex = await Assert.ThrowsAsync<PostgresException>(() =>
        attacker.Orders.ExecuteUpdateAsync(
            s => s.SetProperty(o => o.TenantId, TenantId.New())));

    Assert.Equal(PostgresErrorCodes.InsufficientPrivilege, ex.SqlState);
}
```

Was ein Plugin dokumentieren muss

Plugins ohne Datenbank-Absicherung (Soft Delete, Audit-Felder, Mandantenfilter nur per Query-Filter) müssen dokumentieren, dass Massenoperationen und rohes SQL ihre Regeln umgehen. Wer das verhindern will, kann einen `IQueryExpressionInterceptor` einsetzen: `ExecuteUpdate` und `ExecuteDelete` laufen durch die Query-Pipeline, der Interceptor sieht ihren Ausdrucksbaum und kann etwa ein `SetProperty` auf geschützte Spalten ablehnen. Rohes SQL bleibt trotzdem offen.

28 Value Generation

EF erzeugt Schlüsselwerte entweder in der Datenbank (Identity, Sequenzen) oder clientseitig über einen `ValueGenerator`. Für `Guid`-Schlüssel macht EF das von selbst, für eigene Typen wie `TenantId` nicht. Ein Plugin schließt diese Lücke mit einem Generator, einer Factory und einer Convention:

ValueGeneration/TenantIdValueGenerator.cs (Kernpaket)

```
public class TenantIdValueGenerator : ValueGenerator<TenantId>
{
    // Endgültige Werte, keine temporären Platzhalter bis zum Speichern
    public override bool GeneratesTemporaryValues => false;

    public override TenantId Next(EntityEntry entry) => TenantId.New();   // UUIDv7
}
```

ValueGeneration/TenantIdValueGeneratorFactory.cs (Kernpaket)

```
public class TenantIdValueGeneratorFactory : ValueGeneratorFactory
{
    // Zustandsloser Generator: EF darf ihn cachen und zwischen Contexts teilen
    public override ValueGenerator Create(IProperty property, ITypeBase typeBase)
        => new TenantIdValueGenerator();
}
```

Conventions/TenantIdKeyValueGenerationConvention.cs (Kernpaket)

```
public virtual void ProcessModelFinalizing(
    IConventionModelBuilder modelBuilder,
    IConventionContext<IConventionModelBuilder> context)
{
    foreach (var entityType in modelBuilder.Metadata.GetEntityTypes())
    {
        if (entityType.BaseType is not null
            || entityType.FindPrimaryKey() is not { Properties: [var property] }
            || property.ClrType != typeof(TenantId))
            continue;

        // Convention-Quelle: ein explizites ValueGeneratedNever() gewinnt (-> null)
        if (property.Builder.ValueGenerated(ValueGenerated.OnAdd) is null)
            continue;

        property.Builder.HasValueGeneratorFactory(typeof(TenantIdValueGeneratorFactory));
    }
}
```

> **Factory-Typ statt Generator**
>
> Einen Value Generator kann man auf mehreren Wegen konfigurieren: `HasValueGenerator` mit einem Typ oder einer Factory-Funktion, oder `HasValueGeneratorFactory` mit einem Factory-Typ. Der Generator für Compiled Models in EF Core 10 akzeptiert nur die letzte Variante und bricht sonst mit „The property ... has a value generator configured. Use 'HasValueGeneratorFactory' to configure the value generator factory type.“ ab. Plugins sollten deshalb `HasValueGeneratorFactory(Type)` verwenden und der Factory einen öffentlichen parameterlosen Konstruktor geben.

Die Convention liegt im Kernpaket, weil sie nichts mit PostgreSQL zu tun hat; das Provider-Paket hängt sie über sein Convention-Set-Plugin ein. Der Compiled-Model-Test prüft, dass die Factory im erzeugten Code auftaucht.

29 Die Update-Pipeline

Kapitel 11 beschreibt den Weg von LINQ zu SQL. Für das Schreiben gibt es eine eigene Pipeline, die Plugins für Bulk-Operationen, datenbankseitige Soft-Deletes oder zurückgelieferte Werte erweitern:



Dienst	Lebensdauer	Art
`IModificationCommandFactory`	Singleton	einzeln
`ICommandBatchPreparer`	Scoped	einzeln
`IModificationCommandBatchFactory`	Scoped	einzeln
`IUpdateSqlGenerator`	Singleton	einzeln

Alle Dienste sind Einzeldienste des Providers, mit den Konsequenzen aus Kapitel 5. Bevor man sie ersetzt, lohnt die Prüfung der Alternativen: Ein SaveChanges-Interceptor kann Zustände umschreiben (aus Löschen wird Ändern beim Soft Delete), ein Command-Interceptor kann SQL ergänzen, und für Massenänderungen gibt es `ExecuteUpdate`. Ein eigener `IUpdateSqlGenerator` ist erst dann gerechtfertigt, wenn das SQL selbst eine andere Form braucht, etwa `INSERT ... ON CONFLICT` für Upserts.

30 Complex Types und JSON

Complex Types

Complex Types (seit EF Core 8) sind Werttypen ohne eigene Identität, deren Properties in der Tabelle des Besitzers landen. Für Plugins heißt das: Conventions, die über `entityType.GetProperties()` laufen, sehen deren Properties **nicht**. Sie hängen an `IComplexProperty` und müssen eigens durchlaufen werden:

Muster für Conventions (Skizze)

```
foreach (var complexProperty in entityType.GetComplexProperties())
{
    foreach (var property in complexProperty.ComplexType.GetProperties())
    {
        // Spaltennamen sind präfixiert (z. B. "Address_City"); immer über
        // property.GetColumnName(table) ermitteln, nie selbst zusammensetzen
    }
}
```

Fluent-API-Methoden für Properties sollten zusätzlich auf `ComplexTypePropertyBuilder` angeboten werden, wenn sie auch dort sinnvoll sind. Das Beispiel-Plugin braucht das nicht: Isolation und Wertgenerierung betreffen nur Entitäten.

JSON-Spalten

Owned Types und (seit EF Core 10) Complex Types lassen sich mit `ToJson()` als eine JSON-Spalte speichern. Werte darin liest und schreibt EF nicht über Konverter, sondern über den `JsonValueReaderWriter` des Type Mappings (Kapitel 10). Fehlt er, scheitert der Modellbau. Unit- und Integrationstest des Beispiels prüfen beide Seiten:

tests/.../JsonTests.cs

```
using var ctx = Contexts.Create(model: m =>
    m.Entity<Shipment>().OwnsOne(s => s.Details, d => d.ToJson()));

var property = ctx.DesignModel()
    .FindEntityType(typeof(ShipmentDetails))!
    .FindProperty(nameof(ShipmentDetails.ApprovedBy))!;

Assert.Same(TenantIdJsonReaderWriter.Instance,
    property.GetTypeMapping().JsonValueReaderWriter);
```

> **Hinweis**
>
> PostgreSQL-Arrays (`List<TenantId>` als `uuid[]`) sind kein JSON: Npgsql bildet primitive Collections nativ als Arrays ab. Ob ein Provider Collections mit konvertierten Elementtypen unterstützt, ist providerspezifisch und gehört in einen eigenen Test, bevor man es dokumentiert.

31 Modelle pro Laufzeitkontext

EF baut das Modell einmal pro Context-Typ und cached es (Kapitel 2). Soll das Modell von Laufzeitdaten abhängen, etwa einem Schema pro Mandant, muss der Cache-Schlüssel diese Daten enthalten. Dafür gibt es `IModelCacheKeyFactory`:

`tests/.../SchemaPerTenantTests.cs (gekürzt)`

```
public sealed class SchemaModelCacheKeyFactory : IModelCacheKeyFactory
{
    public object Create(DbContext context, bool designTime)
        => context is ISchemaScoped scoped
            ? (context.GetType(), scoped.Schema, designTime)
            : (object)(context.GetType(), designTime);
}

public sealed class SchemaContext : DbContext, ISchemaScoped
{
    // Konstruktor: (DbContextOptions<SchemaContext> options, string schema)
    public string Schema { get; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
        => modelBuilder.HasDefaultSchema(Schema);
}

// Registrierung
options.UseNpgsql(connectionString)
       .ReplaceService<IModelCacheKeyFactory, SchemaModelCacheKeyFactory>();
```

Der Test `SchemaPerTenantTests` prüft beide Seiten des Cache-Schlüssels: Zwei Schemas ergeben zwei Modelle mit jeweils eigenem Standardschema, und dasselbe Schema verwendet das gecachte Modell wieder, statt es neu zu bauen.

Das ist die Grundlage für „Schema pro Mandant“, eine Alternative zu RLS. Die drei üblichen Ansätze im Vergleich:

Ansatz	Stärken	Schwächen
Gemeinsame Tabellen + RLS (Beispiel)	Ein Modell, ein Schema, einfache Migrationen; Trennung erzwingt die Datenbank	Alle Mandanten teilen Tabellen und Indizes; Fehlkonfiguration der Rollen hebelt die Trennung aus
Schema pro Mandant	Starke Trennung, getrennte Backups pro Schema möglich	Ein Modell pro Mandant im Cache (Speicher!), Migrationen für jedes Schema
Datenbank pro Mandant	Maximale Trennung, unabhängige Skalierung	Ein Modell genügt, aber Verbindungen, Migrationen und Betrieb pro Datenbank

> **Modell-Cache und Speicher**
>
> Jeder Schlüsselwert erzeugt ein eigenes Modell im Speicher. Bei Hunderten Mandanten mit großen Modellen wird das teuer. Außerdem dürfen Werte, die das Modell verändern, nie aus ungeprüften Eingaben stammen.

32 Migrations für Fortgeschrittene

Reihenfolge von Operationen

Generierte und handgeschriebene Operationen laufen in der Reihenfolge, in der sie in `Up()` stehen. Das ist eine echte Fehlerquelle. Die Policy `support_read` des Beispiels gilt `TO shop_support`. EF erzeugt sie direkt nach `CREATE TABLE`. Wird die Rolle erst am Ende von `Up()` angelegt, scheitert die Migration.

`Migrations/20261021_Initial.cs`

```
protected override void Up(MigrationBuilder migrationBuilder)
{
    // 1. Was Policies referenzieren: VOR den generierten Operationen
    migrationBuilder.CreateApplicationRole("shop_support");

    // 2. Von EF generiert: Tabellen, RLS, Policies (TO shop_support)
    migrationBuilder.CreateTable(name: "Customers", /* ... */);

    // 3. Was Tabellen voraussetzt: NACH den generierten Operationen
    migrationBuilder.CreateApplicationRole("shop_app", canLogin: true);
    migrationBuilder.GrantTablePrivileges("shop_app");     // wirkt auf bestehende Tabellen
}
```

Idempotente Skripte

`dotnet ef migrations script --idempotent` erzeugt ein Skript, das gegen jeden Datenbankstand laufen kann. Npgsql verpackt dazu jede Operation in einen Block `DO $EF$ ... $EF$`, der die Historientabelle prüft. Eigene Operationen, die selbst einen `DO $$`-Block erzeugen, stecken dann darin. Das funktioniert, weil die Dollar-Quotes verschiedene Tags haben. Solche Annahmen gehören in einen Test:

`tests/...IntegrationTests/AdvancedTests.cs (Auszug)`

```
var script = db.GetService<IMigrator>()
    .GenerateScript(options: MigrationsSqlGenerationOptions.Idempotent);

await db.Database.ExecuteSqlRawAsync(script);   // verschachteltes DO gültig?
await db.Database.ExecuteSqlRawAsync(script);   // zweiter Lauf wird übersprungen
```

Operationen außerhalb von Transaktionen

Manche Befehle dürfen nicht in einer Transaktion laufen, in PostgreSQL etwa `CREATE INDEX CONCURRENTLY`. Eigene Operationen mit solchem SQL beenden ihren Befehl mit `EndStatement(builder, suppressTransaction: true)`. EF führt ihn dann außerhalb der Migrationstransaktion aus. Der Preis: Schlägt die Migration danach fehl, wird dieser Befehl nicht zurückgerollt; er sollte deshalb idempotent sein.

Migration Bundles

`dotnet ef migrations bundle` erzeugt eine ausführbare Datei, die Migrationen ohne SDK anwendet. Sie enthält das Startprojekt samt Plugin-Assemblies; eigene Operationen und der SQL-Generator des Plugins wirken also auch dort. Wichtig ist, dass die Design-Time-Factory oder der Host dieselben Plugin-Optionen verwendet wie die Anwendung, sonst entsteht SQL für ein anderes Modell.

33 Row Level Security in Produktion

RLS ist nur so stark wie seine Betriebsbedingungen. Diese Fallen machen die Policies stillschweigend wirkungslos oder unbrauchbar:

Falle	Folge	Abhilfe
App läuft als Tabelleneigentümer	Policies werden ignoriert	`FORCE ROW LEVEL SECURITY` (Default im Plugin) und getrennte Rollen
Superuser oder `BYPASSRLS`	RLS wird immer umgangen, auch mit FORCE	App-Rolle mit `NOSUPERUSER NOBYPASSRLS` (eigene Operation, Kap. 19)
Setting nicht gesetzt	Ohne `nullif`: Fehler; mit: leere Ergebnisse	Interceptor setzt den Wert bei jedem Öffnen
Kein Index auf der Mandantenspalte	Jede Abfrage scannt die ganze Tabelle	Index-Convention (Kapitel 8)
Transaction-Pooling (PgBouncer)	Session-Settings landen bei anderen Clients oder gehen verloren	Setting transaktionslokal setzen (siehe unten)
Verbindung außerhalb von EF geöffnet	Die Session behält einen alten Mandanten oder keinen	EF die Verbindungen öffnen lassen oder `ApplyTenantSession()` (Kapitel 15)
„Bypass“-Schalter für die App-Rolle	Wer SQL als App-Rolle ausführen kann, kann ihn umlegen	Eigene Rollen für mandantenübergreifenden Zugriff (siehe unten)

Transaction-Pooling

PgBouncer im Modus „transaction“ teilt eine Serververbindung zwischen Clients pro Transaktion. Ein mit `set_config(..., false)` gesetzter Session-Wert kann dann bei einem anderen Client ankommen, eine Sicherheitslücke. Die sichere Variante: das Setting mit `set_config(..., true)` *transaktionslokal* setzen und jede Datenbankarbeit in einer expliziten Transaktion ausführen. Technisch wechselt der Interceptor dafür von `IDbConnectionInterceptor` zu `IDbTransactionInterceptor` (`TransactionStarted`). Weil dann Abfragen ohne Transaktion gar keinen Mandanten haben, muss die Anwendung das konsequent einhalten. Das Beispiel bleibt beim einfacheren Session-Modell und dokumentiert, dass es Session- oder Npgsql-Pooling voraussetzt.

Performance von Policies

PostgreSQL fügt Policies wie zusätzliche `WHERE`-Bedingungen in jede Abfrage ein. `current_setting` ist `STABLE` und wird pro Abfrage einmal ausgewertet; zusammen mit dem Index ist der Aufwand gering. Teure Policies entstehen durch Unterabfragen (etwa Rechte aus einer anderen Tabelle) oder Funktionen, die nicht `LEAKPROOF` sind und Optimierungen verhindern. Prüfen lässt sich das mit `EXPLAIN (ANALYZE)` als App-Rolle, nicht als Owner, weil der Owner die Policies sonst gar nicht sieht.

Legitimer mandantenübergreifender Zugriff

Früher oder später braucht jemand Daten über Mandanten hinweg: Berichte für den Betreiber, Wartungsjobs, Support. Die verlockende Abkürzung ist ein Schalter in der Anwendung, etwa ein zusätzliches Session-Setting `app.bypass`, das die Policy berücksichtigt. Das ist keine Sicherheitsgrenze: Was die Anwendungsrolle setzen kann, kann auch ein eingeschleustes SQL-Statement setzen. Die Trennung muss stattdessen über Datenbankrollen entstehen:

Bedarf	Lösung	Warum es sicher ist
Berichte, Analysen	Eigene Nur-Lese-Rolle mit `BYPASSRLS`, nur `SELECT`, eigenem Connection-String und einer Context-Konfiguration ohne Mandant	Die App-Rolle kann nicht zu dieser Rolle werden; ihre Zugangsdaten liegen woanders
Support liest ausgewählte Daten	Zusätzliche permissive Policy für eine Support-Rolle: `HasPolicy(...)` mit `For(Select)` und `To("shop_support")`	Beschränkt auf SELECT und auf Mitglieder der Rolle
Wartung pro Mandant	Hintergrundjob, der die Mandanten durchläuft und jeden einzeln setzt (Kapitel 35)	Läuft durch die normalen Policies

Reporting-Rolle (tests/...IntegrationTests/CrossTenantAccessTests.cs)

```
-- Angelegt von einem Administrator: BYPASSRLS erfordert einen Superuser
CREATE ROLE shop_reporting LOGIN PASSWORD '...' NOSUPERUSER BYPASSRLS;
GRANT USAGE ON SCHEMA public TO shop_reporting;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO shop_reporting;
```

Die Anwendungsrolle dagegen kann die Policies nicht abschalten. `SET row_security = off` deaktiviert RLS für eine Rolle ohne `BYPASSRLS` nicht; PostgreSQL lehnt stattdessen jede Abfrage ab, die die Policies betreffen würden. `CrossTenantAccessTests` prüft alle drei Punkte: Die Reporting-Rolle sieht alle Mandanten, sie kann nicht schreiben, und die App-Rolle kann sich nicht selbst helfen.

> **Mandantenübergreifenden Zugriff protokollieren**
>
> Jede Verbindung der Reporting-Rolle ist ein Zugriff auf alle Mandanten. Sie gehört protokolliert (`log_connections`, pgaudit), und die Rolle sollte so wenigen Personen und Diensten wie möglich zugewiesen werden. `CreateApplicationRole` des Plugins legt Rollen bewusst immer mit `NOBYPASSRLS` an: Bypass-Rollen gehören nicht in die Migrationen der Anwendung.

34 Performance von Plugins

Plugin-Code läuft in drei Häufigkeiten. Wer weiß, welcher Code wie oft läuft, weiß, wo sich Optimierung lohnt:

Häufigkeit	Code	Folgerung
Einmal pro Provider	Conventions, Type-Mapping-Plugins, Singleton-Options	Lesbarkeit vor Mikro-Optimierung
Einmal pro Abfrageform	Translators, Evaluatable-Filter, Query-Interceptors	Moderat; Ergebnis wird gecacht
Bei jeder Ausführung	Interceptors für Commands, Verbindungen, SaveChanges, Materialisierung	Heißer Pfad: keine Reflection, keine unnötigen Allokationen

Daraus ergeben sich einige Regeln, die das Beispiel befolgt: `MethodInfo`- und `PropertyInfo`-Lookups liegen in statischen Feldern. Der SaveChanges-Interceptor legt seine Liste erst an, wenn er tatsächlich eine mandantenfähige Änderung findet. Der Connection-Interceptor kostet einen Roundtrip pro Öffnen, was dokumentiert ist. Solche Annahmen sollte man messen, nicht raten. Das Repository enthält ein BenchmarkDotNet-Projekt für die SaveChanges-Logik; es läuft ohne Datenbank:

benchmarks/.../SaveChangesInterceptorBenchmark.cs (gekürzt)

```
[MemoryDiagnoser]
public class SaveChangesInterceptorBenchmark
{
    [Params(1, 100)]
    public int Entities { get; set; }

    // GlobalSetup: Context mit UseAcmeTenancy() (ohne Verbindung) und dem Interceptor

    [Benchmark(Baseline = true)]
    public int TrackOnly()                  // was EF vor dem Speichern ohnehin tut
    {
        AddOrders();
        var count = _db.ChangeTracker.Entries().Count();
        _db.ChangeTracker.Clear();
        return count;
    }

    [Benchmark]
    public int TrackAndApplyTenant()        // dasselbe plus die SaveChanges-Logik des Plugins
    {
        AddOrders();
        _interceptor.ApplyTenant(_db);
        var count = _db.ChangeTracker.Entries().Count();
        _db.ChangeTracker.Clear();
        return count;
    }
}
```

Benchmark ausführen

```
dotnet run -c Release --project benchmarks/Acme.EntityFrameworkCore.Tenancy.Benchmarks
```

Methode	Entitäten	Mittelwert	Verhältnis	Allokiert
TrackOnly	1	1,5 µs	1,00	1,8 KB
TrackAndApplyTenant	1	2,3 µs	1,54	2,7 KB
TrackOnly	100	123 µs	1,00	134 KB
TrackAndApplyTenant	100	170 µs	1,38	202 KB

Beispielergebnisse (Short Job, .NET 10, Rechner des Autors; die absoluten Werte schwanken, aussagekräftiger sind die Verhältnisse). Das Plugin fügt pro gespeicherter Entität etwa eine halbe Mikrosekunde hinzu, rund 40 bis 50 Prozent mehr als das Change Tracking allein. Das klingt nach viel, doch ein einziger Roundtrip zur Datenbank kostet Hunderte Mikrosekunden; der zusätzliche Roundtrip des Connection-Interceptors pro Öffnen (Kapitel 15) wiegt daher weit schwerer als die SaveChanges-Logik. Genau diese Art Erkenntnis liefert ein Benchmark: Er zeigt, wo sich Optimieren lohnt und wo nicht.

> **Tipp**
>
> Für den Start großer Modelle ist das Compiled Model (Kapitel 21) der wirksamste Hebel. Conventions eines Plugins laufen dann gar nicht mehr zur Laufzeit; deshalb lohnt es sich, die Compiled-Model-Fähigkeit zu testen statt nur zu behaupten.

35 Einsatz in ASP.NET Core

Das Beispiel `samples/ShopApi` zeigt den Weg vom Plugin in eine Web-API. Der Mandant kommt aus dem Claim `tenant_id` des angemeldeten Benutzers:

samples/ShopApi/HttpTenantProvider.cs

```
public sealed class HttpTenantProvider : ITenantProvider
{
    public const string ClaimType = "tenant_id";

    private readonly IHttpContextAccessor _httpContextAccessor;

    public HttpTenantProvider(IHttpContextAccessor httpContextAccessor)
    {
        ArgumentNullException.ThrowIfNull(httpContextAccessor);

        _httpContextAccessor = httpContextAccessor;
    }

    // Bei jedem Zugriff neu ermittelt: funktioniert auch als Singleton (Pooling)
    public TenantId? CurrentTenantId
    {
        get
        {
            var value = _httpContextAccessor.HttpContext?.User.FindFirst(ClaimType)?.Value;
            return Guid.TryParse(value, out var id) ? new TenantId(id) : null;
        }
    }
}
```

samples/ShopApi/Program.cs (Auszug)

```
builder.Services.AddHttpContextAccessor();
builder.Services.AddAcmeTenancy<HttpTenantProvider>();
builder.Services.AddDbContext<ShopApiContext>((sp, options) => options
    .UseNpgsql(builder.Configuration.GetConnectionString("Shop"))
    .UseAcmeTenancy(sp));

// Kein Mandantenfilter im Code: RLS in der Datenbank sorgt für die Trennung
app.MapGet("/orders", async (ShopApiContext db) =>
    await db.Orders.OrderBy(o => o.Id).ToListAsync());
```

> **Woher der Mandant kommt**
>
> Der Mandant muss aus einer vertrauenswürdigen Quelle kommen: einem Claim aus einem signierten Token. Ein Header oder Query-Parameter, den der Client frei setzt, hebelt die gesamte Isolation aus. Das Beispiel enthält eine Header-Simulation nur für die Entwicklungsumgebung und kennzeichnet sie ausdrücklich.

Für `AddDbContextPool` registriert man den Provider als Singleton. Das ist möglich, weil `HttpTenantProvider` den Mandanten bei jedem Zugriff neu ermittelt; die Interceptors fragen ihn bei jeder Verbindung und jedem Speichern:

Variante mit Pooling

```
builder.Services.AddAcmeTenancy<HttpTenantProvider>(ServiceLifetime.Singleton);
builder.Services.AddDbContextPool<ShopApiContext>((sp, options) => options
    .UseNpgsql(connectionString)
    .UseAcmeTenancy(sp));   // sp: Root-Provider; Interceptors entstehen einmal
```

DbContext-Factories

`AddDbContextFactory` ist die übliche Wahl in Blazor Server und für parallele Arbeit. Anders als `AddDbContext` registriert es Factory und Options standardmäßig als **Singletons**. Der Options-Callback bekommt dann den Root-Provider, nicht den Scope der aktuellen Anfrage. Für einen Scoped-Tenant-Provider, der Zustand pro Scope hält, hat das Folgen:

Registrierung	Verhalten
Singleton-Factory, Scope-Validierung an (ASP.NET Core in Development)	Exception: „Cannot resolve scoped service 'ITenantProvider' from root provider.“
Singleton-Factory, Scope-Validierung aus (Standard in Produktion)	Kein Fehler. Die Interceptors bekommen eine Instanz aus dem Root-Container, die keine Anfrage je befüllt: kein Mandant, Speichern wirft, Abfragen sehen nichts
Factory als `Scoped` registriert	Jeder Scope bekommt eine eigene Factory mit eigenem Tenant-Provider: korrekt

Factory mit dem Mandanten der Anfrage

```
builder.Services.AddDbContextFactory<ShopApiContext>(
    (sp, options) => options.UseNpgsql(connectionString).UseAcmeTenancy(sp),
    ServiceLifetime.Scoped);         // Factory und Options pro Scope
```

Ein Provider, der den Mandanten bei jedem Zugriff neu ermittelt, etwa `HttpTenantProvider`, funktioniert in jeder Variante; das Problem betrifft Provider mit Zustand pro Scope. `AddPooledDbContextFactory` verhält sich immer wie Pooling: Die Options existieren einmal, der Tenant-Provider muss also ein solches Singleton sein. `DbContextFactoryTests` zeigt alle drei Varianten.

Außerhalb von HTTP-Anfragen

Hintergrunddienste, Message-Handler und geplante Jobs haben keine Anfrage und keinen Claim. Der Mandant kommt dann aus dem Job selbst, etwa aus der Nachricht oder aus einer Liste zu verarbeitender Mandanten. Der `AmbientTenantProvider` des Beispiels speichert ihn in einem `AsyncLocal`, das mit dem asynchronen Kontrollfluss mitwandert:

samples/ShopSample/AmbientTenantProvider.cs (gekürzt)

```
public sealed class AmbientTenantProvider : ITenantProvider
{
    private static readonly AsyncLocal<TenantId?> Current = new();

    public TenantId? CurrentTenantId => Current.Value;

    public static IDisposable Use(TenantId tenant)
    {
        var previous = Current.Value;
```

```
        Current.Value = tenant;
        return new Restore(previous);     // setzt beim Dispose den vorigen Mandanten zurück
    }

    // private sealed class Restore : IDisposable { ... Current.Value = _previous; }
}
```

Job-Muster (tests/.../BackgroundTenantTests.cs)

```
// Erst der Mandant, dann Scope und Context: die Interceptors entstehen im Scope
using (AmbientTenantProvider.Use(tenant))
{
    await using var scope = scopeFactory.CreateAsyncScope();
    var db = scope.ServiceProvider.GetRequiredService<ShopContext>();
    // ... Arbeit für diesen Mandanten ...
}
```

Weil der Provider das `AsyncLocal` bei jedem Zugriff liest, kann er als Singleton registriert werden. Parallele Jobs behalten ihren eigenen Mandanten, und nach dem `using`-Block bleibt kein Mandant zurück; `BackgroundTenantTests` prüft beides mit 20 parallelen Jobs. Eine Anwendung mit Web-Anfragen *und* Jobs kombiniert beide Quellen in einem Provider: zuerst der Claim, sonst der Ambient-Mandant.

> **Nie auf „alle Mandanten“ zurückfallen**
>
> Ohne Mandant muss ein Job scheitern oder nichts sehen, niemals alles. Das Plugin verhält sich so: Der Session-Wert bleibt leer, die Policies treffen keine Zeile, und Speichern wirft. Jobs, die wirklich alle Mandanten brauchen, nutzen die Reporting-Rolle aus Kapitel 33.

36 CI und API-Stabilität

Der Workflow

Der GitHub-Actions-Workflow im Repository setzt die Empfehlungen aus Kapitel 23 und 24 um. Ein Job baut und testet ohne und mit Datenbank; ein zweiter baut das Handbuch; ein dritter baut gegen die nächste EF-Preview und darf fehlschlagen:

.github/workflows/ci.yml (gekürzt)

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with: { global-json-file: global.json }
      - run: dotnet build -c Release
      - run: dotnet test tests/Acme.EntityFrameworkCore.Tenancy.Tests -c Release --no-build
      - run: dotnet test tests/Acme.EntityFrameworkCore.Tenancy.IntegrationTests
          -c Release --no-build
      - run: dotnet test tests/Acme.EntityFrameworkCore.Tenancy.SqlServer.Tests
          -c Release --no-build

  handbook:                           # beide PDFs; scheitert an zu langen Codezeilen
    runs-on: ubuntu-latest
    steps:
      # ... checkout, setup-python, pip install -r docs/handbook/requirements.txt ...
      - run: python docs/handbook/build.py --lang en

  ef-preview:
    runs-on: ubuntu-latest
    continue-on-error: true             # Frühwarnsystem, kein Blocker
    steps:
      # ... setup-dotnet mit 11.0.x preview ...
      - run: dotnet build -c Release -p:EFCorePreview=true
```

Der Preview-Build funktioniert über bedingte Paketversionen in `Directory.Packages.props`:

Directory.Packages.props (Auszug)

```
<PropertyGroup>
  <CentralPackageFloatingVersionsEnabled>true</CentralPackageFloatingVersionsEnabled>
</PropertyGroup>
<ItemGroup Condition="'$(EFCorePreview)' != 'true'">
  <PackageVersion Include="Microsoft.EntityFrameworkCore" Version="[10.0.0,11.0.0)" />
</ItemGroup>
<ItemGroup Condition="'$(EFCorePreview)' == 'true'">
  <PackageVersion Include="Microsoft.EntityFrameworkCore" Version="11.0.0-*" />
</ItemGroup>
```

Öffentliche API absichern

Ein Plugin hat zwei öffentliche APIs: die für Anwender (Fluent API, Options, Typen) und die für Erweiterer (virtuelle Methoden, Dependencies, öffentliche Dienste). Beide brechen leicht unbemerkt.

Der `Microsoft.CodeAnalysis.PublicApiAnalyzers` führt die öffentliche API in zwei Textdateien (`PublicAPI.Shipped.txt`, `PublicAPI.Unshipped.txt`) und meldet jede Abweichung. Jede API-Änderung wird so im Pull Request als Diff sichtbar.

> **Einrichtung**
>
> Beim ersten Build meldet der Analyzer jede öffentliche API als RS0016, weil die Dateien leer sind. Per Code-Fix „Add all items … to the public API“ werden sie gefüllt. Beim Release wandert der Inhalt von Unshipped nach Shipped; ab dann ist das Entfernen einer ausgelieferten API ein sichtbarer Breaking Change.

Teil IX

Tiefe Erweiterungen

Eigene SQL-Ausdrücke, Aggregate, Laufzeit-Metadaten, Scaffolding und ein zweiter Provider: die Werkzeuge für sehr komplexe Plugins.

- Kapitel 37: Eigene SQL-Ausdrucksknoten
- Kapitel 38: Aggregat-Translators
- Kapitel 39: Fluent API auf Property-Ebene und Laufzeit-Annotations
- Kapitel 40: Scaffolding in der Praxis
- Kapitel 41: Was in den Kern gehört
- Kapitel 42: Ein zweiter Provider: SQL Server

37 Eigene SQL-Ausdrucksknoten

Kapitel 12 baut SQL aus vorhandenen Bausteinen: Funktionen, Konstanten, Casts. Das reicht für die meisten Plugins. Ein **eigener Ausdrucksknoten** (eine Ableitung von `SqlExpression`) wird nötig, wenn mindestens eins davon zutrifft:

Grund	Beispiel
Die Syntax ist kein Funktionsaufruf	Operatoren wie `@@`, `<->`, `IS DISTINCT FROM`, Schlüsselwörter wie `AT TIME ZONE`
Die Nullability-Semantik weicht ab	`IS DISTINCT FROM` ist nie NULL, auch wenn ein Operand NULL ist
Andere Teile sollen den Knoten wiedererkennen	Optimierungen, die nur für diesen Ausdruck gelten

Das Beispiel implementiert den null-sicheren Vergleich `EF.Functions.IsNotDistinctFrom(a, b)` und `IsDistinctFrom(a, b)`. Npgsql bietet ihn nicht als Funktion an; er erfüllt alle drei Gründe.

Anatomie eines Knotens

Mitglied	Zweck	Folge, wenn es fehlt
`VisitChildren` / `Update`	Visitors können Kinder ersetzen; unveränderte Kinder ergeben dieselbe Instanz	EF wirft beim ersten Visitor
`Equals` / `GetHashCode`	Strukturvergleich für den Query-Cache und Optimierungen	Cache-Fehltreffer, falsche Deduplizierung
`Print`	Lesbare Darstellung in DebugView und Fehlermeldungen	Unklare Fehler
`Quote`	C#-Code für vorkompilierte Abfragen und NativeAOT	Kein AOT-Support
SQL-Generator	Schreibt den Knoten als SQL	**Stillschweigend** fehlendes SQL
Nullability-Prozessor	Kennt die NULL-Semantik	Exception beim Übersetzen

Query/Expressions/DistinctFromExpression.cs (gekürzt)

```
public class DistinctFromExpression : SqlExpression
{
    private static ConstructorInfo? _quotingConstructor;

    public DistinctFromExpression(SqlExpression left, SqlExpression right, bool negated,
        RelationalTypeMapping? typeMapping)
        : base(typeof(bool), typeMapping)
    {
        // ... ThrowIfNull ...
        Left = left;
        Right = right;
        IsNegated = negated;
    }

    public virtual SqlExpression Left { get; }
```

```
    public virtual SqlExpression Right { get; }
    public virtual bool IsNegated { get; }

    protected override Expression VisitChildren(ExpressionVisitor visitor)
        => Update((SqlExpression)visitor.Visit(Left), (SqlExpression)visitor.Visit(Right));

    public virtual DistinctFromExpression Update(SqlExpression left, SqlExpression right)
        => left == Left && right == Right
            ? this
            : new DistinctFromExpression(left, right, IsNegated, TypeMapping);

    public override Expression Quote()
        => New(
            _quotingConstructor ??= typeof(DistinctFromExpression).GetConstructor(
                [typeof(SqlExpression), typeof(SqlExpression), typeof(bool),
                 typeof(RelationalTypeMapping)])!,
            Left.Quote(), Right.Quote(), Constant(IsNegated),
            RelationalExpressionQuotingUtilities.QuoteTypeMapping(TypeMapping));

    protected override void Print(ExpressionPrinter expressionPrinter)
    {
        expressionPrinter.Visit(Left);
        expressionPrinter.Append(IsNegated ? " IS DISTINCT FROM " : " IS NOT DISTINCT FROM ");
        expressionPrinter.Visit(Right);
    }

    public override bool Equals(object? obj)
        => obj is DistinctFromExpression other
           && (ReferenceEquals(this, other)
               || base.Equals(other) && Left.Equals(other.Left)
                  && Right.Equals(other.Right) && IsNegated == other.IsNegated);

    public override int GetHashCode()
        => HashCode.Combine(base.GetHashCode(), Left, Right, IsNegated);
}
```

Der Translator erzeugt den Knoten und gleicht dabei die Type Mappings beider Seiten an: Hat eine Seite ein Mapping (etwa eine Spalte), bekommt die andere dasselbe. So wird aus einer Konstante das passende Literal:

Query/DistinctFromTranslator.cs (Auszug)

```
var definition = method.GetGenericMethodDefinition();
if (definition != IsNotDistinctFromMethod && definition != IsDistinctFromMethod)
    return null;

var left = arguments[1];                        // arguments[0] ist DbFunctions
var right = arguments[2];
var typeMapping = ExpressionExtensions.InferTypeMapping(left, right)
                  ?? _typeMappingSource.FindMapping(left.Type);

return new DistinctFromExpression(
    _sql.ApplyTypeMapping(left, typeMapping)!,
    _sql.ApplyTypeMapping(right, typeMapping)!,
    negated: definition == IsDistinctFromMethod,
    _typeMappingSource.FindMapping(typeof(bool)));
```

Den SQL-Generator erweitern

Der `QuerySqlGenerator` schreibt SQL per Visitor. Unbekannte Knoten gibt er an `Expression Visitor.VisitExtension` weiter, das nur die Kinder besucht. Ohne eigenen Override entsteht also kein Fehler, sondern **SQL ohne den Operator**. Das Plugin erbt vom Npgsql-Generator:

Query/Internal/TenancyNpgsqlQuerySqlGenerator.cs (gekürzt)

```
public class TenancyNpgsqlQuerySqlGenerator : NpgsqlQuerySqlGenerator
{
    public TenancyNpgsqlQuerySqlGenerator(QuerySqlGeneratorDependencies dependencies,
        IRelationalTypeMappingSource typeMappingSource,
        bool reverseNullOrderingEnabled, Version postgresVersion)
        : base(dependencies, typeMappingSource, reverseNullOrderingEnabled, postgresVersion)
    {
    }

    protected override Expression VisitExtension(Expression extensionExpression)
        => extensionExpression switch
        {
            DistinctFromExpression e => VisitDistinctFrom(e),
            _ => base.VisitExtension(extensionExpression),
        };

    protected virtual Expression VisitDistinctFrom(DistinctFromExpression expression)
    {
        VisitOperand(expression.Left);
        Sql.Append(expression.IsNegated ? " IS DISTINCT FROM " : " IS NOT DISTINCT FROM ");
        VisitOperand(expression.Right);
        return expression;
    }
}
```

Der Generator wird über `IQuerySqlGeneratorFactory` erzeugt, einen Singleton-Einzeldienst. Das Plugin ersetzt ihn durch eine eigene Factory mit denselben Abhängigkeiten wie die von Npgsql (`QuerySqlGeneratorDependencies`, Type-Mapping-Source, `INpgsqlSingletonOptions`).

Den Nullability-Prozessor erweitern

Nach der Übersetzung optimiert EF NULL-Vergleiche. Dafür muss es von jedem Knoten wissen, ob er NULL werden kann. Unbekannte Knoten landen in `VisitCustomSqlExpression`, dessen Basisimplementierung eine Exception wirft. Der Prozessor hängt an einer Kette aus drei Klassen, die alle ersetzt werden müssen:

Query/Internal/TenancyNpgsqlSqlNullabilityProcessor.cs (gekürzt)

```
public class TenancyNpgsqlSqlNullabilityProcessor : NpgsqlSqlNullabilityProcessor
{
    // Konstruktor: (…ProcessorDependencies, …ProcessorParameters) : base(...)

    protected override SqlExpression VisitCustomSqlExpression(
        SqlExpression sqlExpression, bool allowOptimizedExpansion, out bool nullable)
    {
        if (sqlExpression is DistinctFromExpression distinctFrom)
        {
            // Kinder besuchen (sie können optimiert werden), ihre Nullability ist egal
```

```
            var left = Visit(distinctFrom.Left, out _)!;
            var right = Visit(distinctFrom.Right, out _)!;

            nullable = false;                   // IS [NOT] DISTINCT FROM ist nie NULL
            return distinctFrom.Update(left, right);
        }

        return base.VisitCustomSqlExpression(
            sqlExpression, allowOptimizedExpansion, out nullable);
    }
}

public class TenancyNpgsqlParameterBasedSqlProcessor : NpgsqlParameterBasedSqlProcessor
{
    protected override Expression ProcessSqlNullability(
        Expression selectExpression, ParametersCacheDecorator parametersDecorator)
        => new TenancyNpgsqlSqlNullabilityProcessor(Dependencies, Parameters)
            .Process(selectExpression, parametersDecorator);
}

// + TenancyNpgsqlParameterBasedSqlProcessorFactory
//   : IRelationalParameterBasedSqlProcessorFactory
```

Infrastructure/TenancyOptionsExtension.cs › ApplyServices (Ergänzung)

```
// 4) Tiefe Erweiterung: eigener SQL-Ausdrucksknoten
services.Replace(ServiceDescriptor
    .Singleton<IQuerySqlGeneratorFactory, TenancyNpgsqlQuerySqlGeneratorFactory>());
services.Replace(ServiceDescriptor
    .Scoped<IRelationalParameterBasedSqlProcessorFactory,
        TenancyNpgsqlParameterBasedSqlProcessorFactory>());
```

Der Preis eigener Knoten

Eigene Knoten verlangen das Ersetzen zweier Einzeldienste und das Erben von internen Npgsql-Klassen. Zwei Plugins, die beide eigene Knoten einführen, lassen sich nur kombinieren, wenn eines vom anderen erbt. Deshalb gilt: Funktionskomposition zuerst; eigene Knoten nur, wenn einer der drei Gründe vorliegt. Und die Ersetzung gehört an prominente Stelle in die Dokumentation.

Experimentelle API

`Quote` und `RelationalExpressionQuotingUtilities` gehören zur experimentellen API für vorkompilierte Abfragen. Ihre Verwendung meldet der Compiler als EF9100; das Beispiel unterdrückt die Meldung projektweit und dokumentiert das in `Directory.Build.props`.

tests/.../DeepQueryTests.cs (Auszug)

```
[Fact]
public void IsDistinctFrom_in_projection_passes_nullability_processor()
{
    // Ohne eigenen Nullability-Prozessor würde EF hier eine Exception werfen
    using var ctx = Contexts.Create();

    var sql = ctx.Customers
        .Select(c => EF.Functions.IsDistinctFrom(c.Name, (string?)null))
        .ToQueryString();
```

```
    Assert.Contains("IS DISTINCT FROM", sql);
}
```

38 Aggregat-Translators

Aggregate wie `Sum` oder `string_agg` fassen mehrere Zeilen zusammen, typischerweise nach `GroupBy`. Ein `IAggregateMethodCallTranslator` bekommt statt eines einzelnen Arguments eine `EnumerableExpression`, die beschreibt, *was* aggregiert wird:

Eigenschaft	Bedeutung	Herkunft in LINQ
`Selector`	Der aggregierte Ausdruck	`g.Select(o => o.Total > 0)`
`IsDistinct`	Nur verschiedene Werte	`g.Select(...).Distinct()`
`Predicate`	Filter vor dem Aggregat	`g.Where(o => o.Id > 10)`
`Orderings`	Reihenfolge (für reihenfolgeabhängige Aggregate)	`g.OrderBy(...)`

Das Beispiel ergänzt `EF.Functions.Every(values)` für PostgreSQLs `bool_and`:

`Query/EveryAggregateTranslator.cs (Auszug)`

```
public virtual SqlExpression? Translate(MethodInfo method, EnumerableExpression source,
    IReadOnlyList<SqlExpression> arguments,
    IDiagnosticsLogger<DbLoggerCategory.Query> logger)
{
    if (method != EveryMethod || source.Selector is not SqlExpression selector)
        return null;

    // Npgsqls Factory setzt DISTINCT, FILTER (WHERE ...) und ORDER BY im Aggregat um
    if (_sql is not NpgsqlSqlExpressionFactory npgsql)
        return null;

    var boolean = _typeMappingSource.FindMapping(typeof(bool))!;

    // bool_and liefert für eine leere Menge NULL, Enumerable.All liefert true
    return _sql.Coalesce(
        npgsql.AggregateFunction("bool_and", [selector], source,
            nullable: true, argumentsPropagateNullability: [false],
            typeof(bool), boolean),
        _sql.Constant(true, boolean),
        boolean);
}
```

`Erzeugtes SQL für g.Where(o => o.Id > 10).Select(o => o.Total > 0)`

```
SELECT COALESCE(bool_and(o."Total" > 0.0) FILTER (WHERE o."Id" > 10), TRUE)
FROM "Orders" AS o
GROUP BY o."TenantId"
```

Semantik vor Syntax

Ein Aggregat-Translator muss die Semantik der .NET-Seite treffen, nicht nur die Syntax der Datenbank. Typische Fallen: leere Mengen (NULL statt 0, true oder Leerstring), NULL-Werte in der Eingabe und Prädikate. Npgsql selbst gleicht etwa `string.Join` mit `COALESCE` an `string_agg` an; das Beispiel folgt diesem Muster.

Datenbanken ohne `FILTER`-Klausel (etwa SQL Server) emulieren Prädikate mit `CASE WHEN prädikat THEN wert END`. Ein provider-neutraler Aggregat-Translator muss beides kennen; deshalb liegt dieser im PostgreSQL-Paket.

39 Fluent API auf Property-Ebene und Laufzeit-Annotations

Property-Builder

Bisher setzt das Plugin voraus, dass die Mandanten-Property `TenantId` heißt und `ITenantOwned` implementiert ist. Mit einer Fluent-API-Methode auf `PropertyBuilder` geht es ohne beides, etwa bei bestehenden Modellen mit einer Property `MandantId` oder einer expliziten Interface-Implementierung:

Anwendung

```
modelBuilder.Entity<Contract>().Property(c => c.Mandant).IsTenantProperty();
```

Metadata/TenancyPropertyBuilderExtensions.cs (Kernpaket)

```
public static PropertyBuilder IsTenantProperty(
    this PropertyBuilder builder, bool isTenant = true)
{
    ArgumentNullException.ThrowIfNull(builder);

    if (isTenant)
        builder.Metadata.SetAnnotation(TenancyAnnotationNames.IsTenantProperty, true);
    else
        builder.Metadata.RemoveAnnotation(TenancyAnnotationNames.IsTenantProperty);

    return builder;
}

public static PropertyBuilder<TProperty> IsTenantProperty<TProperty>(
    this PropertyBuilder<TProperty> builder, bool isTenant = true)
    => (PropertyBuilder<TProperty>)IsTenantProperty((PropertyBuilder)builder, isTenant);

public static IConventionPropertyBuilder? IsTenantProperty(
    this IConventionPropertyBuilder builder,
    bool isTenant = true,
    bool fromDataAnnotation = false)
    => builder.HasAnnotation(TenancyAnnotationNames.IsTenantProperty,
           isTenant ? true : null, fromDataAnnotation) is null
        ? null
        : builder;
```

Die Methode liegt im Kernpaket, weil sie nur Metadaten setzt. Die Conventions beider Provider-Pakete suchen die Mandanten-Property über dieselbe Kernfunktion:

Metadata/TenancyEntityTypeExtensions.cs (Kernpaket, gekürzt)

```
public static IReadOnlyProperty? FindTenantPropertyCandidate(
    this IReadOnlyEntityType entityType)
{
    var marked = entityType.GetProperties()
        .Where(p => p[TenancyAnnotationNames.IsTenantProperty] is true)
        .ToList();
```

```
    if (marked.Count > 1)
        throw new InvalidOperationException("... mehrere Mandanten-Properties ...");
    if (marked.Count == 1)
        return marked[0];

    return typeof(ITenantOwned).IsAssignableFrom(entityType.ClrType)
        ? entityType.FindProperty(nameof(ITenantOwned.TenantId))
        : null;
}
```

Builder	Typische Plugin-Methoden	Hinweis
`PropertyBuilder` / `<T>`	Spaltenoptionen, Rollen von Properties	Hier: `IsTenantProperty`
`IndexBuilder` / `<T>`	Index-Methoden, Operator-Klassen, Filter	Annotation an `Metadata` (IMutableIndex)
`KeyBuilder`	Schlüsseloptionen (Clustering)	Wie oben
`ComplexTypePropertyBuilder`	Properties in Complex Types	Nur anbieten, wenn die Option dort Sinn ergibt (Kapitel 30)
`ModelBuilder`	Modellweite Optionen	Annotation am Modell

Laufzeit-Annotations

Der SaveChanges-Interceptor muss zur Laufzeit wissen, welche Entitäten isoliert sind und wie ihre Mandanten-Property heißt. Die Convention schreibt das in eine Annotation am Entity-Typ; der Interceptor liest sie aus dem *Laufzeitmodell*:

Convention und Kernpaket (Auszug)

```
// Convention (Design-Time-Modell)
entityType.Builder.HasAnnotation(TenancyAnnotationNames.TenantProperty, property.Name);

// Interceptor (Laufzeitmodell)
public static string? GetTenantPropertyName(this IReadOnlyEntityType entityType)
    => entityType.GetRootType()[TenancyAnnotationNames.TenantProperty] as string;
```

Zwei Fragen entscheiden, ob das funktioniert. Landet die Annotation im Laufzeitmodell? Ja: `RuntimeModelConvention` entfernt nur EFs eigene Design-Time-Annotations und übernimmt alle anderen. Landet sie im Compiled Model? Ja: Einfache Werte schreibt EF automatisch (Kapitel 21). Der eigene `ICSharpRuntimeAnnotationCodeGenerator` hat deshalb die umgekehrte Aufgabe: Er entfernt die RLS-Annotations, die nur Migrations brauchen:

Design/Internal/TenancyCSharpRuntimeAnnotationCodeGenerator.cs (gekürzt)

```
public class TenancyCSharpRuntimeAnnotationCodeGenerator
    : NpgsqlCSharpRuntimeAnnotationCodeGenerator
{
    // Konstruktor: (CSharpRuntimeAnnotationCodeGeneratorDependencies,
    //               RelationalCSharpRuntimeAnnotationCodeGeneratorDependencies) : base(...)

    public override void Generate(IEntityType entityType,
        CSharpRuntimeAnnotationCodeGeneratorParameters parameters)
    {
        if (!parameters.IsRuntime)
```

```
        {
            foreach (var key in parameters.Annotations.Keys
                         .Where(k => k.StartsWith(RlsAnnotationNames.Prefix,
                                                  StringComparison.Ordinal))
                         .ToList())
            {
                parameters.Annotations.Remove(key);
            }
        }

        base.Generate(entityType, parameters);
    }
}
```

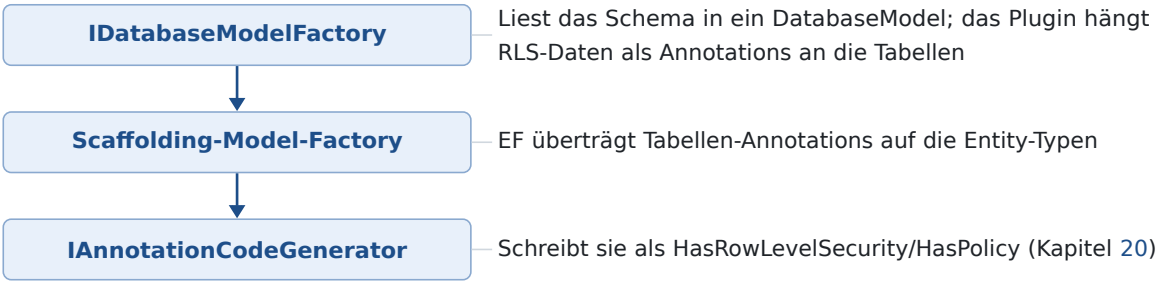
Wird das Compiled Model für NativeAOT erzeugt (`ForNativeAot`), schreibt EF zusätzlich das relationale Modell. Der Annotation-Provider hat die RLS-Annotations auf die Tabellen kopiert, sie tauchen dort also ein zweites Mal auf. Der Generator wendet deshalb denselben Filter in einer Überschreibung von `Generate(ITable, ...)` an.

Annotation	Design-Time	Laufzeit	Compiled Model
`Acme:Rls:*` (Policies)	ja (Migrations)	ja, ungenutzt	gefiltert
`Acme:Tenancy:TenantProperty`	ja	ja (Interceptor)	ja
`Acme:Tenancy:IsTenantProperty`	ja (Convention)	ja, ungenutzt	ja (klein)

> **Tipp**
>
> Für jede Annotation sollte klar sein, wer sie wann liest. Nur zur Design-Zeit gelesene Annotations gehören nicht ins Compiled Model; zur Laufzeit gelesene müssen hinein und brauchen einen Test gegen das generierte Modell. Der Compiled-Model-Test des Beispiels prüft beides.

40 Scaffolding in der Praxis

Beim Scaffolding (`dotnet ef dbcontext scaffold`) liest EF das Schema und erzeugt Entitäten und Context. Ein Plugin, das eigene Schema-Objekte verwaltet, sollte sie dabei erkennen. Der Weg führt über drei Stationen, von denen das Plugin zwei bereits kennt:



Scaffolding/TenancyDatabaseModelFactory.cs (gekürzt)

```
public class TenancyDatabaseModelFactory : NpgsqlDatabaseModelFactory
{
    public TenancyDatabaseModelFactory(
        IDiagnosticsLogger<DbLoggerCategory.Scaffolding> logger)
        : base(logger)
    {
    }

    public override DatabaseModel Create(
        DbConnection dbConnection, DatabaseModelFactoryOptions options)
    {
        var model = base.Create(dbConnection, options);
        // ... Verbindung öffnen, falls nötig ...
        ReadRowLevelSecurity(dbConnection, model);    // pg_class.relrowsecurity / ...force...
        ReadPolicies(dbConnection, model);            // pg_policies
        return model;
    }

    protected virtual void ReadPolicies(DbConnection connection, DatabaseModel model)
    {
        // SELECT schemaname, tablename, policyname, permissive, roles::text[], cmd,
        //        qual, with_check FROM pg_catalog.pg_policies
        // ...
        var policy = new RlsPolicy(
            Name: reader.GetString(2),
            Command: Enum.Parse<PolicyCommand>(reader.GetString(5), ignoreCase: true),
            Permissive: reader.GetString(3) == "PERMISSIVE",
            Roles: roles is ["public"] ? null : roles,    // "public" = Default ohne TO
            Using: reader.IsDBNull(6) ? null : reader.GetString(6),
            WithCheck: reader.IsDBNull(7) ? null : reader.GetString(7));

        table[RlsAnnotationNames.PolicyPrefix + policy.Name] = policy.Serialize();
    }
}
```

Registriert wird die Factory wie der Code-Generator in den Design-Time-Services, mit `RemoveAll` und `AddSingleton`. Das Datenformat ist dasselbe wie bei der Fluent API; deshalb funktionieren Snapshot, Migrations und Code-Generator ohne weitere Anpassung.

> **Normalisierte Ausdrücke**
>
> PostgreSQL speichert Policy-Ausdrücke in normalisierter Form, etwa `(("TenantId" = (NULLIF(current_setting('app.tenant_id'::text, true), ''::text))::uuid))`. Gescaffoldete Policies enthalten genau diesen Text. Das ist korrekt, sieht aber anders aus als die von der Convention erzeugte Form. Teams, die nach dem Scaffolding auf Migrations umsteigen, sollten die Policies einmal vereinheitlichen, sonst erzeugt die erste Migration ein DROP/CREATE mit gleichem Inhalt.

tests/...IntegrationTests/AdvancedTests.cs (Auszug)

```
[Fact]
public void Scaffolding_reads_rls_and_policies()
{
    using var ctx = ContextFor(null);
    var factory = new TenancyDatabaseModelFactory(
        ctx.GetService<IDiagnosticsLogger<DbLoggerCategory.Scaffolding>>());

    var model = factory.Create(
        _fixture.OwnerConnectionString, new DatabaseModelFactoryOptions());
    var orders = model.Tables.Single(t => t.Name == "Orders");

    Assert.Equal(true, orders[RlsAnnotationNames.Enabled]);
    Assert.Equal(true, orders[RlsAnnotationNames.Forced]);
}
```

41 Was in den Kern gehört

Ein zweiter Provider ist der Härtetest für die Paketarchitektur. Er zeigt, welche Teile wirklich provider-unabhängig sind. Die Faustregel: **Was keine SQL-Syntax und keinen Provider-Typ kennt, gehört in den Kern.** Für das Beispiel ergibt sich:

Baustein	Ort	Begründung
`TenantId`, Konverter, JSON-Reader/Writer	Kern	Reine .NET-Typen
`ITenantOwned`, `ITenantProvider`, Attribute	Kern	Für Entitäts-Assemblies ohne Provider-Abhängigkeit
`IsTenantProperty()`, Kandidatensuche	Kern	Nur Metadaten
Laufzeit-Annotation, `GetTenantPropertyName`	Kern	Vom provider-neutralen Interceptor gelesen
SaveChanges-Interceptor	Kern	Arbeitet nur mit dem Change Tracker
`TenantSessionInterceptor` (abstrakt)	Kern	Ablauf gleich, nur der SQL-Befehl unterscheidet sich
Evaluatable-Filter mit Attribut	Kern	Erkennt Funktionsklassen aller Pakete
Value Generation, Logging, `AddAcmeTenancy`	Kern	Ohne SQL-Bezug
`Database.ApplyTenantSession()`	Kern	Nutzt den Interceptor des Providers
Type Mapping, Translators, Migrations, RLS-Conventions	Provider	Store-Typen, SQL-Syntax, Schema-Objekte
Eigene SQL-Knoten, Aggregate	Provider	Abhängig von Syntax und Provider-Factory

> **Namenskonflikte zwischen Provider-Paketen**
>
> Zwei Provider-Pakete, die beide eine Extension-Method `UseAcmeTenancy` auf `DbContextOptionsBuilder` definieren, sind nicht gleichzeitig referenzierbar: Der Aufruf wäre mehrdeutig. Das PostgreSQL-Paket nutzt den Builder von EF, das SQL-Server-Paket den Builder des Providers (`UseSqlServer(cs, x => x.UseAcmeTenancy())`). Wer mehrere Provider-Pakete plant, sollte von Anfang an konsequent den Provider-Builder verwenden, wie das NetTopologySuite-Plugin.

42 Ein zweiter Provider: SQL Server

Das Paket `Acme.EntityFrameworkCore.Tenancy.SqlServer` setzt dieselbe Idee mit den Mitteln von SQL Server um. Die Unterschiede zeigen, was ein Plugin pro Datenbank tatsächlich neu lösen muss:

Aspekt	PostgreSQL	SQL Server
Mandant in der Session	`set_config(name, wert, false)`	`sp_set_session_context`
`CurrentTenant()` in SQL	`nullif(current_setting(...), '')::uuid`	`CAST(SESSION_CONTEXT(N'…') AS uniqueidentifier)`
Spaltentyp	`uuid`	`uniqueidentifier`
RLS-Mechanismus	`CREATE POLICY` pro Tabelle	Predicate-Funktion + `CREATE SECURITY POLICY`
Lesen / Schreiben	`USING` / `WITH CHECK`	`FILTER` / `BLOCK ... AFTER INSERT/UPDATE`
Tabelleneigentümer	Nur mit `FORCE` gebunden	Kein FORCE; Prädikate sind Teil der Abfrage
`DROP TABLE`	Policies verschwinden mit	Policy muss vorher weg (SCHEMABINDING)
DDL in Batches	Transaktional, beliebig kombinierbar	`CREATE FUNCTION/SCHEMA` allein im Batch: `EXEC(N'...')`

Aktivierung über den Provider-Builder

Anwendung und Infrastructure/SqlServerTenancyOptionsBuilderExtensions.cs (gekürzt)

```
services.AddAcmeTenancy<HttpTenantProvider>();                    // Kernpaket
services.AddDbContext<ShopContext>((sp, o) => o
    .UseSqlServer(connectionString, sql => sql.UseAcmeTenancy(sp)));

public static SqlServerDbContextOptionsBuilder UseAcmeTenancy(
    this SqlServerDbContextOptionsBuilder sqlServerOptionsBuilder,
    string sessionKey = "TenantId",
    string policySchema = "acme")
{
    // Vom Provider-Builder zum eigentlichen DbContextOptionsBuilder
    var optionsBuilder =
        ((IRelationalDbContextOptionsBuilderInfrastructure)sqlServerOptionsBuilder)
        .OptionsBuilder;

    var extension = optionsBuilder.Options.FindExtension<SqlServerTenancyOptionsExtension>()
                    ?? new SqlServerTenancyOptionsExtension();
    // ... Settings validieren und setzen ...
    ((IDbContextOptionsBuilderInfrastructure)optionsBuilder)
        .AddOrUpdateExtension(extension.WithSettings(settings));

    return sqlServerOptionsBuilder;
}
```

Migrations: Security Policy statt Policy-Ausdruck

Die Convention vermerkt für SQL Server nur die Mandantenspalte; das SQL entsteht im Generator. Die Predicate-Funktion ist für alle Tabellen dieselbe und wird idempotent angelegt:

Migrations/Internal/SqlServerTenancyMigrationsSqlGenerator.cs (Auszug)

```
protected virtual void CreatePolicy(string table, string? schema, string column,
    MigrationCommandListBuilder builder)
{
    var function = Delimit(PredicateFunctionName, Settings.PolicySchema);
    var target = Delimit(table, schema);
    var predicate = $"{function}({Delimit(column)})";

    // CREATE SCHEMA/FUNCTION müssen allein im Batch stehen -> EXEC(N'...')
    Statement(builder, $"IF SCHEMA_ID({Literal(Settings.PolicySchema)}) IS NULL "
        + $"EXEC({Literal($"CREATE SCHEMA {Delimit(Settings.PolicySchema)}")})");

    var functionSql =
        $"CREATE FUNCTION {function}(@TenantId uniqueidentifier) "
        + "RETURNS TABLE WITH SCHEMABINDING AS RETURN SELECT 1 AS [Allowed] "
        + $"WHERE @TenantId = CAST(SESSION_CONTEXT({Literal(Settings.SessionKey)}) "
        + "AS uniqueidentifier)";
    Statement(builder, $"IF OBJECT_ID({Literal(function)}, N'IF') IS NULL "
        + $"EXEC({Literal(functionSql)})");

    Statement(builder,
        $"CREATE SECURITY POLICY {PolicyName(table)} "
        + $"ADD FILTER PREDICATE {predicate} ON {target}, "
        + $"ADD BLOCK PREDICATE {predicate} ON {target} AFTER INSERT, "
        + $"ADD BLOCK PREDICATE {predicate} ON {target} AFTER UPDATE "
        + "WITH (STATE = ON)");
}

protected override void Generate(DropTableOperation operation, IModel? model,
    MigrationCommandListBuilder builder, bool terminate = true)
{
    DropPolicy(operation.Name, builder);       // SCHEMABINDING: zuerst die Policy
    base.Generate(operation, model, builder, terminate);
}
```

`SCHEMABINDING` akzeptiert nur zweiteilige Namen. Eine Tabelle ohne explizites Schema qualifizieren die Aufrufer deshalb mit dem Default-Schema des Modells oder, falls keines gesetzt ist, mit `dbo`; sonst lehnt SQL Server die Security Policy ab.

> **Verschachteltes SQL**
>
> Das `EXEC(N'...')` enthält SQL in einem String-Literal, das wiederum Literale enthält. Die Verschachtelung entsteht ausschließlich über `GenerateSqlLiteral` des String-Mappings, das Anführungszeichen korrekt verdoppelt; Bezeichner laufen über `DelimitIdentifier`. Session-Schlüssel und Schema werden zusätzlich beim Konfigurieren validiert.

Laufzeit: derselbe Ablauf, anderer Befehl

Runtime/SqlServerTenantSessionInterceptor.cs

```
public class SqlServerTenantSessionInterceptor : TenantSessionInterceptor
{
    public SqlServerTenantSessionInterceptor(ITenantProvider tenants)
        : base(tenants)
    {
    }

    protected override DbCommand CreateSessionCommand(
        DbConnection connection, DbContext? context, TenantId? tenant)
    {
        var key = context?.GetService<ISqlServerTenancySingletonOptions>()
                      .Settings.SessionKey
                  ?? SqlServerTenancySettings.Default.SessionKey;

        var command = connection.CreateCommand();
        command.CommandText = "EXEC sp_set_session_context @key = @key, @value = @value";
        AddParameter(command, "@key", key);
        AddParameter(command, "@value", tenant?.Value);    // NULL: keine Zeilen sichtbar
        return command;
    }
}
```

Bewusst nicht portiert

Das SQL-Server-Paket ist ein vollständiger, aber schlankerer Provider. Fluent-API-Policies, der optionale Query-Filter, eigene SQL-Knoten, Aggregate, Snapshot-Code-Generator und Scaffolding fehlen dort. Das ist eine Produktentscheidung, kein technisches Hindernis: Jedes dieser Kapitel lässt sich mit denselben Mustern auf SQL Server übertragen. Die Liste gehört in die Dokumentation des Pakets, damit Nutzer wissen, was sie erwarten können.

tests/...SqlServer.Tests/SqlServerTests.cs (Auszug)

```
[Fact]
public async Task Block_predicate_rejects_rows_of_other_tenant()
{
    await using var db = ContextFor(TenantId.New());

    var ex = await Assert.ThrowsAsync<SqlException>(() => db.Database.ExecuteSqlAsync(
        $"INSERT INTO [Orders] ([TenantId], [Total]) VALUES ({Guid.NewGuid()}, 1)"));

    Assert.Equal(33504, ex.Number);      // Verstoß gegen BLOCK PREDICATE
}
```

Die Tests laufen wie bei PostgreSQL in zwei Ebenen: ohne Datenbank für Modell, Abfrage- und Migrations-SQL und mit Testcontainers gegen einen echten SQL Server. Dort verbindet sich die Anwendung mit einem eigenen Login statt als `sa`, um die Produktionssituation nachzustellen.

Teil X

Anhänge

Nachschlagen: Erweiterungspunkte mit Lebensdauern, Dateien des Beispiels, Begriffe und ein Debugging-Werkzeugkasten.

- Anhang A: Erweiterungspunkte und Lebensdauern
- Anhang B: Dateien des Beispiel-Plugins
- Anhang C: Glossar
- Anhang D: Debugging-Werkzeugkasten
- Lizenz

A Erweiterungspunkte und Lebensdauern

Lebensdauern und Kardinalität laut den Service-Tabellen von EF Core 10 (`EntityFrameworkServicesBuilder`, `EntityFrameworkRelationalServicesBuilder`). „Mehrfach“ heißt: Plugin-Interface, alle Registrierungen werden verwendet.

Dienst	Lebensdauer	Art	Kap.
`IConventionSetPlugin`	Scoped	mehrfach	8
`ISingletonOptions`	Singleton	mehrfach	5
`ITypeMappingSourcePlugin`	Singleton	mehrfach	10
`IRelationalTypeMappingSourcePlugin`	Singleton	mehrfach	10
`IMethodCallTranslatorPlugin`	Scoped	mehrfach	12
`IMemberTranslatorPlugin`	Scoped	mehrfach	12
`IAggregateMethodCallTranslatorPlugin`	Scoped	mehrfach	38
`IEvaluatableExpressionFilterPlugin`	Singleton	mehrfach	13
`IInterceptor` (injiziert)	Scoped	mehrfach	15
`ISingletonInterceptor`	Singleton	mehrfach	15
`IRelationalAnnotationProvider`	Singleton	einzeln	18
`IMigrationsSqlGenerator`	Scoped	einzeln	18
`IMigrationsModelDiffer`	Scoped	einzeln	19
`IModelValidator`	Singleton	einzeln	9
`IModelCustomizer`	Singleton	einzeln	–
`IModelCacheKeyFactory`	Singleton	einzeln	31
`ISqlGenerationHelper`	Singleton	einzeln	–
`IRelationalTypeMappingSource`	Singleton	einzeln	10
`ISqlExpressionFactory`	Scoped	einzeln	12
`IQuerySqlGeneratorFactory`	Singleton	einzeln	14, 37
`IModificationCommandFactory`	Singleton	einzeln	29
`ICommandBatchPreparer`	Scoped	einzeln	29
`IModificationCommandBatchFactory`	Scoped	einzeln	29
`IUpdateSqlGenerator`	Singleton	einzeln	29
`IRelationalParameterBasedSqlProcessorFactory`	Scoped	einzeln	37

Design-Time-Dienste (über IDesignTimeServices)

Dienst	Zweck
`IAnnotationCodeGenerator`	Annotations als Fluent API in Snapshot und Scaffolding
`ICSharpRuntimeAnnotationCodeGenerator`	Annotations im Compiled Model filtern oder komplexe Werte übersetzen (Kapitel 39)
`ICSharpMigrationOperationGenerator`	Eigene Operationen als C# in Migrationen
`IDatabaseModelFactory`	Schema lesen für Scaffolding (Kapitel 40)
`IProviderConfigurationCodeGenerator`	UseXyz-Aufruf im gescaffoldeten Context

B Dateien des Beispiel-Plugins

Datei	Inhalt	Kap.
Kernpaket		
`TenantId.cs`, `TenantIdConverter.cs`	Stark typisierte ID, Konverter	10
`TenantIdJsonReaderWriter.cs`	JSON-Unterstützung	10
`ITenantOwned.cs`, `ITenantProvider.cs`	Marker, Mandantenquelle	3
`NoTenantIsolationAttribute.cs`	Data Annotation	8
`TenancyDbFunctionsExtensions.cs`	`EF.Functions.CurrentTenant()`	12
`Diagnostics/*`	Event-IDs, Definitionen, Logger-Extensions	16
`Interceptors/TenantSaveChanges Interceptor.cs`	SaveChanges-Logik	15
`Interceptors/TenantSessionInterceptor.cs`	Abstrakte Basis für Session-Befehle	15, 41
`TenancyDatabaseFacadeExtensions.cs`	`Database.ApplyTenantSession()`	15
`Metadata/*` (Kern)	Laufzeit-Annotation, IsTenantProperty, Kandidatensuche	39
`Query/TenancyEvaluatableExpressionFilter Plugin.cs`	Filter für alle Pakete	13, 41
`DependencyInjection/*`	AddAcmeTenancy	6, 41
`ValueGeneration/TenantIdValueGenerator. cs`, `…Factory.cs`	Wertgenerierung	28
`Conventions/TenantIdKeyValueGeneration Convention.cs`	Generator für Schlüssel	28
PostgreSQL-Paket		
`Infrastructure/TenancyOptionsExtension. cs`	Extension, Info, ApplyServices	4, 5
`Infrastructure/TenancySettings.cs`	Einstellungen als Record	4
`Infrastructure/ITenancySingletonOptions. cs`	Singleton-Options	5
`Infrastructure/TenancyOptionsBuilder.cs`	Options-Builder	6
`Infrastructure/Tenancy*Extensions.cs`	UseAcmeTenancy, AddAcmeTenancy	6
`Infrastructure/TenancyIdentifiers.cs`	Validierung von Bezeichnern	8, 19
`Storage/*`	Type Mapping und Plugin	10
`Query/*`	Translators, Evaluatable-Filter	12, 13
`Metadata/*`	Annotations, RlsPolicy, PolicyCommand, Annotation-Provider	7, 18
`Builders/*`	Fluent API, PolicyBuilder	7
`Conventions/*`	Isolation, Attribut, Validierung, Plugin	8, 9
`Migrations/*`	SQL-Generator, eigene Operationen	18, 19
`Design/*`, `Properties/AssemblyInfo.cs`	Code-Generator, Design-Time	20
`Runtime/TenantConnectionInterceptor.cs`	Session-Setting setzen	15

Datei	Inhalt	Kap.
`Query/Expressions/*`, `Query/DistinctFromTranslator.cs`	Eigener SQL-Knoten	37
`Query/Internal/*`	SQL-Generator, Nullability-Prozessor, Factories	37
`Query/EveryAggregateTranslator.cs`	Aggregat-Translator	38
`Design/Internal/TenancyCSharpRuntime…Generator.cs`	Compiled-Model-Filter	39
`Scaffolding/*`	RLS aus der Datenbank lesen	40
SQL-Server-Paket		
`Infrastructure/*`	Extension, Provider-Builder-Aktivierung	42
`Storage/*`, `Query/*`	uniqueidentifier, SESSION_CONTEXT	42
`Conventions/*`, `Metadata/*`	Isolation, Spalten-Annotation	42
`Migrations/*`	Predicate-Funktion, Security Policy	42
`Runtime/*`	sp_set_session_context	42
Weitere		
`samples/ShopSample`	Konsolen-Beispiel mit Design-Time-Factory	3, 32
`samples/ShopApi`	ASP.NET-Core-Beispiel (Minimal API)	35
`.github/workflows/ci.yml`	Build, Tests, Handbuch, Preview-Build	36
`PublicAPI.*.txt`	Public-API-Analyzer	36
`docs/handbook/*`	Quellen dieses Buchs (Deutsch und Englisch)	–
`tests/...Tests`	Unit-Tests ohne Datenbank	23
`tests/.../MinimalPluginTests.cs`	Das Mini-Plugin	2
`tests/.../CompositionTests.cs`	Decorator-Beispiel (Audit-Plugin)	5
`tests/.../DebuggingTests.cs`	Debugging-Techniken	Anh. D
`tests/.../SchemaPerTenantTests.cs`	Modell-Cache-Schlüssel pro Schema	31
`benchmarks/...`	BenchmarkDotNet-Projekt für die SaveChanges-Logik	34
`tests/...IntegrationTests`	Testcontainers-Tests gegen PostgreSQL	15, 23, 27, 33, 40
`tests/...SqlServer.Tests`	Unit- und Testcontainers-Tests für SQL Server	42

C Glossar

Begriff	Erklärung
Annotation	Schlüssel-Wert-Metadatum an einem Modellelement.
ConfigurationSource	Herkunft eines Metadatums (Explicit, DataAnnotation, Convention); bestimmt, wer wen überschreiben darf.
Convention	Regel, die das Modell beim Bau automatisch anpasst.
Decorator	Dienst, der die zuvor registrierte Implementierung umhüllt und an sie delegiert; verträgt sich mit anderen Plugins (Kapitel 5).
Design-Time-Modell	Vollständiges Modell mit allen Annotations; Grundlage für Migrations. Gegenstück: schlankes Laufzeitmodell.
Differ	`IMigrationsModelDiffer`; vergleicht zwei relationale Modelle.
Funcletizer	Erste Query-Stufe; wertet Ausdrücke ohne Datenbezug in .NET aus.
Nullability-Prozessor	Query-Stufe, die NULL-Semantik auswertet und Vergleiche optimiert; muss eigene SQL-Knoten kennen.
SqlExpression	Knoten im SQL-Ausdrucksbaum; eigene Knoten für Syntax, die keine Funktion ist.
Laufzeit-Annotation	Annotation, die zur Laufzeit gelesen wird und deshalb im Laufzeit- und Compiled Model erhalten bleiben muss.
Interner Service Provider	EFs eigener DI-Container pro Konfiguration.
Pubternal	Öffentlich sichtbare, aber interne API ohne Kompatibilitätsgarantie (EF1001).
Relationales Modell	Tabellen, Spalten, Indizes, abgeleitet aus dem EF-Modell.
RLS	Row Level Security: Zeilenfilter, die PostgreSQL selbst erzwingt.
Snapshot	Generierter C#-Code mit dem Modellstand der letzten Migration.
Type Mapping	Verbindung zwischen CLR-Typ und Datenbanktyp inklusive Konvertierung, Vergleich und Literalen.

D Debugging-Werkzeugkasten

Wenn sich ein Plugin nicht wie erwartet verhält, lautet die Frage fast immer eine von fünf. Jede der folgenden Techniken wird in `DebuggingTests` geprüft.

Frage	Technik
Ist meine Annotation im Modell?	`ToDebugString(MetadataDebugStringOptions.LongDefault)` auf dem Design-Time-Modell
Welche Implementierung eines Einzeldienstes hat gewonnen?	`context.GetService<IMigrationsSqlGenerator>().GetType()`
Ist meine Convention registriert?	`GetService<IConventionSetBuilder>().CreateConventionSet()`, dann z. B. `ModelFinalizingConventions`
Wie sieht die Abfrage vor und nach der Übersetzung aus?	`query.Expression.Print()` und `query.ToQueryString()`
Protokolliert mein Plugin?	`LogTo(lines.Add, [eventId], LogLevel.Debug)`, beschränkt auf die Event-IDs des Plugins

Debugging-Snippets (geprüft in tests/.../DebuggingTests.cs)

```
using var ctx = CreateContext();

// 1. Modell: Annotations, Ergebnisse von Conventions, Type Mappings
var model = ctx.GetService<IDesignTimeModel>().Model;
Console.WriteLine(model.ToDebugString(MetadataDebugStringOptions.LongDefault));

// 2. Dienste: welcher Ersatz hat gewonnen?
Console.WriteLine(ctx.GetService<IMigrationsSqlGenerator>().GetType());

// 3. Conventions: ist meine in der Liste?
var conventions = ctx.GetService<IConventionSetBuilder>().CreateConventionSet();
Console.WriteLine(string.Join("\n", conventions.ModelFinalizingConventions));

// 4. Abfragen: LINQ-Baum vor, SQL nach der Übersetzung
var query = ctx.Orders.Where(o => o.TenantId == EF.Functions.CurrentTenant());
Console.WriteLine(query.Expression.Print());
Console.WriteLine(query.ToQueryString());
```

Weitere Werkzeuge

Werkzeug	Einsatz
Debugger mit Source Link, „Just My Code“ aus	In EF Core und Npgsql hineinsteppen; beide liefern Symbole und Source Link mit. Ein Breakpoint in der eigenen Convention zeigt, wann und wie oft sie läuft.
`DebugView` im Debugger	Modell, Entitätstypen, Query-Objekte und der Change Tracker zeigen dieselben Texte wie `ToDebugString`.
`dotnet ef ... --verbose`	Zeigt, welche Assemblies und Design-Time-Dienste die Tools laden; die erste Anlaufstelle, wenn ein Design-Time-Dienst ignoriert zu werden scheint.

Werkzeug	Einsatz
`EnableSensitiveDataLogging`, `EnableDetailedErrors`	Parameterwerte und ausführliche Fehler im Log; nur für die Entwicklung.
`ManyServiceProviders CreatedWarning`	EF warnt, wenn zu viele interne Provider entstehen: fast immer ein Hash-Fehler (Kapitel 4).
Der Quellcode von EF Core auf GitHub	Die maßgebliche Dokumentation interner APIs; die Ordnerstruktur entspricht den Namespaces.

> **Im Test reproduzieren, nicht in der App**
>
> Fast jedes Plugin-Problem lässt sich ohne Datenbank reproduzieren: einen Context mit dem Plugin bauen und das Modell, die aufgelösten Dienste oder `ToQueryString()` ansehen. Ein solcher Test iteriert schneller als die Anwendung und bleibt danach als Regressionstest erhalten.

Lizenz

Dieses Handbuch und der Beispielcode im Repository stehen unter der MIT-Lizenz. Sie dürfen beides nutzen, kopieren, verändern und weitergeben, auch kommerziell, sofern der folgende Copyright- und Lizenzhinweis erhalten bleibt. Maßgeblich ist der englische Originalwortlaut:

Copyright (c) 2026 guenni81

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Die im PDF eingebetteten DejaVu-Schriften stehen unter einer eigenen freien Lizenz (Bitstream-Vera-Schriftlizenz; die DejaVu-Änderungen sind gemeinfrei); siehe `docs/handbook/fonts/LICENSE-DejaVu.txt` im Repository.

Genannte Produkt- und Firmennamen sind Marken ihrer jeweiligen Inhaber, etwa Entity Framework, .NET und SQL Server (Microsoft) sowie PostgreSQL (PostgreSQL Community Association). Sie werden nur beschreibend verwendet. Dieses Handbuch ist ein unabhängiges Projekt und steht in keiner Verbindung zu Microsoft, der PostgreSQL Global Development Group oder dem Npgsql-Projekt und wird von ihnen weder unterstützt noch empfohlen.